neURROn Design Manifest

C. Niederberger
and
the neURROn group

March 15, 2009

Chapter 1

Introduction and History

The original neUROn was coded in C in 1992 when I was at Baylor College of Medicine. It was named "neural computational for UROlogical numericals," a somewhat mercurial reference to its application domain, the medical field of Urology. In actuality, neUROn is a general purpose neural computational modeling environment. neUROn has been, and will continue to be used to model many problems beyond the Urological domain.

Between 1992 and 2000, neUROn evolved in fundamental ways. In 1996, Young Hong migrated neUROn to C++, and the resulting environment was named "neUROn++". Many have coded these fundamental changes, including Vinod Kutty, Yuan Qin, Luke Cho, Joe Jovero, and Kai Liu. In general, beyond migrating neUROn to object-oriented programming, these fundamental changes have taken 2 forms: programming novel neural computational training algorithms and programming statistical analysis of training behavior. Throughout this project, Richard Golden Ph.D. at the University of Texas at Dallas has suggested algorithms which the team has implemented.

The impetus for programming novel training algorithms should be obvious. Neural computational algorithms are iterative, and problems modeled by the neural computational approach can easily become intractable, even with modern computational hardware. A single training session may require several hours, and multiple training sessions may require days or even weeks. There are 3 basic ways in which training sessions may become shorter:

1. Faster hardware may be used. This is clearly the way in which the programmer has the least control.

2. More efficient code may be written. The importance of this cannot be stessed enough, and it will serve as a basic design principle for the neUROn project. In the original neUROn code, for example, arrays were expressed entirely as pointer operations. It was not until the compiler benchmarks demonstrated similar outcomes for array constructs that arrays were used in the code.

3. Faster training algorithms may be encoded. A substantial part of the neUROn project is devoted to investigating novel training algorithms, and this has paid handsomely. Investigated algorithms have been demonstrated to speed training by 2 orders of magnitude or more.

The impetus for programming statistical analysis of network behavior was less obvious, but perhaps more significant. When we began reporting results to the Urological community, we were surprised to find a universal response was, "so the network accurately predicts X, but what were the important variables?" In retrospect, this response is unsurprising. The majority of our audience, our potential users, were physicians or scientists involved in the medical field. It is not enough to accurately model an outcome for these users, it is critical to find the significant input features so that these features may be altered by the user in the clinical environment. For example, if it is determined that medication X is statistically significant in the model Y that predicts cancer recurrence, the user will put that information to use in the clinical domain.

Beginning in 2000, neUROn was entirely redesigned using this document as a specification, and taking full advantage of the object-oriented nature of C++. Whereas the migration from 1996 to 2000 was an incorporation of C code into the C++ paradigm, the hybrid code which resulted was brittle, with previous features breaking as new features were added. In 2000, the code was redesigned from the "ground up". This document is intended to be a description of neUROn programming, past, present, and future. As such, the document is a plan that is expected to evolve as neUROn is redesigned and recoded in the future.

Chapter 2

Methodology

The following chapter outlines algorithms currently implemented in neUROn, as well as many intended to be implemented in the future. As usual, we are indebted to Dr. Golden. The terminology ANN is used to refer to "Artificial Neural Network".

1. **Error Gradient Calculation Algorithm.** The gradient of the objective function is computed by calling a *forward propagation algorithm* followed by a *backward error propagation algorithm.*

2. **Parameter Estimation Algorithm.** The gradient computed by the *Error Gradient Calculation Algorithm* is then used by the parameter estimation algorithm for the purposes of learning.

3. **Data Analysis Algorithm.** If this algorithm decides that reliable statistical inference is possible, this algorithm indicates which input nodes in the network are making statistically significant contributions to the network's performance. Otherwise, this algorithm makes recommendations to the user regarding possible manipulations designed to increase statistical inference reliability. Future work will involve computing confidence intervals on the predictions and relaxing the "Goodness-of-Fit" assumption which is assumed throughout this document.

Note that math calculations for the following algorithms such as matrix inverse, eigenvalues, "line-search", cumulative distribution functions for the chi-square random variable for statistical test result reporting can be done using the routines in *Numerical Recipes in C.* In fact, *Numerical Recipes* has a conjugate gradient routine which could be used to develop a learning algorithm for the ANN system as well.

2.1 Error Gradient Calculation Algorithm

2.1.1 Data Records

Note that the data is assumed to have the following format: $(\mathbf{s}_k, \mathbf{t}_k)$ where $\mathbf{s}_k$ is the d-dimensional input vector for the kth data record and d-dimensional vector $\mathbf{t}_k$ is a list of "targets" for the d output units. If the training data consists of N data records, then we have $k = 1, \ldots, N$.

2.1.2 Objective Function

The *error gradient* is defined as the gradient (i.e., vector first derivative) of a sufficiently smooth *error function* with respect to the "weights" (i.e., parameters) of the ANN. In this document version, only feedforward m-layer ANN systems will be considered where the number of "layers" is defined by "connections". Thus, a 1-layer ANN is essentially a linear feedforward ANN with one set of input units and one set of output units and no hidden units. Let $\mathbf{y}$ denote a vector containing all weights of an m-layer ANN. The activation levels of the units in the output layer of an m-layer ANN generated in response to training stimulus k are denoted by the *activation vector*, $\mathbf{a}_k^{(m)}$. Note that $\mathbf{a}_k^{(m)}$ is a function of both $\mathbf{y}$ and the input vector $\mathbf{s}_k$.

The *error function*, E_k, for the kth training pattern is defined by the formula:

$$E_k(\mathbf{y}) = e(t^k, \mathbf{a}_k^{(m)}) + \frac{1}{2\sigma_w^2} \sum_{i=1}^{m} |\mathbf{y}|^2. \tag{2.1}$$

The second term on the right hand side of (2.1) is required to guarantee reliable statistical inference. It basically "stabilizes" the learning process by preventing the weights in the network from becoming too large or too small. In practice, a connection weight is expected to lie between $-1.96\sigma_w$ and $+1.96\sigma_w$ with probability equal to about 95%. Adjustment of this parameter can "change results" of the statistical analysis so it is best to fix σ_w to be some constant number and try not to change its value. A reasonable choice for σ_w would be $\sigma_w = 100$. This basically constrains the "learning process" to find connection strengths for the ANN such that the connection strength weight ranges between -200 and $+200$. The choice of σ_w should be reported in conjunction with all statistical analysis results. Note that as $\sigma_w \to \infty$, the effect of the summation term becomes negligible.

The first term on the right hand side of (2.1) is called the *prediction error function* term. The prediction error function e must have continuous second partial derivatives with respect to the weights. For example, the classic choice of the prediction error function e is to choose

$$e(\mathbf{t}^k, \mathbf{a}_k^{(m)}) = (1/2)|\mathbf{t}^k - \mathbf{a}_k^{(m)}|^2 \tag{2.2}$$

to be the *least squares prediction error function*. This is a good choice when the target vector $\mathbf{t}^k$ is a continuous (as opposed to a discrete) variable.

When the target vector consists of a set of binary variables taking on the values of zero or one, then a good choice is the *cross-entropy prediction error function* defined by the formula:

$$e(\mathbf{t}^k, \mathbf{a}^{(m)}) = -\mathbf{t}^k .* \mathbf{log}[\mathbf{a}_k^{(m)}] - (\mathbf{1} - \mathbf{t}^k) .* \mathbf{log}[1 - \mathbf{a}_k^{(m)}]. \tag{2.3}$$

Note that the notation $.*$ denotes element by element matrix multiplication, the notation $\mathbf{1}$ denotes a column vector of ones, and the notation **log** denotes a vector-valued function defined such that the $\mathbf{y} = \mathbf{log}(\mathbf{x})$ whenever the ith element of the vector $\mathbf{y}$, y_i is equal to the natural log (log base e) of the ith element of the vector $\mathbf{x}$. When the cross-entropy prediction error function is used, the transfer function for all output units should be the sigmoidal logistic transfer function which is defined formally in the next section of this report.

2.1.3 Forward Propagation of Activations

The response of the artificial neuron units in layer j is computed from the activation levels of the artificial neuron units in layer $j-1$ using the *forward propagation algorithm* formulas. These formulas are defined by an update formula for computing the *netinput*, $\mathbf{u}_k^{(j)}$, to a unit in layer j for the kth training stimulus:

$$\mathbf{u}_k^{(j)} = \mathbf{W}^{(j)}\mathbf{a}_k^{(j)} + \mathbf{b}^{(j)} \tag{2.4}$$

and the *output activation level* for the kth training stimulus in layer $j+1$:

$$\mathbf{a}_k^{(j)} = \mathbf{f}^{(j)}(\mathbf{u}_k^{(j)}) \tag{2.5}$$

using the *layer transfer function*, $\mathbf{f}^{(j)}$. Note that $\mathbf{a}_k^{(0)} = \mathbf{s}_k$ by definition.

The layer transfer function, $\mathbf{f}^{(j)}$, is the chosen vector-valued transfer function for layer j, $\mathbf{W}^{(j)}$ is the connection matrix from the units in layer $j-1$ to the units in layer j, and $\mathbf{b}^{(j)}$ is the bias vector for the units in layer j. The subscript k refers to the kth training stimulus. The function $\mathbf{f}^{(j)}$ must have continuous second partial derivatives.

For example, let $\mathbf{u} = [u_1, u_2, u_3, u_4, u_5]$. A function form for $\mathbf{f}^{(j)}$ might be:

$$\mathbf{f}^{(j)}(\mathbf{u}) = [u_1, u_2, \mathcal{S}(u_3), exp(u_4^2), 2\mathcal{S}(u_5) - 1] \tag{2.6}$$

where $\mathcal{S}(u_3) = 1/(1 + exp(-u_3))$. The layer transfer function in (2.6) illustrates a situation where the first two elements are *linear transfer functions*, the third element is a *logistic sigmoidal* or just *sigmoidal* function, the fourth element is an example of one type of *radial basis* transfer function, and the fifth element is an example of one type of hyperbolic tangent sigmoidal function (i.e., the asymptotes are -1 and +1 unlike the 0 and 1 asymptotes of the logistic sigmoid).

2.1.4 Backward Propagation of Errors

Let the vector $\delta_k^{(j)} = dE_k/d\mathbf{a}_k^{(j)}$ (i.e., the derivative of the error with respect to the activation pattern vector in layer j evaluated at the current set of weights

and the kth training vector). The vector $\delta_k^{(j)}$ is called the *error signal* for the jth layer of units whose activation levels are defined by the vector $\mathbf{u}^{(j)}$ (note this notation may slightly differ from the Rumelhart et al., 1986, paper). Let the matrix $\mathbf{Df}_k^{(j)}$ be the vector first derivative (i.e., the "jacobian") of the vector transfer function $\mathbf{f}_k^{(j)}$ and k refers to the kth training simulus.

Consider a m-layer feedforward network consisting of layers $1, \ldots, m$. The *backward error propagation algorithm* formula for propagating an error signal for the output units of layer j to the output units of layer $j-1$ (i.e., the input units of layer j) using the formula:

$$\delta_k^{(j-1)} = [\mathbf{W}^{(j)}]^T \mathbf{Df}_k^{(j)} \delta_k^{(j)} \tag{2.7}$$

for $j = 1, \ldots, m-1$. The error signal for the mth layer is the vector $\delta_k^{(m)} = de_k/d\mathbf{a}_k^{(m)}$ where e_k is defined as in (2.1).

For example, suppose $\mathbf{f}$ is defined such that $\mathbf{u} = \mathbf{f}(\mathbf{u})$ (i.e., a "linear transfer function"), then $\mathbf{Df}$ is the identity matrix. As another example, suppose that all output units are logistic sigmoidal units so that the activation level of the ith output unit is given by $\mathcal{S}(u_i)$ where u_i is the netinput to the ith output unit and $\mathcal{S}(u_i) = 1/(1 + exp(-u_i))$. Assume there are only three units in a layer. In this case, $\mathbf{f}$ will be a three-dimensional vector whose ith on-diagonal element is defined by $\mathcal{S}(u_i)$ and $\mathbf{Df}$ will be a three-dimensional matrix whose off-diagonal elements are zero and whose ith on-diagonal element is $\mathcal{S}(u_i)(1 - \mathcal{S}(u_i))$.

If the prediction error is a sum-squared error signal then

$$\delta_k^{(m)} = -(\mathbf{t}_k - \mathbf{a}^{(m)}) \tag{2.8}$$

If the prediction error is a cross-entropy error function, then

$$\delta_k^{(m)} = -(\mathbf{t}_k - \mathbf{a}^{(m)})./[\mathbf{a}^{(m)}. * (\mathbf{1} - \mathbf{a}^{(m)}] \tag{2.9}$$

where it is assumed that the m layer consists of logistic sigmoidal functions. Note the notation ./ refers to element by element division.

2.2 Parameter Estimation Formulas

2.2.1 Calculation of the gradient vector (and overall error).

For each data record ($k = 1, \ldots, N$), compute the following:

- *Step 1: Forward Activation Propagation.* Given stimulus $\mathbf{s}_k$, propagate activation levels forward layer by layer through the network using the forward propagation equations (equations (2.4) and (2.5)) in order to generate output activation vector $\mathbf{a}_k^{(m)}$ for the last layer (layer m).
- *Step 2: Compute data record error.* Compute the output error E_k using equation (2.1).

- *Step 3: Backward error Propagation.* Compute the output error signal $\delta_k^{(m)}$ for layer m and propagate this backwards so that an error signal is obtained for layers $m-1$ through layer 1 using equation (2.7).

- *Step 4: Compute and store data record gradient.* Let the column vector $\mathbf{f}_k^{(j-1)}$ be defined such that $\mathbf{f}_k^{(j-1)} = [\mathbf{a}_k^{(j-1)}\ ,\ 1]^T$.

$$\mathbf{G}_k^{(j)} = \delta_k^{(j)} \mathbf{D}\mathbf{f}_k^{(j)} [\mathbf{f}_k^{(j-1)}]^T + (1/\sigma_w^2)[\mathbf{W}\ ,\ \mathbf{b}] \tag{2.10}$$

  for $j = 1, \ldots, m$.

  Let the notation $STACK[\mathbf{G}_k^{(j)}]$ refer to a vector formed by "stacking" the rows of the matrix $\mathbf{G}_k^{(j)}$ one on top of another. Thus, if $\mathbf{G}_k^{(j)}$ has d rows, then the first d elements of $STACK[\mathbf{G}_k^{(j)}]$ will be the first row of $\mathbf{G}_k^{(j)}$, the second d elements of $STACK[\mathbf{G}_k^{(j)}]$ will be the second row of $\mathbf{G}_k^{(j)}$, and so on. It will also be convenient to have the function $UNSTACK$ which is the inverse of the function $STACK$. Now define the column vectors: $\mathbf{g}_k^{(j)} = STACK[\mathbf{G}_k^{(j)}]$ and $\mathbf{g}_k = [\mathbf{g}_k^{(1)}, \ldots, \mathbf{g}_k^{(m)}]$. The vector $\mathbf{g}_k$ is called the *data record gradient* for the kth data record in the training data and should be temporarily stored once it is computed.

2.2.2 Example learning algorithms.

Important indexing issues and definitions. Define $\mathbf{y}^{(j)} = STACK[\mathbf{W}^{(j)} \quad \mathbf{b}^{(j)}]$ and $\mathbf{y} = [\mathbf{y}^{(1)}, \ldots, \mathbf{y}^{(m)}]$. We will use the index t to denote the *iteration number* of the learning algorithm. *The index t is not the same as the index k!! The k index ranges over the set of N data records in the training set, while the t index identifies the learning algorithm iteration!!* Using this notation, $E_k(\mathbf{y}_t)$ will be used to refer to the error for the kth data record at iteration number t of the learning algorithm. Similarly, $\mathbf{g}_k(\mathbf{y}_t)$ refers to the gradient of the error for the kth data record at iteration number t of the learning algorithm. Similarly, the variable N **is NEVER** equal to the number of learning trials but is **ALWAYS** defined as the number of data records in the training set.

Initialization issues. In all of the following learning algorithms the choice for the initial value $\mathbf{y}_0$ is absolutely crucial. The elements of this vector should be chosen so that they are independent and identically distributed with mean zero and variance σ_0^2. The parameter σ_0^2 should be "highly visible" to the user. Learning then proceeds until the parameters of the network converge. If σ_0^2 is too small, then the learning process will be very long. If σ_0^2 is too large, then the learning process will rapidly converge to a crummy solution. It is best to err on the side of selecting σ_0^2 to be too small.

Classic on-line backprop. The *classic on-line* backpropagation learning algorithm is given by the formula:

$$\mathbf{y}_{t+1} = \mathbf{y}_t - \eta \mathbf{g}_k(\mathbf{y}_t) \tag{2.11}$$

where k (the particular training data record index) is chosen at random on iteration t of the learning algorithm. The parameter η is a strictly positive

number called the *learning rate* or *stepsize* for the learning algorithm (e.g., $\eta = 1/N$ is a good choice).

Stochastic approximation on-line backprop. An improved version of the *classic on-line* algorithm called *stochastic approximation on-line* is given by the formula:

$$\mathbf{y}_{t+1} = \mathbf{y}_t - \left(\frac{\eta}{t_0 + t}\right) \mathbf{g}_k(\mathbf{y}_t) \tag{2.12}$$

where t_0 is a positive integer whose magnitude is roughly equal to the number of records, N, in the training data set. The stepsize η is a positive number (e.g., $\eta = 1$ is a good choice).

Classic off-line ("batch") backprop. The *classic off-line* or "classic batch" backpropagation learning algorithm is given by the formula:

$$\mathbf{y}_{t+1} = \mathbf{y}_t - \eta \mathbf{g}(\mathbf{y}_t) \tag{2.13}$$

where the *average gradient* $\mathbf{g}$ is defined by the formula:

$$\mathbf{g} = (1/N) \sum_{k=1}^{N} \mathbf{g}_k. \tag{2.14}$$

A good choice for η for this algorithm would be $\eta = 1$ but as usual η must be a strictly positive real number.

Offline backprop with automatic stepsize selection. Define the *average error* E by the formula:

$$E = (1/N) \sum_{k=1}^{N} E_k. \tag{2.15}$$

An improved version of the *classic off-line* backpropagation learning algorithm is given by the formula:

$$\mathbf{y}_{t+1} = \mathbf{y}_t - \eta_t \mathbf{g}(\mathbf{y}_t) \tag{2.16}$$

where $\mathbf{g}$ is defined as in (2.14) and η_t is chosen such that quantity $E(\mathbf{y}_t - \eta \mathbf{g}(\mathbf{y}_t))$ is minimized with respect to η (treating $\mathbf{y}_t$ as a known constant) subject to the constraint that $0 \leq \eta_t \leq 1$. A "line search" routine from *Numerical Recipes* may be used to solve this one-dimensional optimization problem.

2.3 Data Analysis Algorithms

Conditions for Statistical Inference to be Valid.

The four assumptions that we make here are as follows. All four of these assumptions must be satisfied in order to guarantee reliable statistical inferences.

Log-likelihood Objective Function Assumption. First, it is assumed that the prediction error objective function e_k may be interpreted as the negative natural logarithm of the likelihood of a training data record given the network's

parameters. When the target $\mathbf{t}_k$ takes on a continuous range of values and the least-squares error function defined in (2.2) then this assumption is satisfied by assuming a multivariate conditional Gaussian distribution for the target vector given the input vector and network parameters. When the target consists of elements taking on the values of either zero or one and the output units are sigmoidal, then this assumption is satisfied by interpreting the activation level of an output unit as the probability that the target for that output unit will take on the value of one and using the cross-entropy function defined in (2.3). The best way to satisfy this assumption is to make sure that the user only gets to choose from a set of valid log-likelihood objective functions.

Goodness-of-Fit Assumption. Second, it is assumed that the predictions of the model with respect to the training data would be "perfect" if the model was exposed to a sufficiently large set of training data and was provided with a sufficiently large number of learning iterations. That is, we assume the model provides a good fit to the observed training data. More technically, it is assumed that the probabilistic modeling assumptions introduced in the "log-likelihood objective function assumption" are valid.

We will call the parameters which provide a good fit to the data $\mathbf{y}^*$. Note that in many real-world situations, this assumption will not be satisfied. For example, suppose we teach a linear ANN the "exclusive-or" problem. No matter how long we train the ANN, the ANN will never "fit the data".

To check this assumption, provide the user with plots of training and test set performance. Warn user that statistical inferences may not be reliable if the ANN is not a good data model.

Convergence Assumption. Third, it is assumed that the gradient vector, $\mathbf{g}(\mathbf{y}^*)$, is sufficiently small in magnitude. Typically, we check this assumption by seeing if the element in $\mathbf{g}_t$ with the largest absolute value is less than ϵ. The constant ϵ is typically about 0.0001. If this assumption is not satisfied, the user should be recommended to adjust the parameters of the learning procedure or alter the initial conditions (initialization of the ANN connection weights).

Convergence to a Strict Local Minimum Assumption. Fourth, it is required that this local uniqueness assumption be satisfied. Typically, we check this assumption by computing the matrix:

$$\mathbf{H}^* = (1/N) \sum_{k=1}^{N} \mathbf{g}_k(\mathbf{y}^*)[\mathbf{g}_k(\mathbf{y}^*)]^T. \tag{2.17}$$

All eigenvalues of $\mathbf{H}^*$ can be shown to be always non-negative (if you see any negative eigenvalues then there are numerical problems with the eigenvalue calculation routine). Compute the condition number of $\mathbf{H}^*$ which is defined as the largest eigenvalue of $\mathbf{H}^*$ divided by the smallest eigenvalue of $\mathbf{H}^*$. If the condition number is less than some constant K then we say the local uniqueness assumption is satisfied. A constant K with a value of 1000 is acceptable. Most statisticians use a constant K of value of 100. Usually, $K > 1000000$ is too large. If this assumption is not satisfied, the user should consider collecting more data, changing the network architecture, or decreasing the value of σ_w. Of

course the preferred course of action is collecting more data. Note that altering the value of σ_w is formally equivalent to changing the network architecture.

Wilks Likelihood Ratio Test

Wilk's Likelihood Ratio Test works as follows. Estimate the parameters of the network. Call that set of parameters $\mathbf{y}_{full}$. Now constrain r free parameters of the network equal to the value of zero and re-estimate the network parameters using $\mathbf{y}_{full}$ as the "initialization" for the learning algorithm. The resulting parameter estimates will be called $\mathbf{y}_{reduced}$. It is assumed that $\mathbf{y}_{full}$ satisfies all four assumptions and $\mathbf{y}_{reduced}$ satisfies all assumptions except for the Goodness-of-Fit assumption. It is assumed that $\mathbf{y}_{reduced}$ and $\mathbf{y}_{full}$ are sufficiently close to one another in the sense that under the null hypothesis that any difference between $\mathbf{y}_{reduced}$ and $\mathbf{y}_{full}$ is due to the fact that we only have a finite random sample.

Using equation (2.15), compute

$$G^2 = -2N[E(\mathbf{y}_{full}) - E(\mathbf{y}_{reduced})]. \tag{2.18}$$

If G^2 exceeds the critical value of a chi-square random variable with r degrees of freedom at the α significance level (r=number of parameters in full model minus number of parameters in reduced model), then we reject the null hypothesis at the α significance level that the r parameters could be set equal to zero without hurting the fit of the model to the data.

Another Version of Wilks Test called the Wald Test

A generalization of the Z-test can be used to answer the same questions as Wilks test. This statistical test is called the Wald Test. It has the advantage that you can do the statistical test with only one set of parameter estimates. First, you estimate the parameter vector which we will call $\mathbf{y}^*$. Now suppose you want to test the null hypothesis that r specific elements of $\mathbf{y}^*$ are equal to zero. We will denote this subvector by the r-dimensional vector $\mathbf{q}^*$. Obviously this is just as general as Wilks test because you can select the r elements to the connection strengths leaving a particular input unit (for example).

Now you define a *selection matrix* $\mathbf{S}$ which has r rows and h columns where h is the dimension of $\mathbf{y}^*$. You want to select $\mathbf{S}$ in just such a way so that $\mathbf{q}^* = \mathbf{S}\mathbf{y}^*$. This can always be done and it might make sense to have a set of "canned" selection matrix generation routines available so the user doesn't have to figure this one out.

For example, suppose $\mathbf{y}^* = [1, 2, 11, 4, 6]$ and we want to test the null hypothesis that the third and fourth weights are both equal to zero. Then we would construct a selection matrix $\mathbf{S}$ with two rows whose first row is [0, 0, 1, 0, 0] and whose second row is [0, 0, 0, 1, 0]. The only technical restriction on the choice of the selection matrix $\mathbf{S}$ is that the rank of the matrix $\mathbf{S}$ must be exactly equal to the number of rows in $\mathbf{S}$ (i.e., the rows of $\mathbf{S}$ must be linearly independent vectors). clearly in this example the dot product of the two row vectors in

S is equal to zero implying that the two row vectors are orthogonal and thus linearly independent.

Once you've got the selection matrix the remaining calculations are straightforward. Compute the formulas (using equation (2.17)):

$$\mathbf{C}^* = \mathbf{S}^*[\mathbf{H}^*]^{-1}(\mathbf{S}^*)^T \tag{2.19}$$

and

$$\mathcal{W}_N = N(\mathbf{S}\mathbf{y}^*)^T[\mathbf{C}^*]^{-1}(\mathbf{S}\mathbf{y}^*) \tag{2.20}$$

where N is the number of data records.

If the Wald statistic $\mathcal{W}_N$ exceeds the value of a chi-square random variable with r degrees of freedom (where r is the number of rows of $\mathbf{S}$) at the α significance level, then reject the null hypothesis that the r selected weights take on the value of zero at the α significance level.

Chapter 3

Documentation

There is a reason that documentation is described even before the most general features of neURROn design. Students of computer science often think of documentation as ancillary to code. Nothing could be further from the truth. When one realizes that code is secondary to documentation, one becomes a mature programmer.

The reason for this transcendence of documentation is actually quite simple. Code is merely an instance of an abstract idea, the algorithm. Different computer languages yield different instances, but the algorithm is at the core. In order to access the algorithm, a method is coded with inputs and outputs. These inputs and outputs are specified by the programmer, and allow the method to become a "black box" which can be used at will, and, if all goes well, as part of a library of methods that can be used and reused often.

The job of a student programmer is to write code. The job of a master programmer is to write the specifications for the inputs and outputs of methods, which when combined, solve a specific problem. After the specifications are written, then the algorithms within the methods are coded. Documentation ultimately becomes these specifications.

Documentation thus has several purposes. It serves the programmer who wants to reuse code written in the past, but that use is for minor programming tasks. It serves the programmer who wants the code used by other programmers, a legacy for major programming tasks. Ultimately, it serves the programmer who wants the specification to be used by other programmers, and that is the stuff which drives, for example, national economies.

> The neURROn programmer will document code in several ways, but in all, *documentation will be written before the code.* Not after, not even while. Documentation will be written *before* the code.

Each method (and feature corresponding to a set of methods) coded in neURROn is to be documented in 3 specific places:

1. Here, in the Design Manifest. Here a natural language description, algorithmic description if applicable, and specification is to be documented.

2. Immediately prior to the method in the code, as a specification.

3. Throughout the code.

As an example, here is the code for the dot product method in matrix.h:

3.1 Documentation as it appears in the Design Manifest:

Given a matrix object M, and vectors $\vec{v}$ and $\vec{o}$, where $M \cdot \vec{v} = \vec{o}$, e.g.

$$\begin{bmatrix} M_{11} & M_{12} & \cdots & M_{1c} \\ M_{21} & M_{22} & \cdots & M_{2c} \\ \vdots & \vdots & \vdots & \vdots \\ M_{r1} & M_{r2} & \cdots & M_{rc} \end{bmatrix} \cdot \begin{bmatrix} \vec{v_1} \\ \vec{v_2} \\ \vdots \\ \vec{v_c} \end{bmatrix} = \begin{bmatrix} \vec{o_1} \\ \vec{o_2} \\ \vdots \\ \vec{o_r} \end{bmatrix}$$

the following dot product operators are available in the Matrix class:

- M.dotprod(vector& v) returns a new vector object which is the dot product of the matrix and the passed vector
- M.dotprod(vector& v_{in}, vector& v_{out}) which populates the output vector v_{out} with the dot product of the matrix and the passed vector v_{in}
- M.dotprod(vector& v_{in}, vector& v_{out}, unsigned *begin*, unsigned *end*) which populates the output vector range $v_{out}[begin]$ to $v_{out}[end]$ with the dot product of the matrix and the passed vector v_{in}

Examples

```
// Matrix A has 4 rows and 3 columns, vector v1 has 3 elements
vector< double > v2;
v2 = A.dotprod( v1 ); // v1 is unchanged
                      // v2 now contains the dot product

// vector v3 has 4 elements
A.dotprod( v1, v3 ); // v1 is unchanged
                     // v3 now contains the dot product

// vector v4 has 5 elements
A.dotprod( v1, v4, 0, 3 ); // v4[0] through v4[3] now
                           // contains the dot product
```

Notes

For M.dotprod(v), the number of columns in M *must equal* the number of elements in v. For M.dotprod(v_{in}, v_{out}), the number of columns in M *must equal* the number of elements in v_{in}, and the number of rows in M *must equal* the number of elements in v_{out}. Because M.dotprod(v) must construct a new vector object, it is less efficient than M.dotprod(v_{in}, v_{out}).

3.1.1 General principles of documentation in the Manifest: emphasis on use of the method

The emphasis of documentation in the Manifest is on how the method is used. The programmer calling the code should not need to know anything about the inner workings of the method, only how it is called, and what it returns.

3.1.2 Emphasis on readability

Documentation in the manifest is written in a conversational tone so that it is easy to read and understand.

3.1.3 Examples

Good, simple examples are provided to demonstrate the use of the method.

3.1.4 Important notes

Any specific behavior of the code is noted so that it may be used in the most effective manner.

3.2 Documentation as it would appear immediately before the procedure in the code:

```
// Dot product
// For vectors o and v and Matrix M, where M . v = o
//
//                          [ v1
// [ M11 M21 M31 M41    .     v2  =  [ o1
//   M12 M22 M32 M42 ]        v3       o2 ]
//                            v4 ]
//
// ( Note that the number of columns in M, 4, is the number of rows in
v )
//    by convention, ALL vectors are assumed to be vertical
// Calling sequence: M.dotprod( v, o );
```

3.2.1 General principles of documentation immediately before the procedure in the code: clear argument specification

Inputs and when applicable, returned values to the method are clearly and succinctly stated so that casual inspection will allow the code to be used correctly.

3.2.2 Critical notes

Critical usage issues (such as the assumption that all vectors are assumed to be vertical in this example) are included in the comments immediately prior to the procedure.

3.3 Documentation in the code itself:

Finally, the code itself is written and commented *while it is being written.* Debugging is the *last* step, and comments are *modified* as the method is debugged.

```
template< class T >
vector< T >& Matrix< T >::dotprod( const vector< T >& iVec, vector< T
>& oVec ) const
{
   // Number of Matrix columns must equal number of input vector
   elements
   // Number of Matrix rows must equal number of output vector
   elements
   assert ( iVec.size() == ncols_ && oVec.size() == nrows_);

   // Set a pointer ( pData ) to Matrix's data array
   T* pData = data_;

   // Set a const iterator ( pi ) to the input vector
   vector< T >::const_iterator pi;

   // Set an iterator ( po ) to the output vector
   vector< T >::iterator po;

   // Clear output vector by setting all elements to zero
   std::fill( oVec.begin(), oVec.end(), 0 );

   // Calculate the dot product using vector iterators
   for ( po = oVec.begin(); po != oVec.end(); ++po )
      // Calculate an output vector element = Matrix row . input
      vector
      for ( pi = iVec.begin(); pi != iVec.end(); ++pi )
         *po += ( ( *pData++ ) * ( *pi ) );
```

```
   return oVec; // enables use in vector formulae
}
```

3.3.1 General principles of documentation within code:

Note how easy the code is to read. These basic principles are to be followed:

- *Comments are meaningful.* Nothing is more useless than a comment which simply restates the code, for example:

  ```
  // Set the end of outputText to NULL
  outputText[ 9 ] = '\0';
  ```

  is truly useless, whereas

  ```
  // Truncate output string
  outputText[ 9 ] = '\0';
  ```

  while less verbose, is actually more meaningful.

- *Comments are capitalized correctly.* Using all capitals is equivalent to email shouting, and equally offensive. Use

  ```
  // Set the end of outputText to NULL
  ```

  instead of:

  ```
  // SET THE END OF OUTPUTTEXT TO NULL
  ```

- *Pretty printing makes the code easier to read.* Consistent tabbing and bracket placement go a long way towards making clear code.

- *Frequent meaningful comments make the code easier to read.* Every single code block is documented with a meaningful comment.

3.3.2 Variable names:

Variables are to be meaningfully named. It is difficult to sort through previously written code when variables assume generic names like 'x' or 'px'. Verbose variable names are rarely a hindrance. Often student programmers dislike verbose variable names because they do not allow easy packing of variables into 1 line of code. However, this very restriction often makes the code much easier to read by requiring the programmer to split the code into several lines. Consider the example:

```
irpp = (EXP(LN((EXP(n*LN(1+(i/n)))-1)+1)/(ap*ppy))-1)*ap*ppy;
```

While it does fit on 1 line, it is thoroughly unreadable. As such, it is difficult for the programmer to debug while programming, and nearly impossible for succeeding programmers to intelligently use in future code. Consider the alternative:

```
// Use times_per_year_compounded = 2 for Canada, 12 for U.S.
incremental_rate = interest_rate / times_per_year_compounded;

// Calculate effective annual rate
effective_rate = EXP( times_per_year_compounded
   * LN( 1 + incremental_rate ) ) - 1;

// Finally, calculate incremental_interest = interest rate per period:
amortization = amortization_period * payments_per_year;
incremental_interest = (EXP( LN( effective_rate + 1 ) / amortization )
- 1 )
   * amortization;
```

Not only is the algorithm that the programmer used to calculate the interest rate per period manifestly clear, the variables are so well named as to make the '//' comments complimentary rather than critical. And when a future programmer sees the variable 'incremental_interest' 2000 lines of code later, he or she will have half a clue about what that variable means, whereas the meaning of 'irpp' would be completely opaque.

3.3.3 Space

One of the simplest ways to make code readable is to insert space around operators in code. Consider the example in section 3.3.2,

```
irpp = (EXP(LN((EXP(n*LN(1+(i/n)))-1)+1)/(ap*ppy))-1)*ap*ppy;
```

One of the difficulties in reading this code is the lack of spaces between operators.

Rules for spaces in coding neUROn2++ are:

- Put spaces after binary operators (like =):

  ```
  a = b + c;
  ```

  rather than:

  ```
  a=b+c;
  ```

- After control keywords like 'for' and 'while', put spaces around the ()s, e.g.:

```
for ( i = 1; i <= 10; i++ ) {
```

rather than:

```
for(i = 1; i <= 10; i++){
```

- Inside of brackets, braces or parentheses, put spaces right after the first bracket, brace, or parenthesis, and right before the second, e.g.:

```
a[ i ] = 10;
```

rather than:

```
a[i] = 10;
```

Chapter 4

C++ coding

neUROn was written in C, and took advantage of the high- and low-level compilation features available in that language. While neUROn2++ was written in C++, coding began shortly after the first standardized C++ implementation became available. Because C++ is a superset of C, neUROn++ became a hodgepodge of C and C++ coding styles, and ultimately, became too cumbersome to maintain, let alone extend. neUROn2++ is designed to take advantage of the extensibility offered by the object-oriented nature of C++, but it is also designed to take advantage of other significant advances in C++ such as operator and method overloading. The following C++ style rules are expected to be followed closely, but are not intended to be an exhaustive list. For those interested in an excellent source of good C++ coding style rules, see Deitel and Deitel's *C++ How to Program.*[1]

In general, while coding, think of the 3 cardinal rules of good programming: code must be

- *Readable* (see Chapter 3 on rules for documentation.)
- *Efficient*
- *Bulletproof*

Two excellent resources for C++ coding questions are the C++ Usenet group *comp.lang.c++* and Marshall Cline's companion FAQ at http://www.parashift.com/c++-faq-lite. If you have *any* question regarding the most efficient way to code a particular problem, post it on comp.lang.c++: you'll likely get a response within a couple of hours if not minutes, and the responses often contain excellent strategies that you would not have considered. However, look first in the FAQ, because if it's contained there, your post will likely be met with the terse and somewhat embarrasing reply, "look in the FAQ."

[1] Deitel and Deitel, C++ How to Program, 3rd edition, Prentice Hall, 2001, ISBN 0-13-089571-7

4.1 Cross compilation

All code must be compiled successfully on both GNU C++ (g++) and Microsoft Visual C++ (MSVC++). Code will not be considered complete until it can be successfully compiled and executed on both platforms.

4.2 Use friends sparingly

Friend methods are to be used *only* when absolutely necessary. Consider using a friend a failure to find a good solution. Obviously, friends are used in overloading the ostream operator << and istream operator >>, e.g.:

```
friend ostream& operator << ( ostream&, const ... );
```

because there is simply no other good way to overload the ostream operator for a new object. However, friends should *not* be used to overload mathematical operators such as + = and +. *Do not use a friend unless you discuss it with the neUROn2++ project group first.*

4.3 Use const correctness

Const correctness provides one way to write efficient and bulletproof code right from the start. By being const-correct, you think about which objects can be mutated in a method and which cannot. By not allowing certain methods to mutate objects, you prevent particularly intractable runtime errors, and by judiciously allowing certain methods to mutate objects (e.g. passed array references), you know exactly where your code is behaving efficiently, and exactly where runtime errors may be introduced. *The label const is to be applied while code is written, not after. Constness applied as a patch to already written code is uniformly considered to be very poor programming practice.*

4.4 Use asserts generously

Asserts allow you to easily track down runtime errors that would be otherwise very difficult to find, *but you have to include the asserts in the first place.* For example, in the Matrix class, here is an assert that will catch a programmer trying to add 2 Matrices of different sizes:

```
// Catch different size matrices
assert ( nrows_ == rhs.nrows_ && ncols_ == rhs.ncols_ );
```

If the program encounters an add operation with 2 Matrices of different sizes, it will cease immediately, and print out a meaningful error statement with an appropriate program line, pointing you directly to the problem. Without the assert, the program may bomb out with the ubiquitous stackdump or core runtime error, giving you absolutely no debugging information, and sending you on a debugging task that may take many hours to complete.

Chapter 5

vector and Matrix

Establishment of numerical vector and 2-dimensional matrix objects in neURON2++ serves as the basis by which clear, concise, efficient neural computational code may be written. The header file "vector_ops.h" extends the STL <vector> by adding simple unary and binary mathematical operations, and overloaded ostream << and istream >> operators for simplified output and input. The header file "matrix.h" implements a matrix class for simplified 2-dimensional matrix operations.

5.1 vector_ops.h

Inclusion of the header file "vector_ops.h" overloads operators and adds methods to the <vector> STL. Please see <vector> STL documentation for standard <vector> STL methods.

5.1.1 Unary mathematical operators

The following operators perform element by element addition, subtraction, multiplication and division on the right hand side (rhs) vector and left hand side (lhs) vector, returning the result in the left hand side (lhs) vector:

- $+=$: unary addition
- $-=$: unary subtraction
- $*=$: unary multiplication
- $/=$: unary division

Example

```
// Standard C-style arrays
double data1[ 4 ] = { 1.0, 2.0, 3.0, 4.0 },
```

```
       data2[ 4 ] = { 2, 3, 4, 5 },
       data3[ 3 ] = { 8, 9, 10 };

// vector's std array ctor
vector< double > v1( data1, data1 + 4 ), v2( data2, data2 + 4 );

// Unary addition element by element
v1 += v2;

// Different size vectors
v3 += v1; // v1[ 3 ] will be ignored
```

Notes

The vectors may have different numbers of elements, but the right hand side (rhs) vector size *must be* greater than the left hand side (lhs) vector. If the vectors have different sizes, the remaining elements at the end of the rhs vector that are not represented in the lhs vector will be ignored.

5.1.2 Unary mathematical operators with scalars

The following operators perform scalar addition, subtraction, multiplication and division on a right hand side (rhs) scalar and left hand side (lhs) vector, returning the result in the left hand side (lhs) vector:

- $+=$: unary scalar addition
- $-=$: unary scalar subtraction
- $*=$: unary scalar multiplication
- $/=$: unary scalar division

Example

```
// Standard C-style array
double data1[ 4 ] = { 1.0, 2.0, 3.0, 4.0 };

// vector's std array ctor
vector< double > v1( data1, data1 + 4 );

// Unary scalar addition
v1 += 2.0;
```

Notes

Type resolution may be a problem if the right hand side argument is not specified, e.g. "v1 + = 2;" may give an error. To fix, either cast appropriately, or include decimal points in double values, e.g. "v1 + = 2.0;".

5.1.3 Binary mathematical operators

The following operators perform element by element addition, subtraction, multiplication and division on the right hand side (rhs) vectors, returning the result for allocation in a new left hand side (lhs) vector:

- +: binary addition
- −: binary subtraction
- ∗: binary multiplication
- /: binary division

Example

```
// Standard C-style arrays
double data1[ 4 ] = { 1.0, 2.0, 3.0, 4.0 },
       data2[ 4 ] = { 2, 3, 4, 5 },
       data3[ 3 ] = { 8, 9, 10 };

// vector's std array ctor
vector< double > v1( data1, data1 + 4 ),
   v2( data2, data2 + 4 ), v3( data3, data3 + 3 ),
   v4, v5;

// Binary addition element by element
v4 = v1 + v2;

// Binary addition with different size vectors
v5 = v3 + v2; // v5 will have 3 elements
```

Notes

In the passed arguments to the overloaded operator, the left hand side (lhs) is *not* a reference, but a local copy of the lhs vector, and a local copy is returned. There is thus some inefficiency introduced in the use of binary vector operators. The vectors may have different numbers of elements, but like their unary counterparts, the right hand side (rhs) size *must be* greater than the size of the lhs vector. The final size of the returned vector will be the same as the lhs vector size.

5.1.4 Binary mathematical operators with scalars

The following operators perform scalar addition, subtraction, multiplication and division on a vector and a scalar, and return the result for allocation in a new vector:

- +: binary scalar addition

- $-$: binary scalar subtraction
- $*$: binary scalar multiplication
- /: binary scalar division

Example

```
// Standard C-style arrays
double data1[ 4 ] = { 1.0, 2.0, 3.0, 4.0 };

// vector's std array ctor
vector< double > v1( data1, data1 + 4 ), v2;

// Binary scalar addition
v2 = v1 + 3.0;
```

Notes

Type resolution may be a problem if the right hand side argument is not specified, e.g. "v1 + = 2;" may give an error. To fix, either cast appropriately, or include decimal points in double values, e.g. "v1 + = 2.0;". In the passed arguments to the overloaded operator, the left hand side (lhs) is *not* a reference, but a local copy of the lhs vector, and a local copy is returned. There is thus some inefficiency introduced in the use of binary vector operators.

5.1.5 Overloaded == and ! =

The boolean operators == and ! = perform element by element comparisons between vectors, == returning *true* if they are equal, and ! = returning *false* if they are not.

Example

```
vector< double > v1, v2;

// Code to load the vectors would follow...

if ( v1 == v2 )
   cout << "They're equal" << endl;
else
   cout << "They're not equal" << endl;
```

5.1.6 Overloaded <<

The ostream operator << is overloaded for vector output.

Example

```
cout << "v1 = " << v1 << endl;
```

Notes

This overloaded operator may also be used for vector of vector (etc.), and will insert double (triple, etc.) spaces between the output vectors.

5.1.7 Overloaded >>

The istream operator >> is overloaded for vector input.

Example

```
vector< double > v1( 5 );
cout << "Enter a 5 element vector: ";
cin >> v1;
cout << "Your 5 element vector is " << v1 << endl;
```

5.1.8 Dot product

The *dotprod* method may be used to compute a scalar dotproduct from 2 vectors. Its signature is:

dotprod(vector& v_1, vector& v_2)

Example

```
x = dotprod( v1, v2 );
```

Notes

v_1 and v_2 *must have* the same number of elements.

5.1.9 Function passing

The following methods allow functions to be applied to a vector element by element:

- func(vector& v_{in}, function f, vector& v_{out}): applies function f element by element to vector v_{in}, populating vector v_{out} with the result.
- func(vector& v_{in}, function f, vector& v_{out}, unsigned *begin*, unsigned *end*): applies function f element by element to vector v_{in} in the range $v_{in}[begin]$ to $v_{in}[end]$, populating vector v_{out} in the range $v_{out}[begin]$ to $v_{out}[end]$ with the result.
- func(vector& v, function f): applies function f element by element to vector v, and returns a new vector object.

- func(vector& v, function f, unsigned $begin$, unsigned end): applies function f element by element to vector v in the range $v[begin]$ to $v[end]$, and returns a new vector object of size $v[begin] - v[end] + 1$.

Example

```
\index{Transfer function!Sigmoidal}

#include <vector>
#include <math.h>

using namespace std;

#include "vector_ops.h"

// Define sigmoidal function as a class which extends
// unary_function.  This is necessary for algorithm::transform.
class sigmoidal : public unary_function< double, double >
{
   public:
      inline double operator()( const double& x )
      {
         return 1.0 / ( 1.0 + exp( -x ) );
      }
};

int main() {
   // Standard C-style arrays
   double data1[ 4 ] = { 1.0, 2.0, 3.0, 4.0 },
          data2[ 4 ] = { 2, 3, 4, 5 };

   // vector's std array ctor
   vector< double > v1( data1, data1 + 4 ),
      v2( data1, data1 + 4 ), v3( 4 ), v4;

   // Apply sigmoidal function to each element of v1
   // Populate v3 with the result
   func( v1, sigmoidal(), v3 );

   // Apply sigmoidal function to v1 with itself as result
   func( v1, sigmoidal(), v1 );

   // Apply sigmoidal function to each element of v2
   // After this function call, v4 will contain the result
   v4 = func( v2, sigmoidal() );
```

```
   // Copy vector v1 into a new vector v5
   vector< double > v5( v1 );

   // Apply the sigmoidal function to elements 1 through 2
   // in vector v1, with the result placed in vector v5
   // elements 1 through 2
   func( v1, sigmoidal(), v5, 1, 2 );

   // The receiving vector may be smaller than the sending
   // vector
   vector< double > v6( 3 );
   func( v1, sigmoidal(), v6, 0, 2 ); // v1 has 4 elements
                                      // v6 has 3 elements

   // Make a new vector
   vector< double > v7;

   // And put the sigmoidal function applied to vector v1
   // elements 1 through 2 into it
   v7 = func( v1, sigmoidal, 1, 2 ); // v7 has 2 elements

   return 0;
}
```

Notes

The passed function *must* be defined as a class extension of unary_function as demonstrated in the example. Common functions useful for neural computation (e.g. sigmoidal) are packaged with neUROn2++.

If a vector to be populated is passed without bounds arguments, it *must have* the same number of elements as the input vector. If a vector to be populated is passed with bounds arguments, the size of the vector *must be* greater than the upper bound. As func(vector& v, function f) and func(vector& v, function f, unsigned *begin*, unsigned *end*) must construct new vectors, they are less efficient than func(vector& v_{in}, function f, vector& v_{out}) and func(vector& v_{in}, function f, vector& v_{out}, unsigned *begin*, unsigned *end*). func(vector& v_{in}, function f, vector& v_{out}) and func(vector& v_{in}, function f, vector& v_{out}, unsigned *begin*, unsigned *end*) are preferred.

To use a class extension of unary_function with a scalar, the following may be written (using sigmoidal as an example):

```
double x, y;
y = sigmoidal()( x );
```

is valid if sigmoidal is defined as described above.

5.1.10 Calculating the sum of squares of a vector elements

The method squared(vector& v) returns the value of the sum of squares of vector v elements.

Example

```
vector< double > v; // v is then populated with double values...
double sum;
sum = squared( v );
```

5.1.11 Calculating the maximum absolute value of vector elements

The method maxabs(vector& v) returns the maximum absolute value of vector v elements.

Example

```
vector< double > v; // v is then populated with double values...
double max;
max = maxabs( v );
```

5.1.12 Calculating the minimum absolute value of vector elements

The method minabs(vector& v) returns the minimum absolute value of vector v elements.

Example

```
vector< double > v; // v is then populated with double values...
double min;
min = minabs( v );
```

5.1.13 Randomization

namespace nvec

The method random(vector& v, double n) populates a double precision vector v with double precision random numbers between $-n$ and $+n$.

Example

```
vector< double > v( 6 ); // Make a 6 element vector
// Populate it with random numbers between -0.5 and +0.5,
//    and print it out
cout << "The random vector is " << nvec::random( v, 0.5 );
```

Notes

This method is only coded for vectors of type double, e.g. vector< double >.

5.1.14 Random positions

namespace nvec

The method random_positions(vector& v) populates an unsigned vector v with a series of unique integers from 0 to $n-1$, where n is the number of elements in the vector.

Example

The example code:

```
vector< unsigned > v( 6 );
cout << nvec::random_positions( v ) << endl;
```

May yield:

```
5 2 1 3 4 0
```

Notes

For obvious reasons, this method is only coded for vectors of type unsigned, e.g. vector< unsigned >.

5.1.15 Converting a vector of vector into a vector

The following methods allow conversion of a vector of vector into a vector, thus "flattening" the vector of vector:

- flatten(vector< vector >& *container*, vector& v) converts the data structure vector of vector *container* to passed vector v, returns a reference to vector v.
- flatten(vector< vector >& *container*) converts the data structure vector of vector *container* to a new vector, returns a reference to the new vector.

Example

See section 6.9 for method *variable_parse*.

```
// Create a vector of vector using util::variable_parse
string parse_test = "0-3,5; 4,8; 6,7";
vector< vector< unsigned > > parse_numbers;
parse_numbers = util::variable_parse( parse_test );
cout << "Here it is: " << parse_numbers << endl;
```

```
// Now flatten it
cout << endl << "Flattening:" << endl;
vector< unsigned > v_flat;
flatten( parse_numbers, v_flat );
cout << v_flat << endl;
cout << flatten( parse_numbers ) << endl;
```

Notes

If you're using MSVC, you *must* include in your code the following, as MSVC will give you error warnings for each vector< vector< unsigned > > reference:

```
#ifdef _MSC_VER // MSVC
#pragma warning (disable: 4786)
#endif
```

5.1.16 Binning a vector into a vector of vector

- bin(const vector& v, unsigned b, bool $binFlag$, vector< vector >& vv): partitions vector v into the data structure vector of vector vv according to specified bin size, with flag $binFlag$ indicating number in the last bin:
  - **false** if the last bin contains <= bin size (last bin = remaining elements,
  - **true** if the last bin contains >= bin size (remaining elements are added to the last bin.
- bin(const vector& v, unsigned b, bool $binFlag$): partitions vector v into a new data structure vector of vector according to specified bin size, with flag $binFlag$ indicating number in the last bin:
  - **false** if the last bin contains <= bin size (last bin = remaining elements,
  - **true** if the last bin contains >= bin size (remaining elements are added to the last bin.

  and returns a reference to the new data structure vector of vector.

Example

```
double vpart_[ 10 ] = { 0.0, 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0,
9.0 };
vector< double > vpart( vpart_, vpart_ + 10 );
vector< vector< double > > vbin;

bin( vpart, 3, false, vbin );
cout << vpart << endl << "partitioned with small last bin =" << endl <
< vbin;
```

```
cout << endl << "partitioned with large last bin =" << endl <<
   bin( vpart, 3, true ) << endl;
```

Produces the following output:

```
0 1 2 3 4 5 6 7 8 9
partitioned with small last bin =
0 1 2  3 4 5  6 7 8  9
partitioned with large last bin =
0 1 2  3 4 5  6 7 8 9
```

Notes

As with the previous method *flatten*, if you're using MSVC, you *must* include in your code the following:

```
#ifdef _MSC_VER // MSVC
#pragma warning (disable: 4786)
#endif
```

5.2 matrix.h and the Matrix class

Including the header file matrix.h allows implementation of the Matrix class, and allows simplified numerical operations on 2 dimensional arrays.

5.2.1 Constructors

The following constructors are implemented in the Matrix class:

- Matrix(): Constructs a Matrix object of zero size, with zero rows and zero columns.
- Matrix(unsigned r, unsigned c): Constructs a Matrix object with r rows and c columns.
- Matrix(unsigned r, unsigned c, *value*): Constructs a Matrix object with r rows, c columns, and fills it with *value*.
- Matrix(vector& q, vector& p^t): Constructs a Matrix object which is the outer product of vectors q and the transpose of p (see 5.2.17).
- Matrix(string& $filename$, unsigned c): Constructs a Matrix object, and loads it from a file named "$filename$" with c columns.
- Matrix(bool *report*, string& $filename$): Constructs a Matrix object, and loads it from a file named "$filename$". Delimeters are any whitespace character, or any character that is non-numeric (+, -, e and E are considered numeric, as is a decimal point.) Rows are delimited by carriage

returns in the file, and the number of columns is the maximum number of elements in any one row. Elements in unpopulated rows will be filled with 0, beginning from the *right* hand side of the row. If the bool flag *report* is true, a report will be printed indicating which rows were found to be unpopulated.

Examples

```
// Make some Matrices
Matrix< double > A( 4, 3 ), B, C( 5, 4, 0.01 );

// Get a file name
string filename;
cout << "The file name is ";
cin >> filename;

// Make a Matrix from the file
Matrix< double > C( filename, 3 ), // It is known to have 3 columns
                 D( true, filename ); // Number of columns unknown,
                                      // report will be printed

// Make a matrix from the outer product of vectors v1 and v2
Matrix< double > E( v1, v2 );
```

Notes

A Matrix constructed by *Matrix(r, c)* is **not** initially populated. Consider it to be filled with garbage.

5.2.2 Accessors

Given a Matrix object M, the following accessors are implemented in the Matrix class:

- M(unsigned r, unsigned c): overloaded () to access the element at row r and column c in Matrix *matrix_object*
- M.rows(): returns number of rows in Matrix
- M.cols(): returns number of columns in Matrix

Examples

```
// Make a Matrix with 4 rows and 3 columns
Matrix< double > A( 4, 3 );

// Fill Matrix A with numbers
for ( i = 0; i < 4; i++ )
```

```
   for ( j = 0; j < 3; j++ )
      A( i, j ) = 3 * i + j + 1;

// Print out number of rows and number of columns in Matrix
cout << "Matrix has " << A.rows() << " rows and ";
cout << A.cols() << " columns." << endl;
```

5.2.3 Submatrix accessors

The following accessors return a submatrix from a Matrix:

- M.submatrix(unsigned row_1, unsigned row_N, unsigned col_1, unsigned col_N, Matrix& M_{out}): populates passed matrix M_{out} with Matrix M rows row_1 to row_N, and columns col_1 to col_N.
- M.submatrix(unsigned row_1, unsigned row_N, unsigned col_1, unsigned col_N): returns a new Matrix object with Matrix M rows row_1 to row_N, and columns col_1 to col_N.

Examples

```
// Make some matrices
Matrix< double > P( 5, 6, 2.5 ), R( 2, 4 ), S;

// Populate matrix R with the resulting submatrix
P.submatrix( 3, 4, 2, 5, R );

// Construct a new matrix for result
S = P.submatrix( 3, 4, 2, 5 );
```

Notes

Row and column dimensions must be within the bounds of the originating matrix. If a matrix is passed as an argument, its number of columns *must equal* col_N - col_1 + 1, and its number of rows *must equal* row_N - row_1 + 1. A passed matrix to be populated is more efficient than constructing a new matrix for the result.

5.2.4 Row and column accessors

The following accessors return a vector from a row or column of a Matrix:

- M.row(unsigned r, vector& v): populates vector v with row r from Matrix M.
- M.row(unsigned r): returns a new vector with row r from Matrix M.

- M.col(unsigned c, vector& v): populates vector v with column c from Matrix M.
- M.col(unsigned c): returns a new vector with column c from Matrix M.

Examples

```
// Make a matrix with 5 rows and 6 columns, filled with Pi
Matrix< double > P( 5, 6, 3.14159 );

// Make some vectors
vector< double > v5( 5 ), v6( 6 ), v7, v8;

// Put row 2 of Matrix P in vector v6
P.row( 2, v6 );

// Construct a new vector v7 for row 2 of Matrix P
v7 = P.row( 2 );

// Put column 2 of Matrix P in vector v5
P.col( 2, v5 );

// Construct a new vector v8 for column 2 of Matrix P
v8 = P.col( 2 );
```

Notes

For the methods which take passed vectors as arguments, the size of the vector v *must equal* the number of Matrix columns for M.row(r, v), and the size of the vector v *must equal* the number of Matrix rows for M.col(c, v). The methods which take passed vectors as arguments are more efficient than the methods which construct vectors.

5.2.5 Methods which append vectors to Matrices

The following methods append a vector to a Matrix:

- M.addrow(vector& v, Matrix N): populates Matrix N by appending vector v to the *end* of Matrix M as a *row*.
- M.addrow(vector& v): returns a new Matrix with vector v appended to the *end* of Matrix M.
- M.addcol(vector& v, Matrix N): populates Matrix N by appending vector v to the *right* of Matrix M as a *column*.
- M.addrow(vector& v): returns a new Matrix with vector v appended to the *right* of Matrix M.

Examples

```
// Matrix A has 3 rows and 4 columns, vector v1 has 4 elements
// Matrix B has 4 rows and 4 columns
A.addrow( v1, B );
C = A.addrow( v1 );

// Matrix D has 3 rows and 4 columns, vector v2 has 3 elements
// Matrix E has 3 rows and 5 columns
D.addrow( v2, E );
F = D.addrow( v2 );
```

Notes

The number of columns in M and N *must* be the same as the number of elements of vector v for M.addrow, and the number of rows in M and N *must* be the same as the number of elements of vector v for M.addcol.

Because they construct new Matrices, M.addrow(vector& v) and M.addcol(vector& v) are less efficient than M.addrow(vector& v, Matrix N) and M.addcol(vector& v, Matrix N). M.addrow(vector& v, Matrix N) and M.addcol(vector& v, Matrix N) are to be preferred.

M.addcol is computationally expensive, and its use should be minimized.

5.2.6 Methods which replace a row or column of a Matrix with a vector

The following methods replace a row or column of a Matrix with a vector:

- M.replacerow(unsigned n, vector& v): replaces row n in Matrix M with vector v, returning a reference to Matrix M with the result.
- M.replacecol(unsigned n, vector& v): replaces column n in Matrix M with vector v, returning a reference to Matrix M with the result.

Examples

```
// Replace row 3 of Matrix A with vector v1, and print the result
cout << A.replacerow( 3, v1 );

// Replace column 2 of Matrix A with vector v2, and print the result
cout << A.replacecol( 2, v2 );
```

Notes

The number of columns in M *must* be the same as the number of elements of vector v for M.replacerow, and the number of rows in M *must* be the same as the number of elements of vector v for M.replacecol.

5.2.7 Column inclusion and exclusion

The following operators include or exclude those columns indicated by the passed vector, which includes the (unsigned) column values:

- M.includecols(Matrix& M_{out}, vector< unsigned >& v): includes in the returned Matrix M_{out} only those columns from Matrix M indicated by v, returning a reference to Matrix M_{out} with the result.
- M.includecols(vector< unsigned >& v): returns a new Matrix *with* only those columns from Matrix M indicated by v
- M.excludecols(Matrix& M_{out}, vector< unsigned >& v): excludes in the returned Matrix M_{out} those columns from Matrix M indicated by v, returning a reference to Matrix M_{out} with the result.
- M.excludecols(vector< unsigned >& v): returns a new Matrix *without* those columns from Matrix M indicated by v

Examples

```
// Make a 5x4 Matrix, fill with some numbers
Matrix< double > M( 5, 4 );
for ( unsigned row = 0; row < 5; row++ )
   for ( unsigned column = 0; column < 4; column++ )
      M( row, column ) = 4 * column + row;
cout << "Here is the original Matrix:" << M;

// Make some smaller Matrices
Matrix< double > M1( 5, 2 ), M2, M3( 5, 2 ), M4;

// Make a vector to indicate columns 1 & 3
vector< unsigned > v;
v.push_back( 1 );
v.push_back( 3 );

// Passing a Matrix to include columns
M.includecols( M1, v );
cout << "From the matrix above, including columns "
   << v << ": " << M1;

// A new Matrix to include columns
M2 = M.includecols( v );
cout << "Returning a new Matrix: " << M2;

// Passing a Matrix to exclude columns
M.excludecols( M3, v );
cout << "From the matrix above, excluding columns "
```

```
   << v << ": " << M3;

// A new Matrix to exclude columns
M4 = M.excludecols( v );
cout << "Returning a new Matrix: " << M4;
```

Notes

If the Matrix M_{out} to be populated is passed as an argument, its rows *must* be the same as the number of rows in M, and its number of columns *must* be the same as the difference between the number of columns in M and the number of elements in the passed vector v. The elements in the passed vector v *must* be *unique* (no duplicated values), and no element can be >= to the number of columns in Matrix M.

These methods are useful to include or exclude variables or nodes represented by columns in a Matrix.

5.2.8 Boolean equality operators

The following operators perform boolean equality and inequality comparisons, element by element:

- ==: equality
- ! =: inequality

Example

```
// Make 2 Matrices with 4 rows and 3 columns
Matrix< double > A( 4, 3 ), B( 4, 3 );

// Fill Matrices A and B with numbers

for ( i = 0; i < 4; i++ )
{
   for ( j = 0; j < 3; j++ )
   {
      A( i, j ) = 3 * i + j;
      B( i, j ) = 3 * i + j + 1;
   }
}

// Compare Matrix A to B:
if ( A == B )
   cout << "They're equal" << endl;
if ( A != B )
   cout << "They're not equal" << endl;
```

5.2.9 Unary mathematical operators

The following operators perform element by element addition, subtraction, multiplication and division on the right hand side (rhs) Matrix and left hand side (lhs) Matrix, returning the result in the left hand side (lhs) Matrix:

- $+=$: unary addition
- $-=$: unary subtraction
- $*=$: unary multiplication
- $/=$: unary division

Example

```
// Make 2 Matrices with 4 rows and 3 columns
Matrix< double > A( 4, 3 ), B( 4, 3 );

// Fill Matrices A and B with numbers

for ( i = 0; i < 4; i++ )
{
   for ( j = 0; j < 3; j++ )
   {
      A( i, j ) = 3 * i + j;
      B( i, j ) = 3 * i + j + 1;
   }
}

// Add matrix B to A element by element
A += B;
```

5.2.10 Unary mathematical operators with scalars

The following operators perform scalar addition, subtraction, multiplication and division on a right hand side (rhs) scalar and left hand side (lhs) Matrix, returning the result in the left hand side (lhs) Matrix:

- $+=$: unary scalar addition
- $-=$: unary scalar subtraction
- $*=$: unary scalar multiplication
- $/=$: unary scalar division

Example

```
// Make a Matrix with 4 rows and 3 columns, filled with 1.0
Matrix< double > A( 4, 3, 1.0 );

// Add 3.0 to each element of A
A += 3.0;
```

5.2.11 Binary mathematical operators

The following operators perform element by element addition, subtraction, multiplication and division on the right hand side (rhs) Matrices, returning a new Matrix object:

- $+$: binary addition
- $-$: binary subtraction
- $*$: binary multiplication
- $/$: binary division

Example

```
// Make 2 Matrices with 4 rows and 3 columns, and a 3rd empty Matrix
Matrix< double > A( 4, 3 ), B( 4, 3 ), C;

// Fill Matrices A and B with numbers
for ( i = 0; i < 4; i++ )
{
   for ( j = 0; j < 3; j++ )
   {
      A( i, j ) = 3 * i + j;
      B( i, j ) = 3 * i + j + 1;
   }
}

// Add matrix A and B element by element, the result is matrix C
C = A + B;
```

Notes

The Matrices *must have* the same number of rows and columns. As a new matrix object must be constructed, unary operations are preferred for efficiency.

5.2.12 Binary mathematical operators with scalars

The following operators perform scalar addition, subtraction, multiplication and division on a Matrix, returning a new Matrix object:

- $+$: binary scalar addition
- $-$: binary scalar subtraction
- $*$: binary scalar multiplication
- $/$: binary scalar division

Example

```
// Make some Matrices
Matrix< double > A( 4, 3, 1.0 ), B;

// Add 3.0 to each element of A, B is the result
B = A + 3.0;
```

Notes

As a new matrix object must be constructed, unary operations are preferred for efficiency. **Known bug: the scalar must be on the right side of the Matrix for all operators.**

5.2.13 Transpose

The following operators transpose a Matrix:

- M.t() transposes matrix M, and returns a matrix object
- M.t(O) populates passed matrix O with the transpose of M

Example

```
// A is a 3 x 4 matrix
Matrix< double > B( 4, 3 ), C;

A.t( B );  // B now contains the transpose of A
C = A.t(); // C now contains the transpose of A
```

Notes

For M.t(O), the number of columns in M *must equal* the number of rows in O and the number of rows in M *must equal* the number of columns in O.

Because M.t() must construct a new matrix object, it is less efficient than M.t(O).

5.2.14 Overloaded <<

The ostream operator << is overloaded for Matrix output.

Example

```
cout << "Matrix A = " << A << endl;
```

setHeader

The setHeader method:

- M.setHeader(bool $flag$)

is provided for ostream output. If $flag$ is true, a header line containing the format:

r rows and c columns:

is output to the stream prior to the Matrix, whereas if $flag$ is false, the header line is not output, only the Matrix. *By default, this flag is set to* **true**.

Examples

```
cout << "Matrix A alone = " << A.setHeader( false ) << endl;
A.setHeader( true );
cout << "Matrix A with header = " << A << endl;
```

5.2.15 Overloaded >>

The istream operator >> is overloaded for Matrix input.

Example

```
Matrix< double > A( 3, 2 );
cout << "Enter Matrix A: ";
cin >> A;
```

5.2.16 Dot product

Given a matrix object M, and vectors $\vec{v}$ and $\vec{o}$, where $M \cdot \vec{v} = \vec{o}$, e.g.

$$\begin{bmatrix} M_{11} & M_{12} & \cdots & M_{1c} \\ M_{21} & M_{22} & \cdots & M_{2c} \\ \vdots & \vdots & \vdots & \vdots \\ M_{r1} & M_{r2} & \cdots & M_{rc} \end{bmatrix} \cdot \begin{bmatrix} \vec{v_1} \\ \vec{v_2} \\ \vdots \\ \vec{v_c} \end{bmatrix} = \begin{bmatrix} \vec{o_1} \\ \vec{o_2} \\ \vdots \\ \vec{o_r} \end{bmatrix}$$

or other matrix objects N and O, where $M \cdot N = O$, the following dot product operators are available in the Matrix class:

- M.dotprod(vector& v) returns a new vector object which is the dot product of the matrix and the passed vector
- M.dotprod(vector& v_{in}, vector& v_{out}) which populates the output vector v_{out} with the dot product of the matrix and the passed vector v_{in}
- M.dotprod(vector& v_{in}, vector& v_{out}, unsigned *begin*, unsigned *end*) which populates the output vector range $v_{out}[begin]$ to $v_{out}[end]$ with the dot product of the matrix and the passed vector v_{in}
- M.dotprodt(vector& v) returns a new vector object which is the dot product of the *transpose* of the matrix and the passed vector
- M.dotprodt(vector& v_{in}, vector& v_{out}) which populates the output vector v_{out} with the dot product of the *transpose* of the matrix and the passed vector v_{in}
- M.dotprodt(vector& v_{in}, vector& v_{out}, unsigned *begin*, unsigned *end*) which populates the output vector range $v_{out}[begin]$ to $v_{out}[end]$ with the dot product of the *transpose* of the matrix and the passed vector v_{in}
- M.dotprod(N) returns a new Matrix object which is the dot product of the matrix M and the passed matrix N
- M.dotprod(N, O) which populates the passed matrix O with the dot product of the matrix M and the passed matrix N

Examples

(dotprodt(...) follows the same form as dotprod(...).)

```
// Matrix A has 4 rows and 3 columns, vector v1 has 3 elements
vector< double > v2;
v2 = A.dotprod( v1 ); // v1 is unchanged
                      // v2 now contains the dot product

// vector v3 has 4 elements
A.dotprod( v1, v3 ); // v1 is unchanged
                     // v3 now contains the dot product

// vector v4 has 5 elements
A.dotprod( v1, v4, 0, 3 ); // v4[0] through v4[3] now
                           // contains the dot product

// Matrix B has 3 rows and 5 columns
Matrix< double >C ( 4, 5 ), D;
A.dotprod( B, C );  // C now contains the dot product
D = A.dotprod( B ); // D now contains the dot product
```

Notes

For M.dotprod(v), the number of columns in M *must equal* the number of elements in v. For M.dotprod(v_{in}, v_{out}), the number of columns in M *must equal* the number of elements in v_{in}, and the number of rows in M *must be equal to or greater than* the number of elements in v_{out}. If the number of rows in M is greater than the number of elements in v_{out}, the resulting vector will be truncated to the size of v_{out}, and the remainder ignored.

For M.dotprodt(v), the number of rows in M *must equal* the number of elements in v. For M.dotprodt(v_{in}, v_{out}), the number of rows in M *must equal* the number of elements in v_{in}, and the number of columns in M *must be equal to or greater than* the number of elements in v_{out}. If the number of columns in M is greater than the number of elements in v_{out}, the resulting vector will be truncated to the size of v_{out}, and the remainder ignored.

For M.dotprod(N), the number of columns in M *must equal* the number of rows in N. For M.dotprod(N, O), the number of rows in M *must equal* the number of rows in O, the number of columns in N *must equal* the number of columns in O, and the number of columns in M *must equal* the number of rows in N.

Because M.dotprod(v) and M.dotprodt(v) must construct new vector objects, and M.dotprod(N) must construct a new matrix object, these methods are less efficient than M. dotprod(v_{in}, v_{out}), M.dotprodt(v_{in}, v_{out}) and M.dotprod(N, O).

dotprod_row

The *dotprod_row* method takes a single row from a Matrix, and transposes it to be used as the right hand side argument for a dot product. It is useful for extracting a single row from a dataset and passing it to the first layer of a neural network. It is functionally equivalent to $M \cdot \vec{d^t} = \vec{o}$, where M is the matrix, $\vec{d^t}$ is the transpose of the vector $\vec{d}$ derived from a single row of the dataset matrix, and $\vec{o}$ is the output vector.

Suppose that d is a single row of the dataset matrix D, e.g.:

$$\begin{bmatrix} \cdots & \cdots & \cdots & \cdots \\ d_{r1} & d_{r2} & \cdots & d_{rc} \\ \vdots & \vdots & \vdots & \vdots \\ \cdots & \cdots & \cdots & \cdots \end{bmatrix}$$

Then M.dotprod_row(D, r, o) is equivalent to:

$$\begin{bmatrix} M_{11} & M_{12} & \cdots & M_{1c} \\ M_{21} & M_{22} & \cdots & M_{2c} \\ \vdots & \vdots & \vdots & \vdots \\ M_{r1} & M_{r2} & \cdots & M_{rc} \end{bmatrix} \cdot \begin{bmatrix} \vec{d_{r1}} \\ \vec{d_{r2}} \\ \vdots \\ \vec{d_{rc}} \end{bmatrix} = \begin{bmatrix} \vec{o_1} \\ \vec{o_2} \\ \vdots \\ \vec{o_r} \end{bmatrix}$$

where r is the row number. The following dotprod_row operators are available in the Matrix class:

- M.dotprod_row(Matrix& D, unsigned r, vector& v) which populates the passed vector v with the dot product of Matrix M and the transposed row r in matrix D
- M.dotprod_row(Matrix& D, unsigned r) which returns a new Matrix object with the dot product of Matrix M and the transposed row r in matrix D
- M.dotprod_row(Matrix& D, unsigned r, vector& v, unsigned *begin*, unsigned *end*) which populates the range in the passed vector v[*begin*] to v[*end*] with the dot product of Matrix M and the transposed row r in matrix D

Examples

```
// Matrix A and D both have 3 columns, vector v1 has 3 elements
A.dotprod_row( D, 3, v1 ); // v1 now contains the dot product
                           // of A and the transpose of D's 3rd row

vector< double > v2;
v2 = A.dotprod_row( D, 3 ); // v2 now contains the dot product
                            // of A and the transpose of D's 3rd row

// Matrix A has 4 rows, vector v3 has 5 elements
A.dotprod_row( D, 3, v3, 0, 3 ); // v3[0] through v3[3] now
                                 // contains the dot product of A
                                 // and the transpose of D's 3rd row
```

Notes

Both matrices *must have* the same number of columns. If a vector is passed, it *must have* the same number of elements as the matrices have columns. Because M.dotprod_row(D, r) must construct a new vector object, it is less efficient than M.dotprod_row(D, r, v).

5.2.17 Outer product

The *outprod* method is available in the Matrix class to populate a Matrix with the outer product of 2 vectors. For example, given 2 vectors $\vec{p}$ and $\vec{q}$, e.g.:

$$\vec{q} = \begin{bmatrix} q_1 \\ q_2 \\ \vdots \\ q_m \end{bmatrix}, \vec{p} = \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_n \end{bmatrix}$$

The outer product $\vec{q}\vec{p}^t$ is:

$$\vec{q}\vec{p}^t = \begin{bmatrix} q_1p_1 & q_1p_2 & \cdots & q_1p_n \\ q_2p_1 & q_2p_2 & \cdots & q_2p_n \\ \vdots & \vdots & \vdots & \vdots \\ q_mp_1 & q_mp_2 & \cdots & q_mp_n \end{bmatrix}$$

The outer product method is called as in:
M.outprod(vector& q, vector& p)

Example

```
// Make some vectors
double array1[ 4 ] = { 0, 1, 2, 3 },
   array2[ 5 ] = { 0, 1, 2, 3, 4 };
vector< double > vec1( array1, array1 + 4 ),
   vec2( array2, array2 + 5 );

// Make the right size Matrix
Matrix< double > U( 4, 5 );

// Populate the Matrix with the outer product of the vectors
U.outprod( vec1, vec2 );
```

Notes

The number of elements in the first vector argument *must equal* the number of Matrix rows, and the number of elements in the second vector argument *must equal* the number of Matrix columns.

5.2.18 Function passing

The following methods allow functions to be applied to a Matrix element by element:

- func(Matrix& M_{in}, function f, Matrix& M_{out}): applies function f element by element to Matrix M_{in}, populating Matrix M_{out} with the result.
- func(Matrix& M, function f): applies function f element by element to Matrix M, and returns a new Matrix object.

Example

```
\index{Transfer function!Sigmoidal}

#include <vector>
#include <math.h>
```

```
using namespace std;

#include "vector_ops.h"

// Define sigmoidal function as a class which extends
// unary_function.  This is necessary for algorithm::transform.
class sigmoidal : public unary_function< double, double >
{
   public:
      inline double operator()( const double& x )
      {
         return 1.0 / ( 1.0 + exp( -x ) );
      }
};

int main() {
   // Make some Matrices
   Matrix < double > M1( 4, 3, 0.5 ), M2( 4, 3, 0.0 ),
      M3( 4, 3, 2.0 ), M4;

   // Apply sigmoidal function to each element of M1
   // Populate M2 with the result
   func( M1, sigmoidal(), M2 );

   // Apply sigmoidal function to a Matrix, with itself as result
   func( M3, sigmoidal(), M3 );

   // Apply sigmoidal function to each element of M1
   // After this function call, M4 will contain the result
   M4 = func( M1, sigmoidal() );

   return 0;
}
```

Notes

The passed function *must* be defined as a class extension of unary_function as demonstrated in the example. Common functions useful for neural computation (e.g. sigmoidal) are packaged with neUROn2++.

If a Matrix to be populated is passed, it *must have* the same number of rows and columns as the input Matrix. As func(Matrix& M, function f) must construct a new vector, it is less efficient than func(Matrix& M_{in}, function f, Matrix& M_{out}).

5.2.19 Randomization

The method M.random(double n) populates a double precision Matrix M with double precision random numbers between $-n$ and $+n$.

Example

```
Matrix< double > Z( 6, 5 ); // Make a 6 x 5 Matrix
// Populate it with random numbers between -0.5 and +0.5,
//    and print it out
cout << "The random matrix is " << Z.random( 0.5 );
```

Notes

This method is only coded for Matrices of type double, e.g. Matrix< double >.

5.2.20 Loading a Matrix from a file

The following methods will load a matrix from a file:

- M.loadfile(string& $filename$, unsigned c): loads Matrix M from a file named "$filename$" with c columns. The Matrix will be resized if necessary, and if the file does not exist, the user will be queried until a good filename is entered.
- M.loadfile(bool *report*, string& $filename$): loads Matrix M from a file named "$filename$". Delimeters are any whitespace character, or any character that is non-numeric (+, -, e and E are considered numeric, as is a decimal point.) Rows are delimited by carriage returns in the file, and the number of columns is the maximum number of elements in any one row. Elements in unpopulated rows will be filled with 0, beginning from the *right* hand side of the row. If the bool flag *report* is true, a report will be printed indicating which rows were found to be unpopulated. The Matrix will be resized if necessary, and if the file does not exist, the user will be queried until a good filename is entered.

Examples

```
Matrix< double > A, B;
string filename;
unsigned cols;

cout << "What is the name of the data file? ";
cin >> filename;
cout << "How many columns does it have? ";
cin >> cols;

A.loadfile( filename, cols );
```

```
cout << "What is the name of another data file? ";
cin >> filename;

B.loadfile( true, filename ); // a report will be printed
```

5.2.21 Saving a Matrix to a file

The following method will save a matrix to a file:

- bool M.savefile(string& $filename$): saves Matrix M to a file named "$filename$", and returns a Boolean flag indicating whether the save operation was successful.

Example

```
Matrix< double > A( 5, 4, 3.14159 ); // fill a 5 x 4 Matrix with pi
A.savefile( "pi.txt" ); // and save it to a file named pi.txt
```

5.2.22 Resizing a Matrix

The method M.resize(unsigned r, unsigned c) resizes Matrix M to r rows and c columns.

Example

```
A.resize( 5, 4 ); // resize Matrix A to 5 rows and 4 columns
```

Notes

The following Matrix so resized is filled with **garbage**.

5.2.23 Clearing a Matrix

The method M.clear() clears Matrix M, setting its dimensions and data to zero.

Example

```
A.clear(); // clear Matrix A
```

5.2.24 Populating a Matrix with a value

The method M.fill(v) populates Matrix M with value v of its type.

Example

```
Matrix< double > A( 3, 2 ); // Make a Matrix of type double with
                            // 3 rows and 2 columns

A.fill( 3.14159 ); // and fill it with pi
```

5.2.25 Calculating the sums of columns of a Matrix

The method M.colsums() returns a new vector with each element the sum of the respective column in M. The method M.colsums(vector& v) returns the result in the passed vector v.

Example

```
#include <iostream>
#include <string>
#include "matrix.h"
#include "vector_ops.h"

string filename;
cout << "The file name is ";
cin >> filename;
Matrix< double > M( true, filename ); // load Matrix from file
vector< double > v;
v = M.colsums(); // v will contain the sum of the columns in M
cout << "Sum of columns = " << v << endl;
```

Note

If the vector to be returned is passed as an argument, its size *must* be the same as the number of columns in the Matrix.

5.2.26 Calculating the sum of squares of a Matrix elements

The method M.squared() returns a the value of the sum of squares of Matrix elements.

Example

```
Matrix< double > M; // M is then populated with double values...
double sum;
sum = M.squared();
```

5.2.27 Calculating the maximum absolute value of Matrix elements

The method M.maxabs() returns the maximum absolute value of Matrix elements.

Example

```
Matrix< double > M; // M is then populated with double values...
double max;
max = M.maxabs();
```

5.2.28 Finding the row elements with value 1 in a Matrix

The method (bool) M.rowindex(vector< unsigned >& v) populates a passed (unsigned) vector v with each vector element representing in which Matrix column each of the Matrix row elements was 1, takes passed vector as argument, returns true if successful.

Example

```
vector< unsigned > v( M.rows() );
if ( M.rowindex( v ) )
   cout << v << endl;
else
   cout << "can't do it: goofy Matrix." << endl;
```

Note

The passed Matrix must consist of rows that have a single row element with a value of 1, and the remainder with values of 0. For example, the Matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

would return true and populate the passed vector with

$$\begin{bmatrix} 0 \\ 2 \\ 1 \end{bmatrix}$$

whereas Matrices

$$\begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 3 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

would all return false.
The size of the vector *must* be the same as the number of rows in the Matrix.

5.2.29 Converting a Matrix to a vector, and a vector to a Matrix

A Matrix may be coverted row by row to a vector, e.g. Matrix

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

would be converted to vector

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 \end{bmatrix}$$

A vector may also be converted to a Matrix. Methods to perform the conversion include:

- M.toVector(vector& v): converts Matrix M to passed vector v which is populated with the result, and returns a reference to vector v
- M.toVector(): converts Matrix M to a new vector, and returns a reference to the new vector
- toMatrix(Matrix& M, vector& v): converts vector v to passed Matrix M which is populated with the result, and returns a reference to Matrix M
- toMatrix(vector& v, unsigned r, unsigned c): converts vector v to a new Matrix M with r rows and c columns, and returns a reference to the new Matrix

Example

```
// Make a Matrix as an example, and print it out
Matrix< double > A( 4, 3 );
for ( unsigned i = 0; i < 4; i++ )
```

```
    for ( unsigned j = 0; j < 3; j++ )
        A( i, j ) = 3 * i + j + 1;
cout << "Matrix A: " << A << endl;

// Convert the Matrix to vectors
vector< double > v1( 12 ), v2;
A.toVector( v1 );
cout << "toVector passed existing vector: " << v1 << endl;
v2 = A.toVector();
cout << "toVector passed new vector: " << v2 << endl;

// Convert the vectors back to Matrices
Matrix< double > B( 4, 3 ), C;
toMatrix( B, v2 );
cout << "back to an existing Matrix: " << B << endl;
C = toMatrix( v2, 4, 3 );
cout << "back to a new Matrix: " << C << endl;
```

Notes

If a vector is passed as an argument to toVector, its size *must be* the same as the product of the rows and the columns of the Matrix. If a Matrix is passed as an argument to toMatrix, the product of its rows and columns *must be* the same as the size of the vector which is also passed. If a Matrix is not passed as an argument to toMatrix, the product of the 2^{nd} and 3^{rd} arguments *must be* the same as the size of the passed vector.

5.2.30 First order covariance of a Matrix

The method M.covariance() returns a new Matrix with the first-order covariance Matrix of M. The method M.covariance(Matrix< double >& N) populates the passed Matrix N with the result, and returns a reference to Matrix N.

Example

```
#include <iostream>
#include <string>
#include "matrix.h"

string filename;
cout << "The file name is ";
cin >> filename;
Matrix< double > Data( true, filename ), // load data Matrix from file
   C; // for the covariance Matrix

C = Data.covariance();
```

```
cout << "Covariance Matrix = " << C << endl;
```

Notes

The covariance method is only coded for Matrices of type < double >. If the Matrix to be populated with the result is passed as an argument, it's dimensions must be $Col \times Col$, where Col is the number of columns in the data Matrix.

5.2.31 Matrix inverse and determinant

The following methods calculate the inverse and determinant of a Matrix:[1]

- M.inverse(Matrix< double >& I): calculates the inverse of Matrix M by LU decomposition, populating Matrix I with the result, and returns a reference to Matrix I
- M.inverse(): calculates the inverse of Matrix M by LU decomposition, and returns a new Matrix with the result
- M.inverse(Matrix< double >& I, double& d): calculates the inverse of Matrix M by LU decomposition, populating Matrix I with the result, returns the determinant in the passed argument d, and returns a reference to Matrix I
- M.inverse(double& d): calculates the inverse of Matrix M by LU decomposition, returns the determinant in the passed argument d, and returns a new Matrix with the result
- M.determinant(): returns the determinant of Matrix M by LU decomposition
- M.inverse(Matrix< double >& I, unsigned *choice*): allows the user to force type of inverse calculation by setting *choice* to "GaussJordan," (default is LU decomposition) populating Matrix I with the result, and returns a reference to Matrix I
- M.inverse(unsigned *choice*): allows the user to force type of inverse calculation by setting *choice* to "GaussJordan" (default is LU decomposition), and returns a reference to the new Matrix

In the case of a singular Matrix, these methods throw exception Matrix< double >::Singular& e, where e.what() is "singular Matrix".

[1] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pages 36-48

Example

```
#include <iostream>
#include <string>
#include "matrix.h"

string inverse_filename;
cout << "The file name for Matrix inverse is ";
cin >> inverse_filename;
Matrix< double > I( true, inverse_filename ), J, K;
double determinant;

try
{
   // Test populating a passed Matrix
   J.resize( I.rows(), I.cols() ); // resize receiving Matrix to input
   Matrix
   cout << "The inverse of " << I << "is " << I.inverse( J ) << endl;

   // Test returning a new Matrix
   K = I.inverse();
   cout << "The inverse of " << I << "is " << K << endl;

   // Test populating a passed Matrix, and returning a determinant
   cout << "The inverse of " << I << "is " << I.inverse( J,
   determinant )
      << endl;
   // Note g++ requires this to be on a 2nd line, but MSVC++ does not:
   cout << "The determinant is " << determinant << endl;

   // Test returning a new Matrix with a determinant
   K.clear();
   K = I.inverse( determinant );
   cout << "The inverse of " << I << "is " << K
      << endl << "The determinant is " << determinant << endl;

   // Test returning just the determinant
   cout << "That determinant again is " << I.determinant() << endl;

   // Test populating a passed Matrix, and forcing type of inverse
   procedure
   cout << "The Gauss Jordan inverse of " << I << "is "
      << I.inverse( J, GaussJordan ) << endl;

   // Test returning a new Matrix and forcing type of inverse
   procedure
```

```
   K.clear();
   K = I.inverse( GaussJordan );
   cout << "The Gauss Jordan inverse of " << I << "is " << K << endl;

   // Examine the difference between LU decomposition and Gauss Jordan
   cout << "The difference between LU decomposition and Gauss Jordan
   inverse of "
      << I << "is " << I.inverse() - I.inverse( GaussJordan ) << endl;
}
catch ( Matrix< double >::Singular& e )
{
   cout << "Can't do it: " << e.what() << endl;
}
```

Notes

Use a try block with these methods to catch the Singular exception. If you don't, your program may bomb out if a singular Matrix is encountered.

These methods are only coded for Matrices of type < double >. Matrices to invert must be square, e.g. dimensions must be $Col \times Col$, where Col is the number of columns in the Matrix. If the Matrix to be populated with the result is passed as an argument, it's dimensions must be the same as the Matrix object.

5.2.32 Eigenvalues

The calculation of the eigenvalues of a symmetric matrix is the only current instance of using pre-existing code and libraries, as these calculations can be quite intensive, and are currently beyond the scope of the current program. Gaurav Bansal ported the eigenvalue routines for a symmetric matrix from the Gnu Scientific Library (GSL) in 2003, recoding those portions of GSL to integrate with the matrix specification in neUROn2++. **As a result, do not expect GSL functionality beyond eigenvalue routines in the neUROn2++ environment.**

The best way to understand calling eigenvalue routines in neUROn2++ is to study the given example. A GSL matrix is first created from a neUROn2++ matrix using *gsl_matrix_view*, as is a GSL vector using *gsl_vector*. A GSL workspace is then created using *gsl_eigen_symm_workspace* for symmetric matrix eigenvalue calculations, and the eigenvalues are calculated using *gsl_eigen_symm*. Finally, the GSL vector is returned to an STL vector containing the eigenvalues using *gsl_vector_ptr*, and memory is freed using *gsl_eigen_symm_free*.

For those interested in GSL, it may be found at http://sources.redhat.com/gsl/ as of the time of this writing (August 11, 2003).

Example

```
#include "gsl_eigen.h"

// Create a symmetric matrix using our matrix class, and fill with
random values
Matrix< double > m1( 4, 4 );
m1.random( 5 );

// Check to see that we perform eigenvalue routines to symmetric
routines only
assert ( m1.rows() == m1.cols() );

// Create a gsl matrix from our matrix--begin function is cautioned to
use but
//    here we need the value as double*
gsl_matrix_view m = gsl_matrix_view_array( m1.begin(), 4, 4 );

// Create a gsl vector to store all the eigenvalues
gsl_vector *eval = gsl_vector_alloc( 4 );

// Create gsl workspace for eigenvalue calculations
gsl_eigen_symm_workspace *w = gsl_eigen_symm_alloc( 4 );

// Calculate eigenvalues
gsl_eigen_symm( &m.matrix, eval, w );

// Create vector< double > from gsl vector
vector< double > eigenv( gsl_vector_ptr( eval, 0 ), gsl_vector_ptr(
eval, 4 ) );

// Free workspace
gsl_eigen_symm_free( w );

// Output results
cout << "Matrix: " << m1 << endl << "Eigenvalues are: " << eigenv <<
endl;
cout << "Maximum value = " << maxabs( eigenv ) << endl;
cout << "Minimum value = " << minabs( eigenv ) << endl;
```

Notes

All of the files needed are contained in the directory "gsl". If you are working in a pre-existing environment, you do not need to be concerned about searching for directory paths, they should be there already. However, if you are building a new environment in g++, you'll need to add search functionality in your

Makefile. An example Makefile which searches for code inside of an "msvc" directory, and the gsl subdirectory within it is:

```
vpath %.cpp msvc msvc/gsl
vpath %.h msvc msvc/gsl
vpath %.c msvc msvc/gsl

PROJ = neuron
CPP = g++
CPPFLAGS = -g -O2 -Wall
LDFLAGS = -lm

OBJS = $(PROJ).o ...

default: $(PROJ)

$(PROJ): $(OBJS)
    $(CPP) $(CPPFLAGS) $(OBJS) -Imsvc -Imsvc/gsl -o $(PROJ) $(LDFLAGS)
        -Lmsvc -Lmsvc/gsl -llibgsl -llibgslcblas

$(PROJ).o: $(PROJ).cpp
    $(CPP) $(CPPFLAGS) -Imsvc/gsl -o $@ -c $^
...
```

If you're using a pre-existing MSVC project, the following settings should already be present in your .dsw workspace file. However, for a new MSVC project, you must instruct it to look for the gsl include path, the link path, and the linked libraries libgsl.lib libgslcblas.lib. Choose the "Project→Settings" menu choice, then set the following:

- C/C++ tab
  - Category: Preprocessor
    - Additional Include Directories:
      - gsl
- Link tab
  - Category: Input
    - Additional Library Path:
      - gsl
- Link tab
  - Category: Input
    - Object/Library Modules: add
      - libgsl.lib libgslcblas.lib

5.3 An example of the Matrix and vector classes

The following example constructs a simple neural network with 3 hidden nodes on one layer to train the exclusive-or problem. The file "xor.set" contains the exclusive-or dataset:

```
-0.9 -0.9 0.1
-0.9 0.9 0.9
0.9 -0.9 0.9
0.9 0.9 0.1
```

The following is the program:[2]

```
#include <iostream>
#include <iomanip>
#include <string>
#include <math.h>

#include "matrix.h"
#include "vector_ops.h"
#include "function_defs.h"

using namespace std;

int main()
{
   // Assumes a file named xor.set
   string filename = "xor.set";

   // Load the XOR dataset from the file xor.set
   Matrix< double > dataset( filename, 3 ); // it has 3 columns

   // Get the input Matrix In from the dataset as a submatrix
   Matrix< double > In = dataset.submatrix( 0, 3, 0, 1 );

   // Get the output vector y from the dataset's last column
   vector< double > y = dataset.col( 2 );

   // Make the 3 x 2 hidden weight Matrix hW and the hW update Matrix
   hWup
   Matrix< double > hW( 3, 2 ), hWup( 3, 2 );
   hW.random( 0.5 ); // randomize hW

   // Make the 2 element vector for the inputs I
```

[2]References are to James A. Freeman and David M. Skapura, Neural Networks, Algorithms, Applications and Programming Techniques, Addison-Wesley, 1991, ISBN 0-201-51376-5

```
// and the 3 element vector hidden output hO
// and the 3 element vector output weights oW
vector< double > I( 2 ), hO( 3 ), oW( 3 );
nvec::random( oW, 0.5 ); // randomize oW

// o is the neural net's output, o_err the output error term,
// eta is the learning rate
double o, o_err, eta = 0.05;

// h_err is the vector with hidden error terms
vector< double > h_err( 3 );

for ( unsigned iteration = 0; iteration < 1000000; iteration++ )
{
   for ( unsigned example = 0; example < 4; example++ )
   {
      // Get a single row from the input Matrix
      // (Freeman & Skapura p. 102, #1)
      In.row( example, I );

      // Take the dot product of the hidden weights Matrix hW and
      the
      // transpose of a row of the input Matrix I, and apply the
      // sigmoidal function to the resulting vector
      // (Freeman & Skapura p. 102, #2 & #3)
      func( hW.dotprod( I, hO ), sigmoidal(), hO );

      // Take the dotproduct of the hidden output vector and the
      output
      // weights vector, and apply the sigmoidal function to the
      result
      // (Freeman & Skapura p. 102, #4 & #5)
      o = sigmoidal()( dotprod( hO, oW ) );

      // Calculate the error term for the output unit
      // (Freeman & Skapura p. 102, #6)
      o_err = ( y[ example ] - o ) * d_sigmoidal()( o );

      // Calculate the error terms for the hidden units
      // (Freeman & Skapura p. 102, #7)
      // Note that using func which takes the output vector as the
      // last argument, and *= instead of *, is the most efficient
      way
      ( func( hO, d_sigmoidal(), h_err ) *= o_err ) *= oW;

      // Update the output weights, note again the use of *= for
```

```
         efficiency
         // (Freeman & Skapura p. 102, #8)
         oW += ( hO *= ( eta * o_err ) );

         // Update the hidden weights
         // (Freeman & Skapura p. 102, #9)
         hW += ( hWup.outprod( h_err, I ) *= eta );
      }

      // Every 1000 iterations, print out the error
      if ( iteration % 1000 == 0 )
         cout << setw( 8 ) << iteration << ": LMS error = " <<
         0.5 * ( y[ 3 ] - o ) * ( y[ 3 ] - o ) << endl;
   }

   return 0;
}
```

Chapter 6

Utility methods

The following are contained in the header file "utility.h", and contain global utility methods. These methods throw exception util::utilErr& e, where e.what() specifies the module from which the error originated, and the nature of the error.

6.1 Random number generator

namespace util

The following utilities seed or generate random numbers of double precision:

- d_random(): seeds the random number generator with an internal timestamp
- d_random(double n): returns a double precision random number between $-n$ and $+n$
- d_random(double *lower*, double *upper*): returns a double precision random number between *lower* and *upper*

6.1.1 Examples

```
util::d_random(); // seed the generator

double x, y;

x = util::d_random( 0.5 ); // Generates a random number between -0.5
and +0.5

y = util::d_random( 1.0, 10.0 ); // Between 1.0 and 10.0
```

6.2 Timestamp

The timestamp(unsigned x) utility inserts a timestamp of the form HHHH:MM:SS (where H is hour, M is minute, S is second) into the output stream, given x seconds.

6.2.1 Example

```
unsigned start = time( 0 ); // gets the current time in seconds

// Do something for a while
for ( i = 0; i < 100000; i++ )
   cout << ".";

// Calculate the elapsed time
unsigned increment = time( 0 ) - start;

// Print out how long it took
cout << endl << "That took " << timestamp( increment ) << endl;
```

6.3 Round

namespace util

The following method will round a number to a specified number of significant digits:

- round(double x, unsigned d): rounds the passed value x to d significant digits, returns the rounded value

6.3.1 Example

```
double x;
unsigned d;

cout << "x = ";
cin >> x;
cout << "d = ";
cin >> d;
cout << "round( x, d ) = " << util::round( x, d ) << endl;
```

6.3.2 Notes

Zero significant digits is not allowed. Rounds positive numbers up, and negative numbers down, e.g. util::round(30.9999, 2) = 31, and util::round(-30.9999, 2) = -31.

6.4 Queries with bounds checking

namespace util

The *askI and askD* methods allow user queries with bounds checking. Specifications include:

- askI(string& *query*, int *lower*) which queries the user with the string *query* repeatedly if necessary until the user enters an integer value >= *lower*, and returns the value
- askI(string& *query*, int *lower*, int *upper*) which queries the user with the string *query* repeatedly if necessary until the user enters an integer value >= *lower* and <= *upper*, and returns the value
- askD(string& *query*, double *lower*) which queries the user with the string *query* repeatedly if necessary until the user enters a value >= *lower*, and returns the value
- askD(string& *query*, double *lower*, double *upper*) which queries the user with the string *query* repeatedly if necessary until the user enters a value >= *lower* and <= *upper*, and returns the value

6.4.1 Example

```
int i, h;
double d;

i = util::askI( "How many input nodes? ", 1 );
h = util::askI( "How many hidden nodes? ", 1, 10 );
d = util::askD( "What is the learning rate? ", 0, 1 );
```

6.5 Query for yes/no answer

namespace util

The *askYesNo* method allows user queries with yes/no answers. Specifications include:

- askYesNo(string& *query*) queries the user with the string *query* repeatedly if necessary until the user enters "yes", "y", "no" or "n", and returns boolean true for yes, false for no

6.5.1 Example

```
bool yesNoAnswer;
yesNoAnswer = util::askYesNo( "Shall we clean the closet? " );
```

6.6 Query for existing file name

namespace util

The *getGoodFile* method tests for a file name to see if it actually exists, then queries for an existing file name repeatedly until a file that actually exists is entered, and returns that file name as a *string*:

- getGoodFile(string& $filename$) tests $filename$ to see if it actually exists, then if it doesn't, queries the user for a file name repeatedly if necessary until the user enters an existing file name, and returns it as a string.

6.6.1 Example

```
#include <iostream>
#include <fstream>
#include <string>
#include "utility.h"

string filename; // for the name of a loaded file

cout << "What is the name of your data file (it must exist)? ";
cin >> filename;

// Open the file as read-only, and associate it with the 'label'
inputFile
ifstream inputFile( ( util::getGoodFile( filename ) ).c_str(), ios::in
);
```

6.7 Remove a carriage return

namespace util

The *chopEndl* method removes ASCII 13, generally a carriage return, from the end of a string if it exists, and returns a reference to the string.

- chopEndl(string& *stringObj*) removes the last character from *stringObj* if it is ASCII 13, and returns a reference to *stringObj*.

6.7.1 Example

```
#include <iostream>
#include <fstream>
#include <string>
#include "utility.h"

string lineString; // string to hold the 1st line of file
```

```
// Open the file "test.txt" for input
ifstream loadfile( "test.txt", ios::in );

getline( loadfile, lineString ); // get the 1st line of file
util::chopEndl( lineString ); // remove <cr> from end of string if
exists

loadfile.close(); // close the input file
```

6.8 Parse a variable definition string

namespace util

- variable_parse(string& *stringObj*) parses *stringObj* for variable representations from nodes (see below), and returns a vector of vector of unsigned representing the nodes.
- variable_parse(ifstream& $fileObj$) parses the file $fileObj$ for variable representations from nodes (see below), and returns a vector of vector of unsigned representing the nodes.

The *variable_parse* method parses a string passed as the argument to determine variable representation from nodes, and returns a *new* vector of vector of unsigned. In the string,

- **;** delimits variables
- **,** delimits nodes
- **-** specifies a range of nodes
- spaces are ignored

For example, passed string "0-3,5; 4,8; 6,7" returns vector of vector of unsigned (each line is a vector):

```
0 1 2 3 5
4 8
6 7
```

util::utilErr exceptions thrown by this method include e.what()

- "util variable_parse error: bad character found at string position *position* (*character*)"
- "util variable_parse error: too many hyphens"
- "util variable_parse error: hyphen without a number on both ends "
- "util variable_parse error: 1st number in range greater than 2nd "

- "util variable_parse error: numbers in range are the same"
- "util variable_parse error: missing zero"
- "util variable_parse error: duplicated positions"
- "util variable_parse error: missing positions"

6.8.1 Example

```
#include "stdafx.h" // if you're using MSVC

// Because, apparently, vector< vector< unsigned > > symbol names
exceeds 255
//    characters in length, and MSCV will error with that (ugly!)
#ifdef _MSC_VER // MSVC
#pragma warning (disable: 4786)
#endif

#include <iostream>
#include <fstream>
#include <vector>
#include <string>

#include "vector_ops.h"
#include "utility.h"

using namespace std;

int main()
{
   // The returned structures
   vector< vector< unsigned > > parse_numbers, parse2;

   string parse_test = "0-3,5; 4,8; 6,7"; // a test string

   // Parse the string
   try
   {
      parse_numbers = util::variable_parse( parse_test );

      // Then print out the returned structure, a vector at a time
      cout << "Here it is: " << endl;
      unsigned pCount;
      for ( pCount = 0; pCount < parse_numbers.size(); pCount++ )
         cout << parse_numbers[ pCount ] << endl;
   }
```

```
   catch ( util::utilErr& e )
   {
      cout << e.what() << endl;
   }

   // Do the same from a file
   string reply;
   cout << "What is the name of the file? ";
   cin >> reply;
   ifstream inputFile( ( util::getGoodFile( reply ) ).c_str(), ios::in
   );

   try
   {
      parse2 = util::variable_parse( inputFile );
      cout << "Here it is: " << endl;
      for ( pCount = 0; pCount < parse2.size(); pCount++ )
         cout << parse2[ pCount ] << endl;
   }
   catch ( util::utilErr& e )
   {
      cout << e.what() << endl;
   }

   return 0;
}
```

Notes

If you're using MSVC, you *must* include in your code the following, as MSVC will give you error warnings for each vector< vector< unsigned > > reference:

```
#ifdef _MSC_VER // MSVC
#pragma warning (disable: 4786)
#endif
```

variable_parse(ifstream& *fileObj* **) will close and clear the file for you.** Parsing a file will support multiple line definitions of variables by concatenating the lines prior to parsing the resulting string. However, MSVC places a limit of 2,048 characters per string, and if compiling with MSVC, definitions containing more than 2,048 characters will not be supported.

6.9 Remove a variable from a vector of vector of unsigned data structure

namespace util

- remove_variable(vector< vector< unsigned > >& *variables*, unsigned *n*, vector< vector< unsigned > >& *result*) removes the subvector representing variable *n* from the vector of vector of unsigned data structure *variables*, populates *result* as a vector of vector of unsigned data structure representing the result, returns a reference to the resulting data structure.
- remove_variable(vector< vector< unsigned > >& *variables*, unsigned *n*) as above, except returns a new vector of vector of unsigned data structure with the result.

The *remove_variable* method removes a single subvector, representing a single variable, from a vector of vector of unsigned data structure which represents a set of variables. As neUROn2++ variables may contain more than 1 input node (e.g. a variable may contain 2 nodes, the value of the variable itself, and whether or not that variable was present in the dataset), this method is used to remove a single variable from a set of variables.

6.9.1 Example

```
// Create a vector of vector of unsigned with some variables
string parse_test = "0-3,5; 4,8; 6,7";
vector< vector< unsigned > > parse_numbers, parse_new, parse_new2;
parse_numbers = util::variable_parse( parse_test );

// Test remove_variable
parse_new = parse_numbers;
unsigned position;
position = util::askI( "What is the position to remove? ", 0 );
util::remove_variable( parse_numbers, position, parse_new );
cout << "After I've removed variable " << position << ": " << endl;
cout << "Here is the original: " << parse_numbers << endl;
cout << "Here is the result: " << parse_new << endl;

parse_new2 = util::remove_variable( parse_numbers, position );
cout << "Making a new data structure : " << endl;
cout << "Here is the original: " << parse_numbers << endl;
cout << "Here is the result: " << parse_new2 << endl;
```

Notes

If you're using MSVC, you *must* include in your code the following, as MSVC will give you error warnings for each vector< vector< unsigned > > reference:

```
#ifdef _MSC_VER // MSVC
#pragma warning (disable: 4786)
#endif
```

Chapter 7

Statistical methods

The following are contained in the header file "stats.h", and implemented in "stats.cpp", and contain global statistical methods. Those that are not classes are contained within namespace "stats", and throw exception stats::statsErr& e, where e.what() specifies the module from which the error originated, and the nature of the error.

7.1 Log gamma

namespace stats

The following method returns the natural logarithm of the gamma function $\log\Gamma(a)$[1] where

$$\Gamma(z) = \int_0^\infty t^{z-1}e^{-t}dt \tag{7.1}$$

- gammln(double a): returns the double value $\log\Gamma(a)$

7.1.1 Example

```
double x;
cout << "For log gamma, x = ";
cin >> x;
cout << "Log gamma " << x << " = " << stats::gammln( x ) << endl;
```

7.2 Incomplete gamma by fraction

namespace stats

[1] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, p. 214

The following method returns the incomplete gamma function

$$P(a,x) \equiv \frac{\gamma(a,x)}{\Gamma(a)} \equiv \frac{1}{\Gamma(a)} \int_0^x e^{-t} t^{a-1} dt \quad (a > 0) \tag{7.2}$$

by continued fraction development:[2]

$$\Gamma(a,x) = e^{-x} x^a \left(\frac{1}{x + \frac{1-a}{1 + \frac{1}{x + \frac{2-a}{1 + \frac{2}{x + \cdots}}}}} \right) \quad (x > 0) \tag{7.3}$$

- gcf(double a, double x): returns the double value $\Gamma(a,x)$

7.2.1 Example

```
double a, x;
cout << "For gcf, a = ";
cin >> a;
cout << "x = ";
cin >> x;
try {
     cout << "gcf( " << a << " , " << x << " ) = " << stats::gcf( a, x
     );
} catch ( stats::statsErr& e ) {
     cout << e.what();
}
cout << endl;
```

7.3 Incomplete gamma by series

namespace stats

The following method returns the incomplete gamma function (equation 7.2) by series: [3]

$$\gamma(a,x) = e^{-x} x^a \sum_{n=0}^{\infty} \frac{\Gamma(a)}{\Gamma(a+1+n)} x^n \tag{7.4}$$

- gser(double a, double x): returns the double value $\gamma(a,x)$

[2] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, p. 219

[3] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 218–219

7.3.1 Example

```
double a, x;
cout << "For gser, a = ";
cin >> a;
cout << "x = ";
cin >> x;
try {
     cout << "gser( " << a << " , " << x << " ) = " << stats::gser( a,
     x );
} catch ( stats::statsErr& e ) {
     cout << e.what();
}
cout << endl;
```

7.4 Incomplete gamma composite

namespace stats

The following method returns the incomplete gamma function (equation 7.2) by either continued fraction or series, based on speed of convergence:[4]

- gammp(double a, double x): returns the double value $P(a, x)$

7.4.1 Example

```
double a, x;
cout << "For gammp, a = ";
cin >> a;
cout << "x = ";
cin >> x;
try {
     cout << "gammp( " << a << " , " << x << " ) = " << stats::gammp(
     a, x );
} catch ( stats::statsErr& e ) {
     cout << e.what();
}
cout << endl;
```

7.5 Incomplete gamma complement

namespace stats

[4] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, p. 218

The following method returns the complement of the incomplete gamma function

$$Q(a,x) \equiv 1 - P(a,x) \tag{7.5}$$

(see equation 7.2 for the definition of $P(a,x)$): [5]

- gammq(double a, double x): returns the double value $Q(a,x)$

7.5.1 Example

```
double a, x;
cout << "For gammq, a = ";
cin >> a;
cout << "x = ";
cin >> x;
try {
     cout << "gammq( " << a << " , " << x << " ) = " << stats::gammq(
     a, x );
} catch ( stats::statsErr& e ) {
     cout << e.what();
}
cout << endl;
```

7.6 Beta function

namespace stats

The following method returns the beta function $B(z,w)$ [6] where

$$B(z,w) = B(w,z) = \int_0^1 t^{z-1}(1-t)^{w-1}dt = \frac{\Gamma(z)\Gamma(w)}{\Gamma(z+w)} \tag{7.6}$$

- beta(double z, double w): returns the double value $B(z,w)$

7.6.1 Example

```
double z, w;
cout << "For beta function, z = ";
cin >> z;
cout << "w = ";
cin >> w;
cout << "Beta = " << stats::beta( z, w ) << endl;
```

[5] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, p. 218

[6] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 215–216

7.7 Incomplete Beta function

namespace stats

The following method returns the incomplete beta function $I_x(a,b)$ [7] where

$$I_x(a,b) \equiv \frac{B_x(a,b)}{B(a,b)} \equiv \frac{1}{B(a,b)} \int_0^x t^{a-1}(1-t)^{b-1} dt \tag{7.7}$$

Note that the continued fraction representation is also available, but as a utility function should not be necessary to call directly:

$$I_x(a,b) = \frac{x^a(1-x)^b}{aB(a,b)} = \left(\frac{1}{1 + \frac{d_1}{1+\frac{d_2}{1+\cdots}}} \right) \tag{7.8}$$

where

$$d_{2m+1} = -\frac{(a+m)(a+b+m)x}{(a+2m)(a+2m+1)} \tag{7.9}$$

and

$$d_{2m} = \frac{m(b-m)x}{(a+2m-1)(a+2m)} \tag{7.10}$$

- betai(double a, double b, double x): returns the double value $I_x(a,b)$
- betacf(double a, double b, double x): returns the double value $I_x(a,b)$ by continued fraction evaluation, used by betai

7.7.1 Example

```
double a, b, x;
cout << "For incomplete beta function, a = ";
cin >> a;
cout << "b = ";
cin >> b;
cout << "x = ";
cin >> x;
cout << "Incomplete beta = " << stats::betai( a, b, x ) << endl;
```

[7] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 226–228

7.8 Error functions

namespace stats

The following methods returns the error function (commonly referred to as *erf*) [8] and its complement

$$erf(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt = P(\frac{1}{2}, x^2) \tag{7.11}$$

$$erfc(x) \equiv 1 - erf(x) = \frac{2}{\sqrt{\pi}} \int_x^\infty e^{-t^2} dt = Q(\frac{1}{2}, x^2) \tag{7.12}$$

- erff(double x): returns the double value $erf(x)$
- erfc(double x): returns the double value $erfc(x)$
- erffc(double x): returns the double value $erfc(x)$ by Chebyshev fitting

7.8.1 Example

```
double x;
cout << "x = ";
cin >> x;
try {
     cout << "erf( " << x << " ) = " << stats::erff( x );
} catch ( stats::statsErr& e ) {
     cout << e.what();
}
cout << endl;
try {
     cout << "erfc( " << x << " ) = " << stats::erfc( x );
} catch ( stats::statsErr& e ) {
     cout << e.what();
}
cout << endl;
```

7.8.2 Notes

Note the 2 f s in erff, so that this is not confused with the innate compiler function erf.

7.9 Error functions inverses

namespace stats

[8] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 220–221

The following methods return the inverse of the error function and the inverse of the complement of the error function:[9]

- invErff(double y): returns the double value $erf^{-1}(y)$
- invErfc(double y): returns the double value $erfc^{-1}(y)$

7.9.1 Example

```
double y;
cout << "y = ";
cin >> y;
try {
     cout << "inverse erf( " << y << " ) = " << stats::invErff( y );
} catch ( stats::statsErr& e ) {
     cout << e.what();
}
cout << endl;
try {
     cout << "inverse erfc( " << y << " ) = " << stats::invErfc( y );
} catch ( stats::statsErr& e ) {
     cout << e.what();
}
cout << endl;
```

7.10 Gaussian distribution

namespace stats

The following method returns the unit Gaussian function

$$\varphi(z) = \frac{1}{\sqrt{2\pi}} e^{-z^2/2} \tag{7.13}$$

where given mean μ and standard deviation σ the following transformation $x \to z$ is made:

$$z \equiv \frac{x - \mu}{\sigma}$$

x, μ and σ may also be passed, where the Gaussian function is then

$$\varphi(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/(2\sigma^2)} \tag{7.14}$$

- gaussD(double z): returns the double value $\varphi(z)$
- gaussD(double x, double μ, double σ): returns the double value $\varphi(x, \mu, \sigma)$

[9] Algorithm for $erfc^{-1}(y)$ by Takuya Ooura © 1996, $erf^{-1}(y) = erfc^{-1}(1 - y)$

7.10.1 Example

```
double z, x, u, s;
cout << "for gaussD, z = ";
cin >> z;
cout << "gaussD = " << stats::gaussD( z ) << endl;
cout << "for gaussD, x = ";
cin >> x;
cout << "u = ";
cin >> u;
cout << "s = ";
cin >> s;
cout << "gaussD = " << stats::gaussD( x, u, s ) << endl;
```

7.11 z Area

namespace stats

The following method returns the area under a unit Gaussian distribution

$$\Phi(z) \equiv \frac{1}{\sqrt{2\pi}} \int_0^z e^{-z^2/2} dz = \frac{1}{2} erf(\frac{z}{\sqrt{2}}) \tag{7.15}$$

- Zarea(double z): returns the double value $\Phi(z)$

7.11.1 Example

```
double z;
cout << "for z Area, z = ";
cin >> z;
try {
     cout << "z Area = " << stats::Zarea( z );
} catch ( stats::statsErr& e ) {
     cout << e.what();
}
cout << endl;
```

7.12 z Area inverse

namespace stats

The following method returns the inverse of the area under a unit Gaussian distribution

$$z = Z(A) = \Phi^{-1}(A) = \sqrt{2} erf^{-1}(2A - 1) \tag{7.16}$$

- invZarea(double A): returns the double value $Z(A) = \Phi^{-1}(A)$

7.12.1 Example

```
double A;
cout << "for invZarea, A = ";
cin >> A;
try {
     cout << "invZarea ( " << A << " ) = " << stats::invZarea( A ) <<
     endl;
} catch ( stats::statsErr& e ) {
     cout << e.what();
}
```

7.13 Population metrics

The *Population* class provides statistical metrics such as mean, variance, standard deviation, skew and kurtosis. It's constructor takes vector< double > as its argument, representing the population from which to generate metrics. Errors are caught by *PopulationErr* (see below for examples).

- Population(vector< double > x) defines a population from vector< double > x from which to generate statistical metrics

Accessors include:

- P.mean(): returns the mean
- P.var(): returns the variance
- P.std(): returns the standard deviation
- P.skew(): returns the skew
- P.kurtosis(): returns the kurtosis

7.13.1 Example

```
double numbers[ 4 ] = { 1.0, 2.0, 3.0, 4.0 }; // define the dataset
vector< double > dset( numbers, numbers + 4 );
Population P( dset ); // define the population from the dataset
cout << "average = " << P.mean() << endl;
cout << "variance = " << P.var() << endl;
cout << "standard deviation = " << P.std() << endl;
try {
   cout << "skew = " << P.skew() << endl;
   cout << "kurtosis = " << P.kurtosis() << endl;
} catch ( Population::PopulationErr& e ) {
   cout << e.what() << endl;
}
```

7.13.2 Notes

The only allowable constructor takes vector< double > as an argument, and there is no copy constructor or overloaded = operator. The vector< double > argument *must* contain 2 or more elements.

7.14 F and Student's t

namespace stats

The following methods return probabilities for the F-test that 2 sample variances are different (p=1.0 if they are equal), and Student's t- tests that 2 samples are different. All take 2 vectors of double as the samples S_1 and S_2.

- ftest(vector< double > S_1, vector< double > S_2): returns a double value with the probability for an F-test.
- ttest(vector< double > S_1, vector< double > S_2): returns a double value with the probability for a Student's t- test with equal sample variance.
- tutest(vector< double > S_1, vector< double > S_2): returns a double value with the probability for a Student's t- test with unequal sample variance.
- tptest(vector< double > S_1, vector< double > S_2): returns a double value with the probability for a paired Student's t-test.

Note that[10]

- For the F-test,
$$\alpha = I_{\frac{\nu_2}{\nu_2+\nu_1 F}}(\frac{\nu_2}{2}, \frac{\nu_1}{2}) \tag{7.17}$$
(see equation 7.7), where $F = \frac{\sigma_1^2}{\sigma_2^2}$ ($\sigma_1^2 > \sigma_2^2$), and $\nu_1 = N_1 - 1$ and $\nu_2 = N_2 - 1$
- For Student's distribution, $\alpha = 1 - A(t|\nu)$, where
$$A(t|\nu) = \frac{1}{\sqrt{\nu}B(\frac{1}{2}, \frac{\nu}{2})} \int_{-t}^{t} (1 + \frac{x^2}{\nu})^{\frac{\nu+1}{2}} dx \tag{7.18}$$
(see equation 7.6) and is related to the incomplete beta function by
$$A(t|\nu) = 1 - I_{\frac{\nu}{\nu+t^2}}(\frac{\nu}{2}, \frac{1}{2}) \tag{7.19}$$
(see equation 7.7)

[10] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 228–229, 615–619

– For Student's t with equal sample variance,

$$t = \frac{\mu_1 - \mu_2}{s_D} \tag{7.20}$$

where

$$s_D = \sqrt{\frac{\sum_i^{N_1}(x_i - \mu_1)^2 + \sum_i^{N_2}(x_i - \mu_2)^2}{N_1 + N_2 - 2}\left(\frac{1}{N_1} + \frac{1}{N_2}\right)} \tag{7.21}$$

and $\nu = N_1 + N_2 - 2$

– For Student's t with unequal sample variance,

$$t = \frac{\mu_1 - \mu_2}{\sqrt{\frac{\sigma_1^2}{N_1} + \frac{\sigma_2^2}{N_2}}} \tag{7.22}$$

and

$$\nu = \frac{\left(\frac{\sigma_1^2}{N_1} + \frac{\sigma_2^2}{N_2}\right)^2}{\frac{\left(\frac{\sigma_1^2}{N_1}\right)^2}{N_1 - 1} + \frac{\left(\frac{\sigma_2^2}{N_2}\right)^2}{N_2 - 1}} \tag{7.23}$$

– For paired Student's t,

$$t = \frac{\mu_1 - \mu_2}{s_D} \tag{7.24}$$

where

$$s_D = \sqrt{\frac{\sigma_1^2 + \sigma_2^2 - 2Cov(x1, x2)}{N}} \tag{7.25}$$

where

$$Cov(x1, x2) \equiv \frac{1}{N-1}\sum_{i=1}^{N}(x_{i1} - \mu_1)(x_{i2} - \mu_2) \tag{7.26}$$

and $\nu = N - 1$, where N is the *number of pairs*.

7.14.1 Example

```
double d1[ 9 ] = { 3, 4, 5, 8, 9, 1, 2, 4, 5 };
double d2[ 9 ] = { 6, 19, 3, 2, 14, 4, 5, 17, 1 };
vector< double > v1( d1, d1 + 9 ), v2( d2, d2 + 9 );

cout << "F-test = " << stats::ftest( v1, v2 ) << endl;
cout << "Student's t for same variance = " << stats::ttest( v1, v2 ) <
< endl;
cout << "Student's t for unequal variance = " << stats::tutest( v1, v2
) << endl;
cout << "Paired Student's t = " << stats::tptest( v1, v2 ) << endl;
```

7.14.2 Notes

Note that in all cases, sample (passed vector) sizes *must be* > 1, and for the paired Student's t-test, they *must be* equal.

7.15 Chi-square

namespace stats

The following method returns the probability p for df degrees of freedom and Chi-square value χ^2:

- pX2(unsigned df, double χ^2): returns a double value with the probability.

Note that the probability p for the Chi-square value χ^2 with df degrees of freedom is $Q(\frac{df}{2}, \frac{\chi^2}{2})$ (see equation 7.5).

7.15.1 Example

```
double x; // chi-square value
unsigned df; // degrees of freedom

// Query the user for the value and degrees of freedom
cout << "For chi-square, df = ";
cin >> df;
cout << "X2 = ";
cin >> x;

// Then calculate the probability p
try {
     cout << "p = " << stats::pX2( df, x );
} catch ( stats::statsErr& e ) { // catch a stats error
     cout << e.what();
}
cout << endl;
```

7.16 Kolmogorov-Smirnov statistic

namespace stats

The following method returns the Kolmogorov-Smirnov probability Q_{KS}[11]

$$Q_{KS}(\lambda) = 2\sum_{j=1}^{\infty}(-)^{j-1}e^{-2j^2\lambda^2} \tag{7.27}$$

- double probks(double λ): returns a double value with $Q_{KS}(\lambda)$.

[11] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pages 623-626

7.16.1 Example

```
double alam;
cout << "For Kolmogorov-Smirnov, lambda = ";
cin >> alam;
cout << "Q_KS " << alam << " = " << stats::probks( alam ) << endl;
```

7.17 Functions

The *Funct* class provides functional manipulation such as root finding, etc. The *Funct* constructor takes as its arguments an instance of the function (generally **this*) followed by a reference to the *member function of a class*, for example:

```
class TestIt {
public:
   TestIt(){}
   ~TestIt(){}
   Funct< TestIt > squarer(){ return Funct< TestIt >( *this, &
   TestIt::xsqrd ); }

private:
   double xsqrd( double x ){ return x * x - 4; }
};
```

Note that *Funct* is a template class, and takes as its template argument the name of the class containing the member function. Errors are caught by *functErr* (see below for examples).

7.17.1 Bracketing

The following methods bracket a function:[12]

- F.mnbrak(double $x1$, double $x2$) takes boundary points $x1$ and $x2$ to bracket function F. Accessors below return the bracketed result, where ax, bx and cx are 3 sequential x-axis points, with $fb = f(bx)$ less than $fa = f(ax)$ and $fc = f(cx)$. All returned values are of type double.
- F.mnbrak_ax(): accessor returns ax
- F.mnbrak_bx(): accessor returns bx
- F.mnbrak_cx(): accessor returns cx
- F.mnbrak_fa(): accessor returns fa
- F.mnbrak_fb(): accessor returns fb
- F.mnbrak_fc(): accessor returns fc

[12] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 400–401

Example

```
class TestIt {
public:
   TestIt(){}
   ~TestIt(){}
   void mnbrak( double x1, double x2 )
   {
      try
      {
         Funct< TestIt > f( *this, &TestIt::xsqrd );
         f.mnbrak( x1, x2 );
         cout << "ax = " << f.mnbrak_ax() << endl;
         cout << "bx = " << f.mnbrak_bx() << endl;
         cout << "cx = " << f.mnbrak_cx() << endl;
         cout << "fa = " << f.mnbrak_fa() << endl;
         cout << "fb = " << f.mnbrak_fb() << endl;
         cout << "fc = " << f.mnbrak_fc() << endl;
      }
      catch ( Funct< TestIt >::functErr& e ) { cout << e.what() <<
      endl; }
   }

private:
   double xsqrd( double x ){ return x * x - 4; }
};

int main()
{
   TestIt tester;
   double x1, x2;
   cout << "For testing mnbrak, x1 = ";
   cin >> x1;
   cout << "x2 = ";
   cin >> x2;
   tester.mnbrak( x1, x2 );

   return 0;
}
```

7.17.2 Find Minimum

The following methods isolate the minimum of function F using Brent's method:[13]

[13] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 402–405

- F.brent(double ax, double bx, double cx, double tol): takes initial bracketing triplet of abscissas ax, bx and cx and tolerance tol to isolate minimum of function F.
- F.brent_xmin(): returns the isolated minimum (type double) of function F.

Example

```
class TestIt {
public:
   TestIt(){}
   ~TestIt(){}
   void brent( double a1, double b1, double c1, double tol )
   {
      try
      {
         Funct< TestIt > f( *this, &TestIt::xsqrd );
         cout << "brent = " << f.brent( a1, b1, c1, tol ) << endl;
         cout << "xmin = " << f.brent_xmin() << endl;
      }
      catch ( Funct< TestIt >::functErr& e ) { cout << e.what() <<
      endl; }
   }

private:
   double xsqrd( double x ){ return x * x - 4; }
};

int main()
{
   TestIt tester;
   double a, b, c, tol;
   cout << "For testing brent, a = ";
   cin >> a;
   cout << "b = ";
   cin >> b;
   cout << "c = ";
   cin >> c;
   cout << "tolerance = ";
   cin >> tol;
   tester.brent( a, b, c, tol );

   return 0;
}
```

7.17.3 Find Roots

The following method finds roots of function F using Brent's method:[14]

- F.zbrent(double $x1$, double $x2$, double tol): takes 2 x-axis bounds $x1$ and $x2$, and tolerance tol, and returns the root of function F between those bounds.

Example

```
class TestIt {
public:
   TestIt(){}
   ~TestIt(){}
   void zbrent( double x1, double x2, double tol )
   {
      try
      {
         Funct< TestIt > f( *this, &TestIt::xsqrd );
         cout << "zbrent = " << f.zbrent( x1, x2, tol ) << endl;
      }
      catch ( Funct< TestIt >::functErr& e ) { cout << e.what() <<
      endl; }
   }

private:
   double xsqrd( double x ){ return x * x - 4; }
};

int main()
{
   TestIt tester;
   double x1, x2, tol;
   cout << "For testing zbrent, x1 = ";
   cin >> x1;
   cout << "x2 = ";
   cin >> x2;
   cout << "tolerance = ";
   cin >> tol;
   tester.zbrent( x1, x2, tol );

   return 0;
}
```

[14] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 359–362

7.18 Linear Regression

The *XY* class performs least squares linear regression. The *XY* constructors takes as their arguments vector< double >s containing the x- and y-coordinates, and if present, the x- and y-standard deviations:[15]

- XY(vector< double >& x, vector< double >& y): loads an XY dataset with x-coordinates x and y-coordinates y without standard deviations
- XY(vector< double >& x, vector< double >& y, vector < double >& σ_x): loads an XY dataset with x-coordinates x, y-coordinates y, and x standard deviations σ_x
- XY(vector< double >& x, vector< double >& y, vector < double >& σ_x, vector < double >& σ_y): loads an XY dataset with x-coordinates x, y-coordinates y, x standard deviations σ_x, and y standard deviations σ_y

The following accessors return results of least squares linear regression on *XY* dataset D (all of type double):

- D.a(): returns coefficient a (y-intercept) in least squares linear fit $y = a + bx$
- D.b(): returns coefficient b (slope) in least squares linear fit $y = a + bx$
- D.x0(): returns x-intercept x_0 in least squares linear fit $y = a + bx$
- D.siga(): returns standard errors on a
- D.sigb(): returns standard errors on b
- D.chi2(): returns minimized chi-squared value of least squares fit
- D.q(): returns chi-squared probability of least squares fit (small is a poor fit)
- D.r(): returns correlation coefficient r

7.18.1 Notes

Errors are caught by *XYErr* (see below for examples).

[15] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 661–670

7.18.2 Examples

```
double xs[ 5 ] = { 1.0, 2.0, 3.0, 4.0, 5.0 };
double ys[ 5 ] = { 1.0, 2.2, 3.0, 4.0, 6.0 };
double stdxs[ 5 ] = { 0.2, 0.4, 0.2, 0.2, 1 };
double stdys[ 5 ] = { 0.2, 0.3, 0.2, 0.2, 0.8 };

vector< double > xv( xs, xs + 5 ), yv( ys, ys + 5 ), stdx( stdxs,
stdxs + 5 ),
   stdy( stdys, stdys + 5 );

try {
   XY xyset( xv, yv );
   cout << "For dataset without errors, a = " << xyset.a() << endl;
   cout << "b = " << xyset.b() << endl;
   cout << "x0 = " << xyset.x0() << endl;
   cout << "siga = " << xyset.siga() << endl;
   cout << "sigb = " << xyset.sigb() << endl;
   cout << "chi2 = " << xyset.chi2() << endl;
   cout << "q = " << xyset.q() << endl;
   cout << "r = " << xyset.r() << endl << endl;
}
catch ( XY::XYErr& e ) { cout << e.what() << endl;

try {
   XY xyset2( xv, yv, stdx );
   cout << "For dataset with x errors only, a = " << xyset2.a() <<
   endl;
   cout << "b = " << xyset2.b() << endl;
   cout << "x0 = " << xyset2.x0() << endl;
   cout << "siga = " << xyset2.siga() << endl;
   cout << "sigb = " << xyset2.sigb() << endl;
   cout << "chi2 = " << xyset2.chi2() << endl;
   cout << "q = " << xyset2.q() << endl;
   cout << "r = " << xyset2.r() << endl << endl;
}
catch ( XY::XYErr& e ) { cout << e.what() << endl;

try {
   XY xyset3( xv, yv, stdx, stdy );
   cout << "For dataset with x & y errors, a = " << xyset3.a() <<
   endl;
   cout << "b = " << xyset3.b() << endl;
   cout << "x0 = " << xyset3.x0() << endl;
   cout << "siga = " << xyset3.siga() << endl;
   cout << "sigb = " << xyset3.sigb() << endl;
```

```
   cout << "chi2 = " << xyset3.chi2() << endl;
   cout << "q = " << xyset3.q() << endl;
   cout << "r = " << xyset3.r() << endl << endl;
}
catch ( XY::XYErr& e ) { cout << e.what() << endl; }
```

Chapter 8

Object Design

The design of the network object pursues 2 objectives. The first is a well organized, object oriented representation of a generic network object that is flexible and easily extended to specific network types. In fact, the current design allows for all models, including statistical ones.

The second objective of the network object design is that the code be readable. This was the logic of constructing efficient yet readable matrix and vector operations, as described in Chapter 5.

Figure 8.1 demonstrates the object hierarchy. Objects that are to be built are enclosed in dashed lines. Objects that are built (although features will be added) are shown in solid lines.

- **DataSet** contains and manipulates the data to be modeled.
- **TwoSet** is a specialized object designed to calculate metrics for *one* output *Models*, including sensitivity, specificity, predictive value positive, predictive value negative, and ROC area.
- **Model** is the base object. *Model* provides access to the dataset, and contains the virtual methods common to all modeling techniques.
- **Iterative** extends *Model*. Neural computational algorithms are iterative, and thus use *Iterative* as a base. *Iterative* manages methods and parameters related to iterating through a dataset.
- **Network** extends *Iterative*. Algorithms with hidden layers, weights, learning rates, transfer functions and other general properties of neural networks are encapsulated in *Network*.
- **RegressNet** is a compound object including a *Network* object which implements stepwise regression. See Chapter 9.
- **BackProp** extends *Network*. *BackProp* is a multilayer multiple output back-propagation neural network.

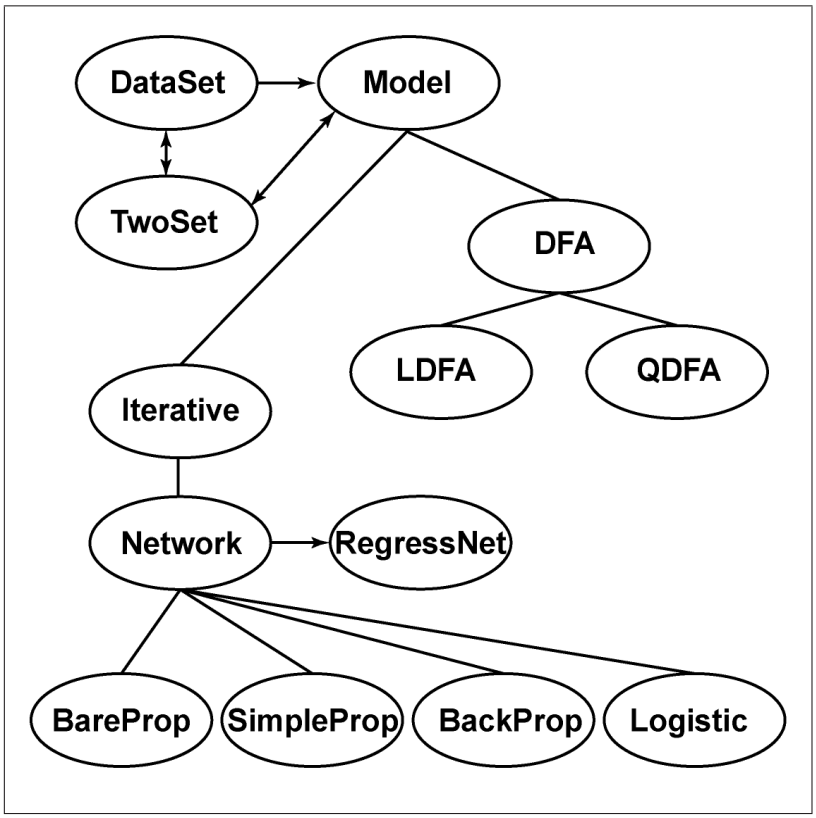

Figure 8.1: Object design

- **SimpleProp** also extends *Network*. *SimpleProp* is a backpropagation neural network which has 1 output node, 1 hidden node layer, and biases.
- **BareProp** also extends *Network*. *BareProp* is an example *Network* object for illustrative purposes which has 1 output node, 1 hidden node layer, and no biases.
- **Logistic** also extends *Network*. *Logistic* implements binary logistic regression, which is mathematically equivalent to a single output node neural network with a bias, the sigmoidal transfer function, no hidden nodes, and the cross-entropy error function.
- **DFA** extends *Model*, and encodes discriminant function analysis. **QDFA** extends *DFA*, and encodes quadratic discriminant function analysis. **LDFA** extends *DFA*, and encodes linear discriminant function analysis.

8.1 Function definitions

For clarity and ease of use, function definitions are placed in the file "function_defs.h". The implementation of function definitions in this file may take a number of forms, depending on how the function is to be used in the code. The reader is strongly encouraged to study "function_defs.h" to see the various forms functions may take.

8.1.1 In-line functions

The simplest form is the in-line function. An example is that which calculates the least mean squared error for a single output node:

```
// LMS error for a single output, takes known value y and model's
//    guess o as arguments, returns LMS error
inline double singleLMSerror( double y, double o )
{
     return 0.5 * ( y - o ) * ( y - o );
}
```

8.1.2 Function classes

For functions that are to be passed to *vector* or *Matrix* objects using the **func** method, classes must be constructed. An example is that which implements the sigmoidal transfer function:

```
// The sigmoidal function
class sigmoidal : public unary_function< double, double >
{
     public:
          inline double operator()( const double& x )
          {
               return 1.0 / ( 1.0 + exp( -x ) );
          }
};
```

8.1.3 The errorFunction object

A very important object defined in **function_defs.h** is *errorFunction*, which handles the calculation of model error. As neural network sigmoidal output is defined as

$$\vec{\mathbf{o}} = \frac{1}{1 + e^{-\vec{\mathbf{x}}}}$$

calculation of the error function may depend on $\vec{\mathbf{x}}$ as well as $\vec{\mathbf{o}}$ (see the **special note about numerical precision and the cross-entropy error** below), and thus *errorFunction* methods may be passed $\vec{\mathbf{x}}$ as well as $\vec{\mathbf{o}}$. *errorFunction* consists of

- *Public Methods*
- errorFunction(double y, double o, double x, bool $errorType$): Constructor for a *single* output, which takes known value y, model output o, argument x in sigmoidal transfer function $1/(1+e^{-x})$ and bool flag $errorType$, where
  - $errorType = 0$ denotes the least mean squared error

$$E = \frac{1}{2}(y_i - o_i)^2$$

  - $errorType = 1$ denotes the cross-entropy error

$$E = -y\log(o) - (1-y)\log(1-o)$$

- errorFunction(vector< double > $\vec{\mathbf{y}}$, vector< double > $\vec{\mathbf{o}}$, vector< double > $\vec{\mathbf{x}}$, bool $errorType$): Constructor for *multiple* outputs, which takes known values $\vec{\mathbf{y}}$, model outputs $\vec{\mathbf{o}}$, arguments $\vec{\mathbf{x}}$ in sigmoidal transfer function $1/(1+e^{-\vec{\mathbf{x}}})$ and bool flag $errorType$, where
  - $errorType = 0$ denotes the least mean squared error for m output nodes

$$E = \frac{1}{2}\sum_{i=1}^{m}(y_i - o_i)^2$$

  - $errorType = 1$ denotes the cross-entropy error for m output nodes

$$E = \sum_{i=1}^{m} -y\log(o) - (1-y)\log(1-o)$$

- double value(): Accessor returns numerical error result
- bool boundsErr() Accessor returns true if numerical bounds error (e.g. log(0)) (see the **special note about numerical precision and the cross-entropy error** below)

Example

```
// Calculate x-entropy error for single output and add to set error
//    which is previously defined as double setError (double y, double
o
//    and double x have also previously been calculated)
errorFunction E( y, o, x, 1 );
setError += E.value();

if ( E.boundsErr() ) // check for out of bounds error in log(o)
    cout << "ERROR: Numerical bounds error" << endl;
```

A special note about numerical precision and the cross-entropy error

As the cross-entropy error function is defined for network output o and target value y:

$$E = -y \log(o) - (1 - y) \log(1 - o)$$

And for the sigmoidal transfer function,

$$o = \frac{1}{1 + e^{-x}}$$

Then for large positive ($>> 0$) and large negative ($<< 0$) x, $o \to 1$, and $o \to 0$ respectively, and thus $log(1 - o) \to \infty$ and $log(o) \to \infty$. In these cases, the following transformations are made (which require those methods to be passed the sigmoidal transfer function argument x for the cross-entropy error.)

Large negative x

If $x << 0$,

$$o = \frac{1}{1 + e^{-x}} \approx \frac{1}{e^{-x}} = e^{x}$$

and thus

$$\log(o) = \log(e^{x}) = x$$

since $o \to 0$,

$$log(1 - o) \to log(1) = 0$$

thus

$$E = -y \log(o) - (1 - y) \log(1 - o) \approx -yx - 0 = -yx$$

Large positive x

$$1 - o = 1 - \frac{1}{1 + e^{-x}} = \frac{1 + e^{-x}}{1 + e^{-x}} - \frac{1}{1 + e^{-x}} = \frac{e^{-x}}{1 + e^{-x}}$$

If $x >> 0$, since $1 + e^{-x} \to 1$

$$1 - o = \frac{e^{-x}}{1 + e^{-x}} \approx e^{-x}$$

thus

$$\log(1 - o) \approx log(e^{-x}) = -x$$

if $x >> 0$, $o \to 1$ and $-y \log(o) \to 0$, thus

$$E = -y \log(o) - (1 - y) \log(1 - o) \approx 0 - (1 - y)(-x) = (1 - y)x$$

8.2 What you really need to know about DataSet and TwoSet

All models use training and test sets. The model builds itself on the training set, and is tested on the test set. *DataSet* is designed for generating and manipulating training and test sets, and *TwoSet* is a special object that is designed to calculate accuracies for one-output models *only*.

All training and test sets are expressed by Matrices in the following format, where i is an input to the model, and o is an output:

$$\begin{array}{cccccccc} i_{11} & i_{12} & \dots & i_{1I} & o_{11} & o_{12} & \dots & o_{1O} \\ i_{21} & i_{22} & \dots & i_{2I} & o_{21} & o_{22} & \dots & o_{2O} \\ i_{31} & i_{32} & \dots & i_{3I} & o_{31} & o_{32} & \dots & o_{3O} \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ i_{N1} & i_{N2} & \dots & i_{NI} & o_{N1} & o_{N2} & \dots & o_{NO} \end{array}$$

for N examples ("exemplars") in the set, I inputs and O outputs. Although there are many highly useful features in *DataSet*, a programmer building a *Model* need only know how to access the training and test Matrices, whether they've been loaded into the *DataSet* object, how many inputs and outputs there are, whether the output is discrete (held to 0 or 1), and the number of examples in each set. These methods are shown in red in the *DataSet* specification.

In addition, the programmer of a *Model* needs only to understand that the methods setTrainTwoSet and setTestTwoSet must be called in order for the method metricsReport, which reports accuracy metrics of a one-output *Model*, to be called. These methods are also shown in red in the *DataSet* specification.

8.3 DataSet

DataSet is a standalone object that contains and manipulates the dataset that *Model* will access. All *DataSets* have raw, training and test datasets, and know the number of inputs and outputs. *DataSet* consists of:

- *Public* methods
  - Raw dataset methods:
    - void loadRaw(string& $filename$): Takes $filename$ as argument (string), and loads a file into the *Raw* dataset Matrix, returns nothing
    - bool rawLoaded(): Returns a flag to indicate if *Raw* dataset is loaded
    - Matrix< double >& getRawMatrix(): returns the *Raw* dataset
    - void setRawMatrix(Matrix< double >& $inMatrix$): loads Matrix $inMatrix$ into the *Raw* dataset Matrix
  - Training set methods:

* void loadTrain(string& $filename$): Takes $filename$ as argument (string), and loads a file into the $TrainSetData$ Matrix, returns nothing
* bool trainLoaded(): Returns a flag to indicate if $TrainSetData$ Matrix is loaded
* bool saveTrain(string& $filename$): Takes $filename$ as argument (string), and saves the $TrainSetData$ Matrix into a file, returns flag indicating if save operation was successful
* bool saveScales(string& $filename$): Takes $filename$ as argument (string), and saves the scaling factors for the training set into the file, returns flag indicating if save operation was successful

  Note: see equation 8.2 below for an explanation of scaling. Saved to the file will be $lbound$ (1 value), and for each x, S and min.

$$x_{norm} = lbound + S(x - min) \tag{8.1}$$

  where x_{norm} is the normalized result, $lbound$ is the lower limit of normalized values, S is the scaling factor, and min is the minimum value of each x in the dataset.
* Matrix< double >& getTrainMatrix(): returns the $TrainSetData$ Matrix
* void setTrainMatrix(Matrix< double >& $inMatrix$): loads Matrix $inMatrix$ into the $TrainSetData$ Matrix

– $TwoSet$ accessors for the training set:

* bool setTrainTwoSet(): Constructs an internal $TwoSet$ object from the training set, populating the 'real' column of the $TwoSet$ object with the output column from the training set $Matrix$, returns true if successful
* TwoSet& getTrainTwoSet(): Returns the constructed $TwoSet$ object from the training set
* bool TrainTwoSetLoaded(): Returns true if the internal training $TwoSet$ object was successfully constructed
* bool saveTrainTwoSet(string& $filename$): Takes $filename$ as argument (string), and saves the $TwoSet$ object from the training set into a file, returns flag indicating if save operation was successful

– Test set methods:

* void loadTest(string& $filename$): Takes $filename$ as argument (string), and loads a file into the $TestSetData$ Matrix, returns nothing
* bool testLoaded(): Returns a flag to indicate if $TestSetData$ Matrix is loaded

* bool saveTest(string& *filename*): Takes *filename* as argument (string), and saves the *TestSetData* Matrix into a file, returns flag indicating if save operation was successful
* Matrix< double >& getTestMatrix(): returns the *TestSetData* Matrix
* void setTestMatrix(Matrix< double >& *inMatrix*): loads Matrix *inMatrix* into the *TestSetData* Matrix

– *TwoSet* accessors for the test set:

* bool setTestTwoSet(): Constructs an internal *TwoSet* object from the test set, populating the 'real' column of the *TwoSet* object with the output column from the test set *Matrix*, returns true if successful
* TwoSet& getTestTwoSet(): Returns the constructed *TwoSet* object from the test set
* bool TestTwoSetLoaded(): Returns true if the internal test *TwoSet* object was successfully constructed
* bool saveTestTwoSet(string& *filename*): Takes *filename* as argument (string), and saves the *TwoSet* object from the test set into a file, returns flag indicating if save operation was successful

– General methods:

* void setInput(unsigned *I*): Set the number of input nodes to *I*
* unsigned getInput(): Returns number of input nodes
* void setOutput(unsigned *O*): Set the number of output nodes to *O*
* unsigned getOutput(): Returns number of output nodes
* unsigned getNumTrain(): Returns the number of examples in *TrainSetData*, the training set
* unsigned getNumTest(): Returns the number of examples in *TestSetData*, the test set
* void removeInputs(vector< unsigned >& *v*): Given a vector representing input positions within a dataset, removes the corresponding columns in *Raw*, *TrainSetData*, and *TestSetData*, thus effectively removing those inputs from a dataset

Note: In the following methods, a variate x is normalized according to:

$$x_{norm} = lbound + [(\frac{x - min}{max - min}) \times (hbound - lbound)] \qquad (8.2)$$

where x_{norm} is the normalized value, *lbound* is the lower limit of the normalized result, *hbound* is the upper limit of the normalized result, *min* is the minimum value of x in the dataset, and *max* is the maximum value of x in the dataset.

- Methods related to normalizing dataset variates
  - void setInUpper(double x): Sets the upper limit of normalized input variates to x
  - double getInUpper(): Returns the upper limit of normalized input variates
  - void setInLower(double x): Sets the lower limit of normalized input variates to x
  - double getInLower(): Returns the lower limit of normalized input variates
  - void setOutUpper(double x): Sets the upper limit of normalized output variates to x
  - double getOutUpper(): Returns the upper limit of normalized output variates
  - void setOutLower(double x): Sets the lower limit of normalized output variates to x
  - double getOutLower(): Returns the lower limit of normalized output variates
- Methods related to identifying output as discrete (held to 0 or 1)
  - void setDiscrete(bool $flag$): Sets output to be discrete if $flag$ is true, continuous if false
  - bool getDiscrete(): Returns the state of discrete output
  - void setThreshold(double x): Sets the threshold value for discrete output to be x, any output $>= x$ will be 1, and any output $< x$ will be 0
  - double getThreshold(): Returns the threshold value
- Methods related to transforming datasets:
  - bool raw2train(): Converts the raw dataset to a training set by normalizing its input variates, and if nondiscrete, its output variates, returning true if the operation was successful
  - bool randomize(unsigned n): Randomizes the raw dataset into training and test sets which are then normalized, preserving the frequency of outcomes in the training and test sets, takes the number of exemplars n to be placed in the test set as an argument *(number of outputs must be only 1, and it must be discrete)*, returns true if successful
  - bool randomize(unsigned n, unsigned d): Randomizes the raw dataset into training and test sets which are then normalized, preserving the frequency of outcomes in the training and test sets, takes the numerator n and denominator d of a fraction representing the portion of examplars to be placed in the test set as arguments, *(number of outputs must be only 1, and it must be discrete)*, returns true if successful

    - bool randomizeD(double x): Randomizes the raw dataset into training and test sets which are then normalized, preserving the frequency of outcomes in the training and test sets, takes the fraction x as a double value from 0 to 1 representing the portion of examplars to be placed in the test set as argument *(number of outputs must be only 1, and it must be discrete)*, returns true if successful
  - Methods related to calculating accuracy
    - void metricsReport(ostream& *streamObj*): Outputs all internal *TwoSet* metrics for a 1-output *Model*, takes ostream object *streamObj* as argument
  - Methods related to logging to the history file:
    - void setHistory(bool $flag$): Sets history logging, which *appends* all operations to the history file
    - bool getHistory(): Returns the current state of history logging
    - void setHistoryFilename(string& $filename$): Sets the name of the history file to string $filename$
    - bool addHistory(ostringstream& *outputStream*): appends the ostringstream object *outputStream* to the history file.

- *Private* members
  - Matrix< double > *Raw*: the raw dataset
  - Matrix< double > $TrainSetData$: the training set
  - Matrix< double > $TestSetData$: the test set
  - TwoSet $TrainTwoSet$: the *TwoSet* object for the training set
  - TwoSet $TestTwoSet$: the *TwoSet* object for the test set
  - vector< double > *minima*: a vector containing the minimum values of the columns in *Raw*
  - vector< double > *maxima*: a vector containing the maximum values of the columns in *Raw*
  - unsigned $nInput$: the number of input nodes
  - unsigned $nOutput$: the number of output nodes
  - double *threshold*: the threshold for discrete output
  - double $inUpperLimit$: the upper limit for a normalized input variate
  - double $inLowerLimit$: the lower limit for a normalized input variate
  - double $outUpperLimit$: the upper limit for a normalized output variate
  - double $outLowerLimit$: the lower limit for a normalized output variate

- bool *discreteFlag*: a flag to indicate if the output is to be considered discrete (held to 0,1)
- bool *rawLoadedFlag*: a flag to indicate if *Raw* loaded
- bool *trainLoadedFlag*: a flag to indicate if $TrainSetData$ loaded
- bool *testLoadedFlag*: a flag to indicate if $TestSetData$ loaded
- bool *trainTwoSetFlag*: a flag to indicate if the training set *TwoSet* object has been successfully constructed
- bool *testTwoSetFlag*: a flag to indicate if the test set *TwoSet* object has been successfully constructed
- bool *historyFlag*: a flag to indicate if operations will be logged to a history file, this file is *appended*
- bool *minimaxFlag*: a flag to indicate if minima and maxima vectors have been set
- string *historyFilename*: the name of history file

- *Private* Utility functions
  - void minimax(Matrix< double >& *data*): computes the *minima* and *maxima* vectors from *Matrix data*
  - bool checkDiscrete(Matrix< double >& *data*): returns true if the output columns of *Matrix data* are discrete, false if otherwise
  - void normalize(Matrix< double >& *data*): normalizes input variates of *Matrix data*, and if nondiscrete, its output variates *(note: minima and maxima vectors must have been previously set)*
  - void clear(): clears *DataSet Matrix* members and flags

8.4 TwoSet

TwoSet is a specialized standalone object that is designed to calculate outcome metrics such as sensitivity, specificity, predictive value positive, predictive value negative and ROC area from a *one* output *Model*. The *TwoSet* data *Matrix* contains 2 columns, one for the known outcomes and one for the *Model's* outputs. The *Model* output *must be* discrete. The format of this data *Matrix* is:

$$\begin{array}{cc} real_0 & test_0 \\ real_1 & test_1 \\ real_2 & test_2 \\ \dots & \dots \\ real_{n-1} & real_{n-1} \end{array}$$

where *real* represents a known outcome, and *test* is the *Model's* output, for n examples in the data *Matrix*, e.g.:

```
0.000000 0.002851
0.000000 0.002978
0.000000 0.002857
1.000000 0.571710
```

For calculating ROC (receiver-operator characteristic) curve area, 2 methods are available. One is the traditional method using trapezoids and numerical calculation to calculate the area under the curve. By default, the number of trapezoids is set to the number of data points, but that may be changed by the *setTrapThresholds()* method. However, as should be evident, this method *underestimates* the ROC area. Wickens[1] provides a statistical method to calculate ROC area. Here, $z = Z(A) = \Phi^{-1}(A)$ is linearly modeled for sensitivity and 1 - specificity, and this linear model is used to calculate ROC area. If enough data is available to bin and generate errors in the abscissa and ordinate of this line, then a p-value will be generated to reflect the probability of a least squares fit to that line, where a smaller p-value is *worse* (closer to 1 is more linear).

The method *ROCarea*(*ostream*&) may be used to automatically choose which method of calculating ROC curve are is used, depending on how many data points are available.

Two errors may be thrown from this class. The first is *DivisionByZero* where division by zero is encountered, generally in calculating positive and negative predictive value. The second is *twoSetErr*, which will contain errors generated in statistical calculation or linear modeling. *DivisionByZero* always contains the message "division by zero", whereas *twoSetErr*'s *what*() member will contain a message reflecting the type of error. An example of the latter would be:

```
string reply;
cout << "What is the name of the file? ";
cin >> reply;
TwoSet data;
data.load( reply );
try {
   data.ROCarea( cout );
} catch ( TwoSet::twoSetErr& e ) { cout << e.what() << endl; }
```

The *TwoSet* object consists of:

- *Public* methods

  The constructor TwoSet(Matrix< double >& *M*) constructs and loads *Matrix M* into a *TwoSet* object

  - Methods for loading Matrices and files into a *TwoSet* object:
    - void setMatrix(Matrix< double >& *M*): Loads *Matrix M* into a *TwoSet* object, returns nothing

[1] Wickens, Thomas D., Elementary Signal Detection Theory, Oxford University Press, New York, NY 2002, pp. 60–74

    - void load(string& *filename*): Takes *filename* as argument (string), and loads a file into the *TwoSet* data *Matrix*, returns nothing
    - bool loaded(): Returns a flag to indicate if the *TwoSet* data *Matrix* is loaded
- *TwoSet* accessors:
    - Matrix< double >& getData(): Returns the *TwoSet* data *Matrix*
    - bool save(string& *filename*): Takes *filename* as argument (string), and saves the *TwoSet* object into a file, returns flag indicating if save operation was successful
    - unsigned getNumElements(): Returns the number of elements in the *TwoSet* data *Matrix*
    - TwoSet& insertReal(vector< double >& *v*): Insert vector *v* into the real column of the *TwoSet* data *Matrix*, the number of elements of *v must equal* the number of rows in the *TwoSet* data *Matrix*
    - vector< double >& real(vector< double >& *v*): Populate vector *v* with the real column of the *TwoSet* data *Matrix*, the number of elements of *v must equal* the number of rows in the *TwoSet* data *Matrix*
    - vector< double > real(): Get a new vector from the real column of the *TwoSet* data *Matrix*
    - TwoSet& insertTest(vector< double >& *v*): Insert vector *v* into the test column of the *TwoSet* data *Matrix*, the number of elements of *v must equal* the number of rows in the *TwoSet* data *Matrix*
    - vector< double >& test(vector< double >& *v*): Populate vector *v* with the test column of the *TwoSet* data *Matrix*, the number of elements of *v must equal* the number of rows in the *TwoSet* data *Matrix*
    - vector< double > test(): Get a new vector from the test column of the *TwoSet* data *Matrix*
    - double real(unsigned *r*): An accessor to set or retrieve the element in row *r* in the real column of the *TwoSet* data *Matrix*
    - double test(unsigned *r*): An accessor to set or retrieve the element in row *r* in the test column of the *TwoSet* data *Matrix*
- Accessors and methods for *TwoSet* calculated metrics
    - void setThreshold(double *x*): Sets threshold value to x, any test element $>= x$ will be counted as 1, whereas any test element $< x$ will be counted as 0
    - double getClassAcc(): Returns classification accuracy
    - double getSens(): Returns sensitivity, throws exception TwoSet::DivisionByZero& *e*, where *e*.what() is "division by zero"

    * double getSpec(): Returns specificity, throws exception TwoSet::DivisionByZero& e, where e.what() is "division by zero"
    * double getPVP(): Returns predictive value positive, throws exception TwoSet::DivisionByZero& e, where e.what() is "division by zero"
    * double getPVN(): Returns predictive value negative, throws exception TwoSet::DivisionByZero& e, where e.what() is "division by zero"
    * void ClassTable(ostream& *outputStream*): Outputs to ostream& the classification table
- ROC area calculation by trapezoidal method
    * void setTrapThresholds(unsigned n): Sets number of trapezoids to calculate ROC area to n
    * unsigned getTrapThresholds(): Returns number of trapezoids to calculate ROC area
    * double getTrapROCarea(): Returns ROC area calculated by trapezoidal method
- ROC area calculation by statistical method
    * void setNbins(unsigned n): If data can be binned, sets the number of bins to n
    * unsigned getNbins(): Returns number of bins specified by user
    * void setBinSize(unsigned n): If data can be binned, sets number of elements to be contained in a bin to n
    * unsigned getBinSize(): Return bin size specified by user
    * void NbinThreshold(unsigned n): Sets the number of data points below which data will not be binned
    * unsigned getNBinThreshold(): Returns the number of data points below which data will not be binned
    * bool getBinned(): Returns *true* if data was binned
    * void setNbinsSetsSize(bool $flag$): Set $flag$ if number of bins sets bin size
    * bool getNbinsSetsSize(): Returns *true* if number of bins sets bin size
    * double getStatROCarea(): Returns statistical ROC area, throws exception TwoSet::twoSetErr& e, where e.what() contains a statistical or linear regression error message
    * double getStatP(): Returns p-value for fitted line (smaller is worse)
    * double getStatChi2(): Returns chi-squared value for fitted line
- Methods and accessors which output to ostream a ROC report which uses either or both the trapezoidal and/or statistical method

* void ROCarea(ostream& *outputStream*): Outputs to ostream& ROC area, and a report detailing method and results of calculation
* void setROCReportFlag($flag$): Set $flag$ true if both statistical and trapezoidal methods are to be reported, false to use either the trapezoidal method if number of data points < threshold ($calcThresh$) or statistical method if >= threshold
* bool getROCReportFlag(): Returns flag indicating if either (false) or both (true) of the statistical and trapezoidal methods are used and reported, according to the above description of setROCReportFlag(...)
* void setROCthresh(unsigned n): Sets threshold for number of non-zero, non-one sensitivity and 1 - specificity data points under which statistics will not be used
* unsigned getROCthresh(): Returns number of non-zero, non-one sensitivity and 1 - specificity data points under which statistics will not be used
* void setROCSearchFlag(bool $flag$): Set $flag$ true if best p and ROC areas are searched and reported within a specified range of number of bins
* bool getROCSearchFlag(): Returns flag indicating if best p and ROC areas are searched and reported within a specified range of bins
* void setMinROCSearchBins(unsigned n): Sets minimum number of bins to search for best p and ROC areas (must be > 2)
* unsigned getMinROCSearchBins(): Returns minimum number of bins to search for best p and ROC areas
* void setMaxROCSearchBins(const unsigned n): Sets maximum number of bins to search for best p and ROC areas
* unsigned getMaxROCSearchBins(): Returns maximum number of bins to search for best p and ROC areas

– Accessors for the Kolmogorov-Smirnov test[2]

* double getKSD(): Returns Kolmogorov-Smirnov statistic D
* double getKSP(): Returns Kolmogorov-Smirnov p- value

- *Private* members

– Matrix< double > A: the *TwoSet* data *Matrix*

– unsigned n: the number of elements in the *TwoSet* data *Matrix*

– unsigned tp: the number of true positives

– unsigned tn: the number of true negatives

[2]Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 623–626

- unsigned fp: the number of false positives
- unsigned fn: the number of false negatives
- unsigned nThresholds: the number of thresholds for trapezoidal method
- unsigned nBins: the number of bins for statistical ROC
- unsigned binSize: the bin size for statistical ROC
- unsigned binThresh: the number of data points under which data will not be binned
- unsigned calcThresh: the number of data points under which statistics will not be used
- double threshold: the threshold value
- double statP: p-value for fitted line for statistical ROC calculation
- double statChi2: chi-squared value for fitted line for statistical ROC calculation
- double KSD: Kolmogorov-Smirnov test D value
- double KSP: Kolmogorov-Smirnov test *p*-value
- bool loadedFlag: a flag to indicate if the *TwoSet* data *Matrix* is loaded
- bool thresholdFlag: a flag to indicate if the threshold is set
- bool nBinFlag: flag indicates if number of bins determines bin size
- bool binnedFlag: flag indicates if data was binned for statistical ROC calculation
- bool statROCcalcFlag: flag indicates if statistical ROC area calculated
- bool reportFlag: flag directs how to report statistical and/or trapezoidal ROC
- bool searchFlag: flag indicates if largest p, largest ROC is searched
- bool KScalcFlag: flag indicates if Kolmogorov-Smirnov test calculated

- *Private* Utility functions
  - void initialize(): Sets common initial values for constructors
  - void calculate(): Given a threshold, calculates tp, tn, fp, and fn
  - bool checkDiscrete(Matrix< double >& M): Returns true if the first column (representing real outcomes) of *Matrix* M is discrete, false if otherwise
  - void statReport(ostream& $outputStream$, unsigned $goodData$, unsigned $nBins$, double AUC, double $chi2$, double p): Utility method for ROCarea, outputs statistical report given ostream& $outputStream$, number of non-zero, non-one data points $goodData$, number of bins $nBins$, area under the curve AUC, chi-square value $chi2$, and p-value p

– void KScalc(): Utility method to calculate Kolmogorov-Smirnov test[3]

8.5 What you really need to know about programming a Model

8.5.1 Object Type

All concrete derived objects of *Model* inherit its protected member, (string) *objType*, which is the name of the object. Typically, the class constructor defines this member, for example:

```
class BareProp : public Network {
public:
   BareProp() { objType = "BareProp"; } // default constructor
```

Abstract classes *do not* define *objType*. This member is accessed by the *Model* method getType().

8.5.2 Copy constructor and overloaded = operator

All derived objects of *Model* **must** have a copy constructor and overloaded = operator. Typically, both call a protected utility function so that both copy the same members. The protected utility function typically begins with a call to the copy method of its immediate base class so that it inherits all of the members of its base class(es). For example, suppose a hypothetical neural network object named "LittleNet" is derived from *Network*, and contains the following members:

```
unsigned nNodes; // number of nodes

Matrix< double > W; // weight Matrix

vector< double > I, // input vector
                 y; // output vector
```

The following would be defined in the header file:

```
class LittleNet : public Network {
public:
   // Copy constructor
   LittleNet( const LittleNet& rhs );

   // Overloaded = operator
```

[3]Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 623–626

```
   LittleNet& operator= ( const LittleNet& rhs );

private: // or protected if the object is a base class
   // Copy utility
   void copy( const LittleNet& rhs );
```

Which would then be coded in the $C++$ file as:

```
// Copy constructor
LittleNet::LittleNet( const LittleNet& rhs )
{
   LittleNet::copy( rhs ); // use the copy utility
}

// Overloaded = operator
LittleNet& LittleNet::operator= ( const LittleNet& rhs )
{
   if ( &rhs != this ) // check for self-assignment
      LittleNet::copy( rhs ); // use the copy utility

   return *this; // enables A = B = C
}

// Copy utility
void LittleNet::copy( const LittleNet& rhs )
{
   Network::copy( rhs ); // call immediate base object copy
   nNodes = rhs.nNodes;
   W = rhs.W;
   I = rhs.I;
   y = rhs.y;
}
```

Remember: if you add any member to any existing class, you *must* insert a corresponding line of code to copy it in the copy utility function of that class.

8.5.3 Implementation of pure virtual methods in *Model*

Ultimately, all models map the inputs of each exemplar to the outputs. Thus, most of the *Model* base class methods pertain directly to this process. If you are writing a Model, you need to implement the following pure virtual methods:

setDataSet

The pure virtual method

```
virtual void setDataSet( DataSet& $dataObj$ )
```

loads a *DataSet* object into the derived *Model*. The protected member inherited from the base class *Model* is named *theData*, so the first line of

```
void YourModel::setDataSet( DataSet& dataObj )
```

where "YourModel" is the name of your derived *Model* object, should be

```
theData = dataObj; // set "theData" to incoming DataSet
```

The remainder of YourModel::setDataSet would then contain code that tailors your model to the incoming *DataSet* object. See SimpleProp::setDataSet and BareProp::setDataSet for examples.

train

The pure virtual method

```
virtual double train()
```

should train a *Model*, and return the final error (double) for that model. In the case of neural networks, training would involve finding the weight vectors and Matrices that minimize final error. See SimpleProp::train and BareProp::train for examples.

reportAccuracy

The pure virtual method

```
virtual void reportAccuracy( ostream& outputStream )
```

should report the accuracy of the trained model to an ostream outputStream. *If you are programming an object that extends* Network, *e.g. a neural network, you* **do not** *need to implement this virtual method, as* Network *implements reportAccuracy for you.*

outputHeader

The pure virtual method

```
virtual void outputHeader( ostream& outputStream )
```

should output a header to an ostream outputStream describing the architecture of the derived *Model*. See SimpleProp::outputHeader and BareProp::outputHeader for examples.

8.5.4 *Model* protected members

The following are the protected members in *Model* that your object needs to set or get:

theData

The protected member

```
DataSet theData
```

represents the primary dataset, in the form of a DataSet object. Your setDataSet method needs to set this member. See SimpleProp::setDataSet and BareProp::setDataSet for examples.

Train

The protected member

```
Matrix< double > Train
```

represents a Matrix that contains the training set input columns only, typically constructed by a call within your setDataSet method to the *Model* protected utility function extractInputMatrices(). See SimpleProp::setDataSet and BareProp::setDataSet for examples.

Test

The protected member

```
Matrix< double > Test
```

represents a Matrix that contains the test set input columns only, typically constructed by a call within your setDataSet method to the *Model* protected utility function extractInputMatrices(). See SimpleProp::setDataSet and BareProp::setDataSet for examples.

TrainOutput

The protected member

```
Matrix< double > TrainOutput
```

represents a Matrix that contains the training set output columns only, typically constructed by a call within your setDataSet method to the *Model* protected utility function extractOutputMatrices(). See BackProp::setDataSet for an example.

TestOutput

The protected member

```
Matrix< double > TestOutput
```

represents a Matrix that contains the test set output columns only, typically constructed by a call within your setDataSet method to the *Model* protected utility function extractOutputMatrices(). See BackProp::setDataSet for an example.

builtFlag

Set the protected member

```
bool builtFlag
```

to *true* when the Model has been formally constructed, and its architecture set. See SimpleProp::setHidden and BareProp::setHidden for examples.

historyFlag

Use the protected member

```
bool historyFlag
```

to decide if operations will be logged to a history file using the protected *Model* utility function addHistory. See SimpleProp and BareProp code for examples, as most methods use addHistory.

errorType

Use the protected member

```
bool errorType
```

to determine if the type of error will be least mean squared ($errorType = 0$) or cross-entropy ($errorType = 1$). Typically, this member is used as an argument in *errorFunction* constructors (see section 8.1.3, and SimpleProp::trainSet() and BareProp::trainSet() as examples.)

8.5.5 *Model* protected utility functions

The following are the protected utility functions in *Model* that your object will want to call:

extractInputMatrices

Call the utility function

```
void extractInputMatrices()
```

in your setDataSet method to extract the input columns of the training set into the protected *Model* Matrix $Train$, and the input columns of the test set into the protected *Model* Matrix $Test$.

extractOutputMatrices

Call the utility function

```
void extractOutputMatrices()
```

in your setDataSet method to extract the output columns of the training set into the protected *Model* Matrix *TrainOutput*, and the output columns of the test set into the protected *Model* Matrix *TestOutput*.

addHistory

Pass an ostringstream object to the utility function

```
bool addHistory( ostringstream $outputStream$ )
```

for that object to be added to the history file. This method should be called for every operation. See SimpleProp and BareProp code for examples, as most methods use addHistory.

copy

See section 8.5.2 for details.

8.6 Model

In the following specification, methods and members that pertain directly to coding a concrete derived *Model* implementation are shown in red .

Model is the base object. *Model* provides access to the dataset, and contains the virtual methods common to all modeling techniques. *Model* consists of:

- *Public* methods
  - Copy constructor and overloaded = operator, see section 8.5.2 for discussion:
    - Model& operator= (const Model& rhs)
    - Model(const Model& rhs)
  - Methods related to accessing the dataset to model:
    - virtual void setDataSet(DataSet& *data*): A pure virtual method that loads the *DataSet* object *data* into the *Model*
    - DataSet& getDataSet(): Returns a the *Model's* dataset in the form of a *DataSet* object
  - Methods related to training the model:
    - virtual double train(): A pure virtual method that trains a *Model*, and returns the final error (double) for that model

        * virtual void reportAccuracy(ostream& *outputStream*): A pure virtual method that reports the accuracy of the trained model to an ostream *outputStream*

    - Methods related to logging to files or reporting to user:
        * virtual void outputHeader(ostream& *outputStream*): A pure virtual method which outputs a header to an ostream *outputStream* describing the architecture of the model, *Model* type dependant
        * void setHistory(bool $flag$): Sets history logging, which *appends* all operations to the history file
        * bool getHistory(): Returns the current state of history logging
        * void setHistoryFilename(string& $filename$): Sets the name of the history file to string $filename$
        * string getHistoryFilename(): Returns the name of the history file
        * void setLastop(bool $flag$): Sets last operation logging, which *overwrites* a file to report the building (training in the case of *Iterative*) of a specific model
        * bool getLastop(): Returns the current state of last operation logging
        * void setLastopFilename(string& $filename$): Sets the name of the last operation logging file to $filename$
        * string getLastopFilename(): Returns the name of the last operation logging file

    - Methods related to setting the output error function
        * void setLMSerror(): Sets the output error function to least mean squared, which is

$$E = \frac{1}{2}\sum_{i=1}^{m}(y_i - o_i)^2 \tag{8.3}$$

          where E is the error, m the number of outputs, y is the known value and o the Model's output.
        * void setXEerror(): Sets the output error function to be cross-entropy, which is

$$E = \sum_{i=1}^{m} -y\log(o) - (1-y)\log(1-o) \tag{8.4}$$

          Note: for the cross-entropy error function, the output **must** *be* **discrete**.

- *Protected* members
    - DataSet *theData*: the primary dataset, in the form of a *DataSet* object

- Matrix< double > *Train*: a Matrix that contains the training set input columns *only*, constructed by a call to the protected utility function *extractInputMatrices()*
- Matrix< double > *Test*: a Matrix that contains the test set input columns *only*, constructed by a call to the protected utility function *extractInputMatrices()*
- Matrix< double > *TrainOutput*: a Matrix that contains the training set output columns *only*, constructed by a call to the protected utility function *extractOutputMatrices()*
- Matrix< double > *TestOutput*: a Matrix that contains the test set output columns *only*, constructed by a call to the protected utility function *extractOutputMatrices()*
- bool *builtFlag*: a flag to indicate if the *Model* has been formally constructed
- bool *historyFlag*: a flag to indicate if operations will be logged to a history file, this file is *appended*
- bool *lastopFlag*: a flag to indicate if model building (training in the case of *Iterative*) will be logged to a file describing the last operation, this file is *overwritten*
- bool *errorType*: flag indicates type of error function, 0 if least mean squared, 1 if cross-entropy
- bool *builtFlag*: a flag to indicate the type of error function, 0 if least mean squared, 1 if cross-entropy
- string *objType*: the name of the object, see section 8.5.1
- string *historyFilename*: the name of history file
- string *lastopFilename*: the name of the file to log the last operation
- string *errorLabel*: a string containing the name of the output error function

- *Protected* Utility functions
  - void extractInputMatrices(): Utility function which is intended to be called by the derived *Model* in its *setDataSet* method, and places the input columns of the training set in Matrix *Train*, and the input columns of the test set in Matrix *Test*
  - void extractOutputMatrices(): Utility function which is intended to be called by the derived *Model* in its *setDataSet* method, and places the output columns of the training set in Matrix *TrainOutput*, and the output columns of the test set in Matrix *TestOutput*
  - bool addHistory(ostringstream& *outputStream*): Utility function to append the ostringstream object *outputStream* to the history file.
  - void copy(const Model& rhs): Copy utility function, see section 8.5.2 for discussion.

8.7 What you really need to know about programming an Iterative Model

Iterative inherits methods setDataSet, train, reportAccuracy and outputHeader from *Model*. If you are programming an *Iterative Model*, there are 2 pure virtual methods in *Iterative* that you must also implement:

trainSet

Iterative handles models that require multiple passes (iterations) to train. Objects derived from *Iterative* must thus provide a method to adjust a model by a single pass through the training set, and *Iterative* will call this method iteratively to reach the final trained state. Thus the method:

```
virtual double trainSet()
```

changes the components of a *Model* (e.g. weight Matrices and vectors in a neural network), and returns the set error. See SimpleProp::trainSet() and BareProp::trainSet() for examples.

classAccuracy

With each iteration, *Iterative* reports the set error and classification accuracy for the training and/or test sets if the outputs are discrete. Thus, the pure virtual method:

```
virtual void classAccuracy( ostream& outputStream )
```

must output to an ostream object the classification accuracy for the training and/or test sets. *If you are programming a* Network *object, you* **do not** *need to implement this virtual method, as* Network *implements classAccuracy for you.*

getGradMax

Most iterative algorithms employ some form of gradient descent, and the iterative class can report the maximum absolute gradient and use it to determine cessation of training. The pure virtual method:

```
virtual double getGradMax()
```

must be coded to return the maximum absolute gradient of the derived object. *If you are programming a* Network *object, you* **do not** *need to implement this virtual method, as* Network *implements getGradMax for you.* However, you must code Network::pack() to create the *Network* protected member *stackG* so that Network::getGradMax() returns the proper value.

runHeader

If you are adding parameters specific to an *Iterative* derived object that should be output to screen or log files prior to an *Iterative* run, you will need to implement a runHeader method in the derived class:

```
virtual void runHeader( ostream& outputStream)
```

which outputs to an ostream object the specific parameters to be displayed. *If you are programming a* Network *object, you will need to add code to this method in* Network.

As with any derivation of *Model*, a copy utility function, copy constructor and overloaded = operator must be implemented. See section 8.5.2 for details.

iteration

You may access the iteration index in your derived object by the protected member unsigned *iteration*.

boundsErrorFlag

If your object can generate a numerical bounds error (see section 8.1.3 for an example of a method which generates a numerical bounds error), then use the protected member bool *boundsErrorFlag* to indicate if such an error has been thrown.

gradMaxFlag

Use the protected member bool *gradMaxFlag* to determine if a gradient is to be constructed in your derived object.

8.8 Iterative

Iterative extends *Model*. Neural computational algorithms are iterative, and thus use *Iterative* as a base. *Iterative* manages methods and parameters related to iterating through a dataset.

In the following specification, methods and members that pertain directly to coding a concrete derived *Iterative* implementation are shown in red .

- *Public* methods
  - Copy constructor and overloaded = operator, see section 8.5.2 for discussion:
    - Iterative& operator= (const Iterative& rhs)
    - Iterative(const Iterative& rhs)

- Methods pertaining to stopping training
  - void setMaxIterations(unsigned *maxIterations*): Sets the maximum number of iterations through the training set to *maxIterations*
  - unsigned getMaxIterations(): Returns the maximum number of iterations through the training set
  - void setMinStop(bool *minStopFlag*): Flag sets if a minimum error threshold will be used to stop training
  - bool getMinStop(): Returns flag indicating if a minimum error threshold will be used to stop training
  - void setMinError(double *minError*): Sets minimum error threshold value to *minError*
  - double getMinError(): Returns minimum error threshold value
  - void setChangeStop(bool *changeStopFlag*): Flag sets if training stops after error increases over 1 iteration
  - bool getChangeStop(): Returns flag indicating if change over 1 iteration will be used to stop training
  - void setChange(double Δ): Sets limit of change value to Δ
  - double getChange(): Returns limit of change value
  - void setWindowStop(bool *windowStopFlag*): Flag sets if training stops if set error increases over a specified window width
  - bool getWindowStop(): Returns flag indicating if set error increases over a window width will be used to stop training
  - void setWindow(unsigned *window*): Sets the value of the window width to *window*
  - unsigned getWindow(): Returns specified window width value
  - void setGradStop(bool *gradMaxFlag*): Flag sets if training will stop when the maximum absolute gradient decreases below the limit
  - bool getGradStop(): Returns flag indicating if training will stop when the maximum absolute gradient decreases below the limit
  - void setGradMaxLimit(double *limit*): Sets the value of the maximum absolute gradient limit to *limit*
  - double getGradMaxLimit(): Returns the value of the maximum absolute gradient limit
- Methods pertaining to printing while training
  - void setLogPrint(bool *logPrintFlag*): Sets if prints out by powers of 10 (1-10 by ones, 10-100 by 10s, 100-1000 by 100s, etc.)
  - bool getLogPrint(): Returns flag indicating if printing by powers of 10
  - void setPrintCount(unsigned *printCount*): Sets the linear print counter to *printCount*

        * unsigned getPrintCount(): Returns the linear print counter
    - General methods
        * virtual void setDataSet(DataSet& *data*): A pure virtual method inherited from *Model* that loads the *DataSet* object *data* into the *Model*
        * virtual double train(): A pure virtual method inherited from *Model* that trains a *Model*, and returns the final error (double) for that model
        * virtual void reportAccuracy(ostream& *outputStream*): A pure virtual method inherited from *Model* that reports the accuracy of the trained model to an ostream *outputStream*
        * virtual void outputHeader(ostream& *outputStream*): A pure virtual method inherited from *Model* which outputs a header to an ostream *outputStream* describing the architecture of the model, *Model* type dependant
        * virtual double trainSet(): A pure virtual method which trains one iteration through the training set, *Iterative Model* type dependant, and returns the set error
        * virtual void classAccuracy(ostream& *outputStream*): A pure virtual method which outputs classification accuracies to an ostream *outputStream* for an *Iterative* output table

- *Protected* members
    - unsigned *iteration*: the iteration index
    - unsigned $maxIterations$: the maximum number of iterations through the training set
    - bool $logPrintFlag$: flag indicates if printing by powers of 10
    - unsigned *printCount*: the value for the linear print counter
    - bool $minStopFlag$: flag indicates if a minimum error threshold will be used to stop training
    - double *minError*: minimum error threshold value
    - bool *changeStopFlag*: flag indicates if change in error over 1 iteration will be used to stop training
    - double *delta*: value of change over 1 iteration
    - bool $gradMaxFlag$: flag indicates if training will stop when maximum absolute gradient decreases below $gradMaxLimit$
    - double $gradMaxLimit$: value of the maximum absolute gradient limit
    - bool $windowStopFlag$: flag indicates if set error increases over a window width will be used to stop training

- unsigned *window*: value of the window width
- bool *boundsErrorFlag*: flag indicates if out of numerical bounds error encountered (e.g. log(0))

- *Protected* Utility functions
  - void copy(const Iterative& rhs): Copy utility function, see section 8.5.2 for discussion.
  - virtual void runHeader(ostream& *outputStream*): A pure virtual utility function that outputs parameters specific to derived objects preceeding an Iterative run
  - virtual double getGradMax(): A pure virtual utility function that returns the maximum absolute gradient value

8.9 What you really need to know about programming a Network Model

8.9.1 Implementation of pure virtual methods in *Network*

Network inherits methods setDataSet, train, reportAccuracy and outputHeader from *Model*, and trainSet from *Iterative*, so you must code all of these methods. (See the respective methods in SimpleProp and BareProp as examples.) If you are programming a *Network Model*, there are 7 additional pure virtual methods in *Network* that you must implement in the concrete derived object:

df

The method

```
virtual unsigned df()
```

must be coded to return the number of degrees of freedom of your *Network* object, which is equal to the number of nodal connections. See SimpleProp::df and BareProp::df as examples.

randomize

The method

```
virtual void randomize()
```

must be coded to randomize the weights in your *Network* object. See SimpleProp::randomize and BareProp::randomize as examples.

load and save

The methods

```
virtual bool load( string& filename )
virtual bool save( string& filename )
```

must be coded to load and save your *Network* derived object from and to a file named "filename". Typically the method save would use your coded method outputHeader to write the *Network* object architecture to a file, then the Matrices and vectors representing the *Network* model. As expected, load should do the opposite. See SimpleProp::load, SimpleProp::save, BareProp::load and BareProp::save as examples.

innerTrainSet

The method

```
virtual double innerTrainSet()
```

must be coded in a *Network* object to calculate the gradient and error. The reason that another method in addition to the pure virtual method trainSet inherited from *Iterative* must be coded is to support automatic stepsize selection. The strategy proceeds as follows: trainSet is intended to call innerTrainSet to determine if the set error would increase, and set the learning rate ("step size") accordingly lower if it would. Thus, innerTrainSet calculates the gradient and error, but does not necessarily update the weights. If automatic stepsize *is not* selected, trainSet simply calls innerTrainSet to calculate the gradient and error, and updates the weights. If automatic stepsize *is* selected, trainSet calls innerTrainSet to calculate the gradient and error, and reduces the stepsize before updating the weights if the error is found to increase. See SimpleProp::innerTrainSet, BareProp::innerTrainSet and BackProp::innerTrainSet as examples of the way trainSet and innerTrainSet are coded to work together.

Richard Golden has suggested the following automatic stepsize selection algorithm, which he terms the "Crummy Back-tracking Line Search Algorithm":

- Let DELTAERROR be a small non-negative number (e.g., DELTAERROR = 1e-10 or 0)
- Let MAXLOOPS = a small positive integer (e.g., MAXLOOPS = 2 or 3 or 4)
- Let GAMMA be some positive number less than 1 (e.g., GAMMA = 0.5 or 0.8)
- Let LoopCounter = 0
- Set "candidate" learning rate equal to 1.0
- While Exit Loop is FALSE

- LoopCounter = LoopCounter + 1
- Compute change to weights using current candidate learning rate
- Evaluate the new ERROR at the weights you obtained to get a "candidate error change"
- If ("candidate error change" < DELTAERROR) OR (LoopCounter > MAXLOOPS), then EXITLOOP is set to TRUE
- Else set current candidate learning rate = GAMMA [candidate learning rate]
- End While Loop

Don't be fooled by the name–this algorithm is far from "crummy".

Note: This algorithm is only to be used with off-line or batch/epoch learning.

forward

The method

```
virtual void forward( Matrix< double >& M, const unsigned r )
```

must be coded to take a Matrix containing only the inputs of a training or test set as one argument (typically the *Train* or *Test* inherited protected members from *Model*), the position of the exemplar in the training or test set as a second argument, and derive the *Network's* output. *This is not returned by the method. This is very important: in single output* Networks, *the inherited* Network *protected member o would be set by this method, and in multiple output* Networks, *the inherited* Network *protected member vector< double > output would be set by this method. Also very important, if sigmoidal output nodes are supported, the inherited* Network *protected member x would be set by this method, and in multiple output* Networks, *the inherited* Network *protected member vector< double > xs would be set by this method.*

removeInputs

The method

```
virtual void removeInputs( vector< unsigned >& v )
```

must be coded to take as its argument a vector of unsigned representing positions of input nodes, and to remove those nodes from the *Network* derived object. Typically, this would be accomplished first by a call to *DataSet*::removeInputs(vector< unsigned >& *v*), followed by a call to *Model*::extractInputMatrices(), followed by code which removes input nodes in the structures representing weights in the *Network* object, typically *Matrix* columns. See SimpleProp::removeInputs and BareProp::removeInputs as examples.

8.9.2 Code that must be added to *RegressNet*

If you are creating a regressable *Network* object (and chances are that you are,) you *must* add code to the methods RegressNet::copy_network() and RegressNet::network_name(). See section 9.6 for details.

8.9.3 *Network* protected members

The following are the protected members in *Network* that your object needs to set or get:

trainingType

The protected member

```
unsigned trainingType
```

determines the type of gradient descent algorithm of your neural network. $trainingType = 0$ is defined as canonical backpropagation. If you add other types of training to the *Network* class, you must define a unique $trainingType$ for them. Currently coded algorithms include $trainingType =$

1. Conjugate gradient descent
2. Shanno algorithm

NOTE: Look closely at the code for Network::getGradMax(). You will find that this code assumes that if $trainingType$ is not 0, then $stackG$ will be constructed. If this is not true for your $trainingType$, then you must modify the if block in Network::getGradMax().

o

The protected member

```
double o
```

is the output for a single output network. *If your* Network *object supports single outputs, it must set this protected member,* typically in the method forward. Since the method trainSet will typically call the method forward, trainSet will generally access the member o. See SimpleProp::forward and BareProp::forward for examples in setting the protected member o, and SimpleProp::trainSet and BareProp::trainset for examples in accessing the protected member o.

x

The protected member

```
double x
```

contains the x term for the sigmoidal output o in a single output network:

$$o = 1/(1 + e^{-x})$$

If your Network *object supports single sigmoidal outputs, it must set this protected member,* typically in the method forward. See SimpleProp::forward and BareProp::forward for examples in setting the protected member x.

output

The protected member

```
vector< double > output
```

is the output for a multiple output network. *If your* Network *object supports multiple outputs, it must set this protected member,* typically in the method forward. Since the method trainSet will typically call the method forward, trainSet will generally access the member *output*.

xs

The protected member

```
vector< double > xs
```

contains the $\vec{\mathbf{x}}$ terms for the sigmoidal outputs $\vec{\mathbf{o}}$ in a multiple output network:

$$\vec{\mathbf{o}} = 1/(1 + e^{-\vec{\mathbf{x}}})$$

If your Network *object supports multiple sigmoidal outputs, it must set this protected member,* typically in the method forward. See BackProp::forward for an example in setting the protected member xs.

randomLimit

The protected member

```
double randomLimit
```

is used to specify the range of randomized weights, which will be between $-randomLimit$ and $+randomLimit$. Access this protected member in your randomize method. See SimpleProp::randomize and BareProp::randomize for examples.

weightsSetFlag

Your derived *Network* object must set the protected member

```
bool weightsSetFlag
```

to *true* when the weights are set, either by randomization or loading weights from a file. See SimpleProp::randomize, BareProp::randomize, SimpleProp::load and BareProp::load for examples of setting $weightsSetFlag$ to $true$, and SimpleProp::setHidden and BareProp::setHidden for examples of setting $weightsSetFlag$ to $false$.

biasFlag

Use the protected member

```
bool biasFlag
```

to determine if your *Network* object has biases.

eta

Use the protected member

```
double eta
```

for the learning rate η of your *Network* object.

batchEpochFlag

Use the protected member

```
bool batchEpochFlag
```

to determine if your *Network* object uses batch/epoch learning or not. (See the description of classic off-line ("batch") backprop in section 2.2.2.) *Pay special attention to how the learning rate is set.* It is expected that the driver will guide the user to the appropriate setting of the learning rate. Note also that "off- line" learning is synonymous with " batch" or "epoch" learning, whereas "on-line" learning refers to updating the weights with each exemplar. Here, if batchEpochFlag is *true*, "offline" learning is invoked, and if batchEpochFlag is *false*, "online" learning is used.

weightDecayFlag

Use the protected member

```
bool weightDecayFlag
```

to determine if your *Network* object uses weight decay. (See the discussion of equation 2.1 in section 2.1.2 for an explanation of weight decay.) The learning rate equivalent of equation 2.1 is

$$\begin{aligned} \vec{w}_{t+1} &= \vec{w}_t - \eta \left.\frac{\partial E'}{\partial w}\right|_{w_t} \\ &= \vec{w}_t - \eta \left.\frac{\partial E}{\partial w}\right|_{w_t} - 2\eta\lambda\vec{w}_t \\ &= (1 - 2\eta\lambda)\vec{w}_t - \eta \left.\frac{\partial E}{\partial w}\right|_{w_t} \end{aligned}$$

where $(1 - 2\eta\lambda)$ is "weight decay". λ may be set by either of 2 methods (N is the sample number size, see equation 2.1 in section 2.1.2 for a discussion of σ_w):

- The complexity of the population objective function >> sample objective function:

$$\lambda = \frac{1}{N 2\sigma_w^2} \tag{8.5}$$

- The complexity of the population objective function $\simeq$ sample objective function:

$$\lambda = \frac{1}{2\sigma_w^2} \tag{8.6}$$

It is anticipated that the driver will guide the user to set the appropriate value of λ, and *insure that the user knows what he or she's doing if they change it!*

decay

You can use the protected member

```
double decay
```

for the weight decay term 2λ of your *Network* object. Note the multiple of 2 so that calculation of decay with the gradient as in the right hand term of equation 2.10, $(1/\sigma_w^2)[\mathbf{W}\ ,\ \mathbf{b}]$, does not require the additional step of multiplication by 2.

regularizer

You can use the protected member

```
double regularizer
```

if you need to access λ directly, e.g. for of calculation of your error function in equation 2.1, which like $decayTerm$ only needs to be calculated once prior to iterative calls to the TrainSet method, and is thus placed in the member function runHeader, where the *Iterative* class will call it once prior to iteratively calling the TrainSet method.

decayTerm

In the protected member function runHeader in the *Network* class,

```
double decayTerm
```

is defined to be $1 - 2\eta\lambda$ (or $1 - \eta decay$), which only needs to be calculated once prior to iterative calls to the TrainSet method, and is thus placed in the member function runHeader, where the *Iterative* class will call it once prior to iteratively calling the TrainSet method. You may therefore use $decayTerm$ as the weight multiple for weight decay in your *Network* object if the decay is not calculated with the gradient. If the decay is calculated with the gradient as in equation 2.10, then $decay$ is used. See BareProp::TrainSet and SimpleProp::TrainSet for examples using $decayTerm$.

stackG

The protected member

```
vector< double > stackG
```

is a single vector which contains the entire weight gradient of your *Network* derived object. It is used by the protected utility functions pack and unpack (see below).

8.9.4 *Network* protected utility functions

Copy utility function, constructor and overloaded = operator

As with any derivation of *Model*, a copy utility function, copy constructor and overloaded = operator must be implemented. See section 8.5.2 for details.

runHeader utility function

If you are adding parameters specific to a *Network* object that should be output to screen or log files prior to an *Iterative* run, you will need to add code to the runHeader method in *Network*. See section 8.7.

pack and unpack

The methods

```
virtual void pack()
virtual void unpack()
```

must be coded to convert the weight gradient structure of your *Network* derived object to the member vector *stackG* (pack), and vice versa (unpack). See SimpleProp::pack, SimpleProp::unpack, BareProp::pack and BareProp::unpack as examples.

8.9.5 Built-in gradient descent algorithms

One of the really neat things about *Networks* that construct the *stackG* vector is that they may use the built-in advanced gradient descent algorithms, Shanno's and conjugate gradient descent.[4] All the programmer needs to do is to construct the *stackG* vector, add the line of code

```
engine( trainingType, iteration );
```

after the *Network* object gradient structure is calculated and before the weights are updated, and these algorithms will automagically be available for the user. See SimpleProp::trainSet() and BareProp::trainSet() for examples.

Shanno algorithm

Briefly, Shanno's algorithm proceeds as follows: let the computed gradient be $\vec{g}$ (*stackG*), and the iteration be t (**Note:** the iteration is *not* the exemplar index through the training set. It is the actual iteration number for each calculated gradient. It should also be noted that this algorithm, as well as conjugate gradient descent *is intended for off-line or batch/epoch learning.*) Let the learning rate be γ.

- Step 0: Let $t = 0$.
- Step 1: Set $\vec{f}(t) = -\vec{g}_t$. Go to step 3.
- Step 2: If t is a multiple of the degrees of freedom of the *Network*, then go to step 1. Otherwise, compute $\vec{u}_t = \vec{g}_t - \vec{g}_{t-1}$ and the scalars a_t, b_t, c_t such that
  - $a_t = \frac{\vec{f}(t-1)^T \vec{g}_t}{\vec{f}(t-1)^T \vec{u}_t}$,
  - $b_t = \frac{\vec{u}^T \vec{g}_t}{\vec{f}(t-1)^T \vec{u}_t}$,
  - $c_t = \gamma + \frac{|\vec{u}_t|^2}{\vec{f}(t-1)^T \vec{u}_t}$,

[4] Richard M. Golden, Mathematical Methods for Neural Network Analysis and Design, 1996, MIT Press, ISBN 0-262-07174-6, pp. 217–218, 221–222

and descent vector

$$\vec{f}(t) = -\vec{g}_t + a_t\vec{u}_t + (b_t - c_t a_t)\vec{f}(t-1)$$

- Step 3: Update the weights by $\vec{w}(t+1) = \vec{w}(t) + \gamma\vec{f}(t)$. Let $t = t+1$. Go to step 2.

Interested readers are directed to Richard Golden's book, pp. 217–218. It should be noted that in the highly optimized code in Network::shanno(unsigned *iteration*), *lastF* refers to $-\vec{f}(t-1)$.

Conjugate gradient descent

Conjugate gradient descent is a special case of the Shanno algorithm, and briefly proceeds as follows: let the computed gradient be $\vec{g}$ (*stackG*), and the iteration be t (**Note:** again, the iteration is *not* the exemplar index through the training set. It is the actual iteration number for each calculated gradient. It should also be noted that this algorithm, as well as Shanno's, *is intended for off-line or batch/epoch learning.*) Let the learning rate be γ.

- Step 0: Let $t = 0$.
- Step 1: Set $\vec{f}(t) = -\vec{g}_t$. Go to step 3.
- Step 2: If t is a multiple of the degrees of freedom of the *Network*, then go to step 1. Otherwise, compute $\vec{u}_t = \vec{g}_t - \vec{g}_{t-1}$ and the scalar b_t such that

$$b_t = \frac{\vec{u}^T\vec{g}_t}{\vec{u}_t^T\vec{f}(t-1)}$$

  and descent vector

$$\vec{f}(t) = -\vec{g}_t + b_t\vec{f}(t-1)$$

- Step 3: Update the weights by $\vec{w}(t+1) = \vec{w}(t) + \gamma\vec{f}(t)$. Let $t = t+1$. Go to step 2.

Interested readers are directed to Richard Golden's book, pp. 221–222. It should be noted that in the highly optimized code in Network::CGD(unsigned *iteration*), *lastF* refers to $-\vec{f}(t-1)$.

8.9.6 Calculation of the condition number

The overloaded storeGrads(...) method, the reportCondNum(ostream& *outputStream*) method and the $finalFlag$ flag may be used to compute the condition number of a *Network* object. The condition number is the ratio of the largest to the smallest eigenvalue of the symmetric matrix **B** computed by the outer product of the average gradient vector and its transpose:

$$\mathbf{B} = \mathbf{g} \cdot \mathbf{g}^T$$

The *Gnu Scientific Library (GSL)* eigenvalue routines were ported for use in these methods. Note that condition numbers of neural networks are highly experimental, and in the current driver, the condition number is calculated automatically only for logistic regression (e.g. *Logistic*) type *Network* objects. However, implementation of condition number calculation is as general utility methods in the *Network* class so that future programmers may report condition numbers of *Network* objects if desired.

Computation of the condition number for a *Network* derived object proceeds as follows:

1. The overloaded storeGrads(unsigned d, unsigned n) method is first used to initialize a gradient accumulator matrix ($grads$, which is transparent to the programmer and does not need to be included in added code) which will ultimately be used to compute the **B** matrix. Arguments for this overloaded method are d, the dimension (size) of the gradient vector, and n, the number of exemplars in the dataset.

2. The gradient vector *stackG* must be created using the *pack*() method.

3. Because methods involved in computation of the condition number are highly intensive, and because the condition number only requires the final average gradient vector, the $finalFlag$ flag is used to indicate a final iteration for the following step ...

4. The overloaded storeGrads(unsigned n) method is used to accumulate exemplar n's gradient *stackG* (which does *not* need to be passed as an argument, as it must be generated in code prior to use of this method.) Note that the *Network* method reportCondNum(ostream& $outputStream$) will automatically compute the average gradient, hence that particular exemplar's gradient is loaded (again, transparent to the programmer.)

5. reportCondNum(ostream& $outputStream$) is used in methods which report results to the user, e.g. reportAccuracy(ostream& $outputStream$)

See *Logistic* for example code, and see *SimpleProp* for comments which indicate where code would be placed in a neural network style implemented *Network* object.

8.10 Network

In the following specification, methods and members that pertain directly to coding a concrete derived *Network* implementation are shown in red .

Network extends *Iterative*. Algorithms with hidden layers, weights, learning rates, transfer functions and other general properties of neural networks are encapsulated in *Network*.

Note: It is expected that in objects derived from *Network*, the method setDataSet inherited from *Model* be called *before* the derived methods that specify object architecture, so that the derived object may know the number of input nodes and size the Matrices related to the dataset prior to completing its architecture. This sequence of loading a dataset using the inherited *Model* method setDataSet prior to establishing network architecture will be invoked for all *Network* objects.

- *Public* methods
  - Network& operator= (const Network& rhs): see section 8.5.2 for discussion
  - Network(const Network& rhs): see section 8.5.2 for discussion
  - virtual void setDataSet(DataSet& *data*): A pure virtual method inherited from *Model* that loads the *DataSet* object *data* into the *Model*
  - virtual void outputHeader(ostream& *outputStream*): A pure virtual method inherited from *Model* which outputs a header to an ostream *outputStream* describing the architecture of the model, *Model* type dependant
  - void setEta(double η): Sets the learning rate to η
  - double getEta(): Returns the learning rate η
  - void setBias(bool $flag$): Sets bias nodes in *Network*
  - bool getBias(): Gets boolean value indicating if bias nodes specified in *Network*
  - void setBatchEpoch(bool $flag$): Sets if batch/epoch learning is used in a *Network* object (See the description of classic off-line ("batch") backprop in section 2.2.2)
  - bool getBatchEpoch(): returns if batch/epoch used
  - void setWeightDecay(bool $flag$): Sets if weight decay is used in a *Network* object (See the discussion of weight decay in section 8.9.3)
  - bool getWeightDecay(): returns if weight decay used
  - void setDecay(double 2λ): Sets weight decay term to 2λ in a *Network* object
  - double getDecay(): returns weight decay term λ
  - void setTrainingType(unsigned T): Sets training type to T
  - unsigned getTrainingType(): returns training type

- void setAutoStepSize(bool $flag$): Sets if automatic stepsize selection is used (See the discussion of automatic stepsize selection in section 8.9.1)
- bool getAutoStepSize(): returns if automatic stepsize selection is used
- virtual unsigned df(): Returns the number of degrees of freedom, equal to the number of nodal connections
- void setRandomLimit(double L): Sets the limit for the random weights $-L$ to $+L$
- virtual void randomize(): A pure virtual method which randomizes initial weights in a *Network* object
- getWeightsSet(): Returns a flag to indicate if weights set
- virtual bool save(string& $filename$): A pure virtual method which saves the *Network* object architecture and weights to a file named $filename$, returns true if successful
- virtual bool load(string& $filename$): A pure virtual method which loads the *Network* object architecture and weights from a file named $filename$, returns true if successful
- virtual double trainSet(): A pure virtual method which trains one iteration through the training set, *Network* type dependant, and returns the set error
- virtual double innerTrainSet(): A pure virtual method which is called by trainSet() to calculate the gradient and return set error, used for automatic stepsize selection
- virtual void forward(Matrix< double >& M, unsigned n): A pure virtual method which forward propagates for one input row n in a Matrix of inputs M (e.g. either the inherited *Model* Matrix $Train$ or $Test$)
- virtual void reportAccuracy(ostream& *outputStream*): Implementation of the pure virtual method inherited from *Model* that reports the accuracy of the trained model to an ostream *outputStream*
- virtual void classAccuracy(ostream& *outputStream*): A pure virtual method inherited from *Iterative* which outputs classification accuracies to an ostream *outputStream* for an *Iterative* output table
- virtual void removeInputs(vector< unsigned >& v): A pure virtual method which takes as its argument a vector of unsigned v representing positions of input nodes, and removes those nodes from the *Network* object

- *Protected* members

- unsigned *trainingType*: the type of gradient descent algorithm, $trainingType = 0$ is reserved for canonical backpropagation ($trainingType = 1$ has been coded for conjugate gradient descent, and $trainingType = 2$ for Shanno algorithm)
- double o: the output for a single-output *Network*
- double x: the x term for the sigmoidal output o in a single output network:
$$o = 1/(1 + e^{-x})$$
- vector < double > *output*: the outputs vector for a multiple output *Network*
- vector < double > xs: the $\vec{\mathbf{x}}$ terms for the sigmoidal outputs $\vec{\mathbf{o}}$ in a multiple output network:
$$\vec{\mathbf{o}} = 1/(1 + e^{-\vec{\mathbf{x}}})$$
- double *randomLimit*: the limit for the random weights
- bool *weightsSetFlag*: a flag to indicate if weights set
- bool *biasFlag*: a flag to indicate if bias nodes specified in *Network*
- bool *batchEpochFlag*: a flag to indicate if batch/ epoch training is used in the *Network* object (See the description of classic off-line ("batch") backprop in section 2.2.2)
- bool *weightDecayFlag*: a flag to indicate if weight decay is used in the *Network* object (See the discussion of weight decay in section 8.9.3)
- double *decay*: the weight decay term 2λ
- double *regularizer*: the weight decay term λ
- double *decayTerm*: the weight decay multiple $1 - 2\eta\lambda$ calculated in the protected member function runHeader (see section 8.9.3 for an explanation)
- double *eta*: the learning rate η
- bool *automaticStepSizeFlag*: a flag to indicate if automatic step size algorithm is used
- double *o_errAccumulate*: stores average output error accumulation over the examplars used in innerTrainSet() method
- double *oldErrorAccumulate*: stores average output error accumulation for previous run over the examplars, used in innerTrainSet() method
- double *deltaError*: deltaError constant for automatic stepsize selection
- double *currGradMax*: current absolute maximum gradient for shanno/CGD

- double *gamma*: gamma constant for automatic stepsize selection
- unsigned *maxLoops*: maxLoops constant for automatic stepsize selection
- vector < double > *stackG*: a single vector which contains the entire weight gradient of the *Network* derived object
- vector < double > *lastG*: $\vec{g}(t-1)$ for shanno/CGD
- vector < double > *lastF*: $\vec{f}(t-1)$ for shanno/CGD
- bool *finalFlag*: a flag to indicate that the train methods (innerTrainSet() or trainSet()) are at the final iteration, for code to be run only during the final iteration
- Matrix < double > *grads*: a matrix to hold exemplar gradients to be used to compute the B matrix to calculate the condition number

- *Protected* Utility functions
  - void copy(const Network& rhs): Copy utility function, see section 8.5.2 for discussion.
  - virtual void runHeader(ostream& *outputStream*): A pure virtual utility function inherited from the Iterative class that outputs Network specific parameters preceeding an Iterative run
  - virtual void pack(): A pure virtual utility function that converts the entire weight gradient structure of the *Network* derived object to the protected member *stackG*
  - virtual void unpack(): A pure virtual utility function that converts the protected member *stackG* back to the weight gradient structure of the *Network* derived object
  - virtual double getGradMax(): A pure virtual utility function inhertied from *Iterative* that returns the maximum absolute gradient
  - void engine(unsigned *type*, unsigned *iteration*): an utility function which calls the appropriate training algorithm depending on training type *type*, and passes it the current iteration
  - void engine(unsigned *type*, unsigned *iteration*): chooses which kind of advanced gradient descent algorithm (CGD or shanno) based on *type*, and passes *iteration* to it
  - void CGD(unsigned *iteration*): implements conjugate gradient descent (see Golden pp. 221–222)[5]
  - void shanno(unsigned *iteration*): implements Shanno algorithm (see Golden pp. 217–218)

[5]Richard M. Golden, Mathematical Methods for Neural Network Analysis and Design, 1996, MIT Press, ISBN 0-262-07174-6

- void storeGrads(unsigned d, unsigned n): (used in computing condition number) overloaded utility method (see below), with 2 arguments d and n, initializes *grads* matrix with dimension d of packed gradient (size), and number of exemplars n
- void storeGrads(unsigned n): (used in computing condition number) overloaded utility method (see above), with 1 argument n accumulate grads matrix with that exemplar n's gradient *stackG* (note that *stackG* must have been previously packed using pack()
- void reportCondNum(ostream& *outputStream*): utility method to report out the condition number to *outputStream*

8.11 BareProp

The **BareProp** class is a *Network* object that serves to illustrate neural network design in neUROn2++. *BareProp* is a one output node, one hidden layer neural network without biases.

Note: It is expected that the method setDataSet inherited from *Model* be called *before* the *BareProp* method setHidden, so that the *BareProp* object may know the number of input nodes and size the Matrices related to the dataset prior to completing its architecture. This sequence of loading a dataset using the *Model* method setDataSet prior to establishing network architecture will be invoked for all *Network* objects.

- *Public* methods
  - BareProp& operator= (const BareProp& rhs): see section 8.5.2 for discussion
  - BareProp(const BareProp& rhs): see section 8.5.2 for discussion
  - virtual void setDataSet(DataSet& *data*): Implementation of the pure virtual method inherited from *Model* that loads the *DataSet* object *data* into the *BareProp Model*
  - virtual void outputHeader(ostream& *outputStream*): Implementation of the pure virtual method inherited from *Model* which outputs a header to an ostream *outputStream* describing the architecture of the *Bareprop Model*
  - void setHidden(unsigned H): Sets number of hidden nodes to H, which is all that is required to specify the architecture for a *BareProp* object
  - virtual unsigned df(): Implementation of the pure virtual method inherited from *Network* which returns the number of degrees of freedom, equal to the number of nodal connections
  - virtual void randomize(): Implementation of the pure virtual method inherited from *Network* which randomizes initial weights

- virtual bool save(string& $filename$): Implementation of the pure virtual method inherited from *Network* which saves the *BareProp* object architecture and weights to a file named $filename$, returns true if successful
- virtual bool load(string& $filename$): Implementation of the pure virtual method inherited from *Network* which loads the *BareProp* object architecture and weights from a file named $filename$, returns true if successful
- virtual double trainSet(): Implementation of the pure virtual method inherited from *Iterative* which trains one iteration through the training set, and returns set error
- virtual double innerTrainSet(): Implementation of the pure virtual method inherited from *Network* which is called by trainSet() to calculate the gradient and return set error, used for automatic stepsize selection
- virtual void forward(Matrix< double >& M, unsigned n): Implementation of the pure virtual method inherited from *Network* which forward propagates for one input row n in a Matrix of inputs M (e.g. either the inherited *Model* Matrix $Train$ or $Test$)
- virtual void removeInputs(vector< unsigned >& v): Implementation of the pure virtual method which takes as its argument a vector of unsigned v representing positions of input nodes, and removes those nodes from the *BareProp* object

- *Private* members
  - unsigned $nHidden$: number of hidden nodes
  - Matrix< double > hW: hidden weight Matrix
  - Matrix< double > $hWup$: hidden weight update Matrix
  - Matrix< double > hG: hidden weight gradient Matrix
  - vector< double > y: vector containing dataset outputs
  - vector< double > I: vector for single exemplar inputs
  - vector< double > hO: hidden output vector
  - vector< double > oW: output weight vector
  - vector< double > oG: output weight gradient vector
  - vector< double > h_err: hidden error vector
  - double o_err: output error term

- *Private* Utility functions
  - void copy(const BareProp& rhs): Copy utility function, see section 8.5.2 for discussion.

- virtual void pack(): Implementation of the pure virtual utility function inherited from *Network* that converts the entire weight gradient structure to the *Network* inherited protected member *stackG*
- virtual void unpack(): Implementation of the pure virtual utility function inherited from *Network* that converts the *Network* inherited protected member *stackG* back to the weight gradient structure

8.12 SimpleProp

The **SimpleProp** class is a *Network* object that serves to illustrate neural network design in neURON2++. *SimpleProp* is a one output node, one hidden layer neural network with biases.

Note: It is expected that the method setDataSet inherited from *Model* be called *before* the *SimpleProp* method setHidden, so that the *SimpleProp* object may know the number of input nodes and size the Matrices related to the dataset prior to completing its architecture. This sequence of loading a dataset using the *Model* method setDataSet prior to establishing network architecture will be invoked for all *Network* objects.

- *Public* methods
  - SimpleProp& operator= (const SimpleProp& rhs): see section 8.5.2 for discussion
  - SimpleProp(const SimpleProp& rhs): see section 8.5.2 for discussion
  - virtual void setDataSet(DataSet& *data*): Implementation of the pure virtual method inherited from *Model* that loads the *DataSet* object *data* into the *SimpleProp Model*
  - virtual void outputHeader(ostream& *outputStream*): Implementation of the pure virtual method inherited from *Model* which outputs a header to an ostream *outputStream* describing the architecture of the *Bareprop Model*
  - void setHidden(unsigned H): Sets number of hidden nodes to H, which is all that is required to specify the architecture for a *SimpleProp* object
  - virtual unsigned df(): Implementation of the pure virtual method inherited from *Network* which returns the number of degrees of freedom, equal to the number of nodal connections
  - virtual void randomize(): Implementation of the pure virtual method inherited from *Network* which randomizes initial weights
  - virtual bool save(string& $filename$): Implementation of the pure virtual method inherited from *Network* which saves the *SimpleProp* object architecture and weights to a file named $filename$, returns true if successful

  - virtual bool load(string& $filename$): Implementation of the pure virtual method inherited from *Network* which loads the *SimpleProp* object architecture and weights from a file named $filename$, returns true if successful
  - virtual double trainSet(): Implementation of the pure virtual method inherited from *Iterative* which trains one iteration through the training set, and returns set error
  - virtual double innerTrainSet(): Implementation of the pure virtual method inherited from *Network* which is called by trainSet() to calculate the gradient and return set error, used for automatic stepsize selection
  - virtual void forward(Matrix< double >& M, unsigned n): Implementation of the pure virtual method inherited from *Network* which forward propagates for one input row n in a Matrix of inputs M (e.g. either the inherited *Model* Matrix $Train$ or $Test$)
  - virtual void removeInputs(vector< unsigned >& v): Implementation of the pure virtual method which takes as its argument a vector of unsigned v representing positions of input nodes, and removes those nodes from the *SimpleProp* object

- *Private* members
  - unsigned $nHidden$: number of hidden nodes
  - unsigned nH: $nHidden - 1$, used for indexing with biases
  - Matrix< double > hW: hidden weight Matrix
  - Matrix< double > $hWup$: hidden weight update Matrix
  - Matrix< double > hG: hidden weight gradient Matrix
  - vector< double > in_bias: input bias vector to add to input Matrix
  - vector< double > y: vector containing dataset outputs
  - vector< double > I: vector for single exemplar inputs
  - vector< double > hO: hidden output vector
  - vector< double > oW: output weight vector
  - vector< double > oG: output weight gradient vector
  - vector< double > h_err: hidden error vector
  - double o_err: output error term

- *Private* Utility functions
  - void copy(const SimpleProp& rhs): Copy utility function, see section 8.5.2 for discussion.

- virtual void pack(): Implementation of the pure virtual utility function inherited from *Network* that converts the entire weight gradient structure to the *Network* inherited protected member *stackG*
- virtual void unpack(): Implementation of the pure virtual utility function inherited from *Network* that converts the *Network* inherited protected member *stackG* back to the weight gradient structure

8.13 BackProp

The **BackProp** class is a *Network* object that allows multiple output nodes and multiple hidden layers in a neural network, and biases may or may not be present.

Note: It is expected that the *Model* method setDataSet be called *before* the *BackProp* method setHidden, so that the *BackProp* object may know the number of input nodes and size the Matrices related to the dataset prior to completing its architecture. This sequence of loading a dataset using the *Model* method setDataSet prior to establishing network architecture will be invoked for all *Network* objects.

- *Public* methods
  - BackProp& operator= (const BackProp& rhs): see section 8.5.2 for discussion
  - BackProp(const BackProp& rhs): see section 8.5.2 for discussion
  - virtual void setDataSet(DataSet& *data*): Implementation of the pure virtual method inherited from *Model* that loads the *DataSet* object *data* into the *BackProp Model*
  - virtual void outputHeader(ostream& *outputStream*): Implementation of the pure virtual method inherited from *Model* which outputs a header to an ostream *outputStream* describing the architecture of the *Backprop Model*
  - void setHidden(const vector< unsigned >& v): Sets the hidden layers, with the number of hidden nodes on each layer indicated by the corresponding vector v element
  - virtual unsigned df(): Implementation of the pure virtual method inherited from *Network* which returns the number of degrees of freedom, equal to the number of nodal connections
  - virtual void randomize(): Implementation of the pure virtual method inherited from *Network* which randomizes initial weights
  - virtual bool save(string& $filename$): Implementation of the pure virtual method inherited from *Network* which saves the *BackProp* object architecture and weights to a file named $filename$, returns true if successful

- virtual bool load(string& $filename$): Implementation of the pure virtual method inherited from *Network* which loads the *BackProp* object architecture and weights from a file named $filename$, returns true if successful
- virtual double trainSet(): Implementation of the pure virtual method inherited from *Iterative* which trains one iteration through the training set, and returns set error
- virtual double innerTrainSet(): Implementation of the pure virtual method inherited from *Network* which is called by trainSet() to calculate the gradient and return set error, used for automatic stepsize selection
- virtual void forward(Matrix< double >& M, unsigned n): Implementation of the pure virtual method inherited from *Network* which forward propagates for one input row n in a Matrix of inputs M (e.g. either the inherited *Model* Matrix $Train$ or $Test$)
- virtual void removeInputs(vector< unsigned >& v): Implementation of the pure virtual method which takes as its argument a vector of unsigned v representing positions of input nodes, and removes those nodes from the *BackProp* object

- *Private* members
  - vector< unsigned > $nLayer$: describes the number of hidden nodes on each layer indicated by the corresponding vector element in $nLayer$
  - unsigned $nLayers$: number of hidden layers
  - vector< double > in_bias: input bias vector to add to input Matrix if bias nodes present
  - vector< double > I: vector for single exemplar inputs
  - vector< double > y: vector containing dataset outputs
  - vector< double > o_err: vector containing output errors
  - vector< Matrix< double > > $Weights$: vector of weight Matrix for all hidden layers and output layer
  - vector< Matrix< double > > $WeightsUp$: vector of Matrices to hold updated weights one for each weights Matrix
  - vector< Matrix< double > > $WeightsAccumulate$: vector of Matrices to hold accumulated weight updates for batch/epoch learning
  - vector< vector< double > > $HOutputs$: vector containing a vector of outputs for each hidden layer
  - vector< vector< double > > $HErrors$: vector containing a vector of errors for each hidden layer

- *Private* Utility functions

- void copy(const BackProp& rhs): Copy utility function, see section 8.5.2 for discussion.
- virtual void pack(): Implementation of the pure virtual utility function inherited from *Network* that converts the entire weight gradient structure to the *Network* inherited protected member *stackG*
- virtual void unpack(): Implementation of the pure virtual utility function inherited from *Network* that converts the *Network* inherited protected member *stackG* back to the weight gradient structure

8.14 Logistic

The **Logistic** class is a *Network* object that implements logistic regression. This form of data modeling is mathematically equivalent to a single output node neural network with a bias, the sigmoidal transfer function, no hidden nodes, and the cross-entropy error function. As a result, logistic regression is implemented as a derived Network class. This strategy also allows the user to perform stepwise forward and reverse regression on the built logistic regression model using the RegressNet class.

Note that the error is cross-entropy by definition, that the gradient descent method is batch-epoch by definition, and a bias node (y-intercept) is present by definition. At the end of a run, the β weights and standard error are reported and p-values for the Wald test that the β weights are nonzero.[6]

Note: It is expected that the method setDataSet inherited from *Model* be called *before* the *Logistic* method randomize () (which sets initial weights to zero or random values, depending on the value of the flag *randInitCondFlag* which is set by the method setInitCond()), so that the *Logistic* object may know the number of input nodes and size the vectors and Matrices related to the dataset prior to completing its architecture. This sequence of loading a dataset using the *Model* method setDataSet prior to establishing network architecture will be invoked for all *Network* objects.

- *Public* methods
  - Logistic& operator= (const Logistic& rhs): see section 8.5.2 for discussion
  - Logistic(const Logistic& rhs): see section 8.5.2 for discussion
  - virtual void setDataSet(DataSet& *data*): Implementation of the pure virtual method inherited from *Model* that loads the *DataSet* object *data* into the *Logistic Model*

[6] see Hosmer and Lemeshow, Applied Logistic Regression, 2nd edition, John Wiley & Sons, Inc., New York, 2000, pp 36–42

- virtual void outputHeader(ostream& *outputStream*): Implementation of the pure virtual method inherited from *Model* which outputs a header to an ostream *outputStream* describing the architecture of the *Logistic Model*
- virtual unsigned df(): Implementation of the pure virtual method inherited from *Network* which returns the number of degrees of freedom, equal to the number of inputs + 1 for the y- intercept
- virtual void randomize(): Implementation of the pure virtual method inherited from *Network* which randomizes initial weights
- virtual void reportAccuracy(ostream& *outputStream*): overrides (calls base *Network* reportAccuracy() method first) *Network* method inherited from *Iterative*, adding binary logistic regression specific footer information, including reporting β weights, standard error, and Wald test (p for β weights nonzero)
- virtual bool save(string& $filename$): Implementation of the pure virtual method inherited from *Network* which saves the *Logistic* object β weights and y-intercept to a file named $filename$, returns true if successful
- virtual bool load(string& $filename$): Implementation of the pure virtual method inherited from *Network* which loads the *Logistic* object β weights and y-intercept from a file named $filename$, returns true if successful
- virtual double trainSet(): Implementation of the pure virtual method inherited from *Iterative* which trains one iteration through the training set, and returns set error
- virtual double innerTrainSet(): Implementation of the pure virtual method inherited from *Network* which is called by trainSet() to calculate the gradient and return set error, used for automatic stepsize selection
- virtual void forward(Matrix< double >& M, unsigned n): Implementation of the pure virtual method inherited from *Network* which applies the sigmoidal function for one input row n in a Matrix of inputs M (e.g. either the inherited *Model* Matrix $Train$ or $Test$)
- virtual void removeInputs(vector< unsigned >& v): Implementation of the pure virtual method which takes as its argument a vector of unsigned v representing positions of input nodes, and removes those nodes from the *Logistic* object

- *Private* members
  - vector< double > y: vector containing dataset outputs
  - vector< double > I: vector for single exemplar inputs
  - vector< double > W: vector for β weights (last is y-intercept)

- vector< double > G: vector for β weight gradients
- double *o_err*: output error term

- *Private* Utility functions
  - void copy(const Logistic& rhs): Copy utility function, see section 8.5.2 for discussion.
  - virtual void pack(): Implementation of the pure virtual utility function inherited from *Network* that converts the β weight gradient structure to the *Network* inherited protected member *stackG*, as the β weight structure is already a single vector, this simply copies it
  - virtual void unpack(): Implementation of the pure virtual utility function inherited from *Network* that converts the *Network* inherited protected member *stackG* back to the weight gradient structure, which, since it is a vector, just copies it

8.15 DFA

The **DFA** class is a *Model* object that encodes discriminant function analysis. It is an abstract class, whose derived concrete classes are *LDFA* for linear discriminant function analysis, and *QDFA* for quadratic discriminant function analysis.[7]

- *Public* methods
  - DFA& operator= (const DFA& rhs): see section 8.5.2 for discussion
  - DFA(const DFA& rhs): see section 8.5.2 for discussion
  - virtual void setDataSet(DataSet& *data*): Implementation of the pure virtual method inherited from *Model* that loads the *DataSet* object *data* into the *DFA Model*
  - virtual void outputHeader(ostream& *outputStream*): Implementation of the pure virtual method inherited from *Model* which outputs a header to an ostream *outputStream* describing the architecture of the *DFA Model*
  - virtual double train(): A pure virtual method inherited from *Model* that trains a *Model*, and returns the final error (double) for that model
  - virtual void reportAccuracy(ostream& *outputStream*): A pure virtual method inherited from *Model* that reports the accuracy of the trained model to an ostream *outputStream*

[7] An excellent treatment of discriminant function analysis can be found in Richard O. Duda and Peter E. Hart, Pattern Classification and Scene Analysis, 1973, John Wiley & Sons, New York, ISBN 0-471-22361-1, pp. 24-32

- *Protected* members
  - Matrix< double > $D0$, $D1$: Matrices for inputs of 1-output datasets
  - vector< Matrix< double > > D: a vector of Matrices for inputs of multiple output datasets
  - vector< double > X: holds inputs for 1 exemplar
  - vector< double > $U0$, $U1$: mean vectors for 1-output datasets
  - vector< vector< double > > U: a vector of mean vectors for multiple output datasets
  - double $P0$, $P1$: a priori probabilities for 1-output datasets
  - vector< double > P: a priori probabilities for multiple output datasets
  - double $K0$, $K1$: constants for 1-output datasets
  - vector< double > K: constants for multiple output datasets
  - double $d0$, $d1$: discriminant function results for 1-output datasets
  - vector< double > d: discriminant function results for multiple output datasets
  - vector< unsigned > $trainClasses$: outputs for multiple output training sets
  - vector< unsigned > $testClasses$: outputs for multiple output test sets
- *Protected* Utility functions
  - void copy(const DFA& rhs): Copy utility function, see section 8.5.2 for discussion.

8.16 LDFA

The **LDFA** class is a *DFA* object that encodes linear discriminant function analysis.

- *Public* methods
  - LDFA& operator= (const LDFA& rhs): see section 8.5.2 for discussion
  - LDFA(const LDFA& rhs): see section 8.5.2 for discussion
  - virtual double train(): Implementation of the pure virtual method inherited from *Model* that trains a *LDFA Model*
  - virtual void reportAccuracy(ostream& *outputStream*): Implementation of the pure virtual method inherited from *Model* that reports the accuracy of the trained *LDFA* model to an ostream *outputStream*

- *Private* members
  - Matrix< double > C: the common covariance Matrix
  - Matrix< double > S: inverse of the common covariance Matrix
- *Private* Utility functions
  - void copy(const LDFA& rhs): Copy utility function, see section 8.5.2 for discussion.

8.17 QDFA

The **QDFA** class is a *DFA* object that encodes quadratic discriminant function analysis.

- *Public* methods
  - QDFA& operator= (const QDFA& rhs): see section 8.5.2 for discussion
  - QDFA(const QDFA& rhs): see section 8.5.2 for discussion
  - virtual double train(): Implementation of the pure virtual method inherited from *Model* that trains a *QDFA Model*
  - virtual void reportAccuracy(ostream& *outputStream*): Implementation of the pure virtual method inherited from *Model* that reports the accuracy of the trained *QDFA* model to an ostream *outputStream*
- *Private* members
  - Matrix< double > $C0$, $C1$: covariance Matrices for 1-output datasets
  - vector< Matrix< double > > C: a vector of covariance Matrices for multiple output datasets
  - Matrix< double > $S0$, $S1$: covariance Matrix inverses for 1-output datasets
  - vector< Matrix< double > > S: a vector of covariance Matrices for multiple output datasets
  - double $Det0$, $Det1$: covariance Matrix determinants for 1-output datasets
  - vector< double > Det: covariance Matrix determinants for multiple output datasets
- *Private* Utility functions
  - void copy(const QDFA& rhs): Copy utility function, see section 8.5.2 for discussion.

Chapter 9

Stepwise regression

Statistical analysis of neural computational models is based on the observation that given 2 fully trained neural networks with final errors $E(\mathbf{y}_{reduced})$ and $E(\mathbf{y}_{full})$ that are sufficiently close may be discriminated by a chi-square probability distribution, where

$$G^2 = -2N[E(\mathbf{y}_{full}) - E(\mathbf{y}_{reduced})].$$

with G^2 the chi-square probability p with degrees of freedom the difference in free parameters (connections between nodes) between the 2 neural models, and N the number of exemplars in the dataset (see section 2.3.) E is expressed as the cross-entropy error (see equation 2.3):

$$e(\mathbf{t}^k, \mathbf{a}^{(m)}) = -\mathbf{t}^k . * \mathbf{log}[\mathbf{a}_k^{(m)}] - (\mathbf{1} - \mathbf{t}^k) . * \mathbf{log}[1 - \mathbf{a}_k^{(m)}]. \tag{9.1}$$

In stepwise regression, the 2 networks $\mathbf{y}_{reduced}$ and $\mathbf{y}_{full}$ are chosen to differ by 1 input variable (which may be represented by 1 or more input nodes.)

9.1 Stepwise reverse regression

Stepwise reverse regression proceeds by training the full network, and recording that network's final error. Input variables are removed 1 at a time, the subnetworks so generated are trained to completion and their errors recorded, and the variable with the largest p-value (least significant) is then recorded with its p-value. Using the final error of the network which is deficient the one node with the largest p-value as a starting point, input variables are then removed 2 at a time, with each pair containing the 1 variable with the largest p-value and 1 of the remaining input variables. The pair of variables with the largest p-value (least significant) is then recorded with its p-value, and the process repeats until a chosen threshold p-value is reached.

Figure 9.1 demonstrates stepwise reverse regression.

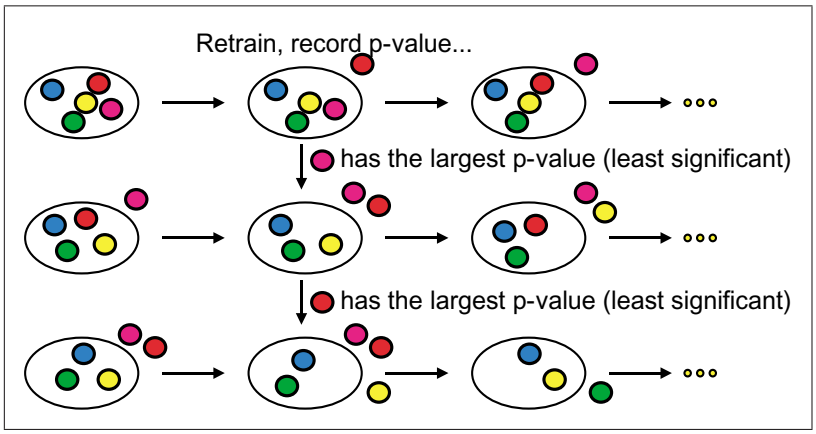

Figure 9.1: Stepwise reverse regression

9.2 Stepwise forward regression

Stepwise forward regression proceeds by training a baseline network with no input nodes, and recording that network's final error. Input variables are added 1 at a time, the subnetworks so generated are trained to completion and their errors recorded, and the variable with the smallest p-value (most significant) is then recorded with its p-value. Using the final error of the network which is the one node with the smallest p-value as a starting point, input variables are then added 2 at a time, with each pair containing the 1 variable with the smallest p-value and 1 of the remaining input variables. The pair of variables with the smallest p-value (most significant) is then recorded with its p-value, and the process repeats until a chosen threshold p-value is reached.

Figure 9.2 demonstrates stepwise forward regression.

9.3 The RegressNet class

RegressNet is a compound class based on the *Network* object that implements stepwise regression analysis on neural computational models.

- *Public* methods
  - RegressNet& operator= (const RegressNet& rhs): see section 8.5.2 for discussion
  - RegressNet(const RegressNet& rhs): see section 8.5.2 for discussion
  - void setInputStructure(const vector< vector< unsigned > >& v): Sets a vector of vector of unsigned v as the structure of input nodes representing the variables for the stepwise regression. For example,

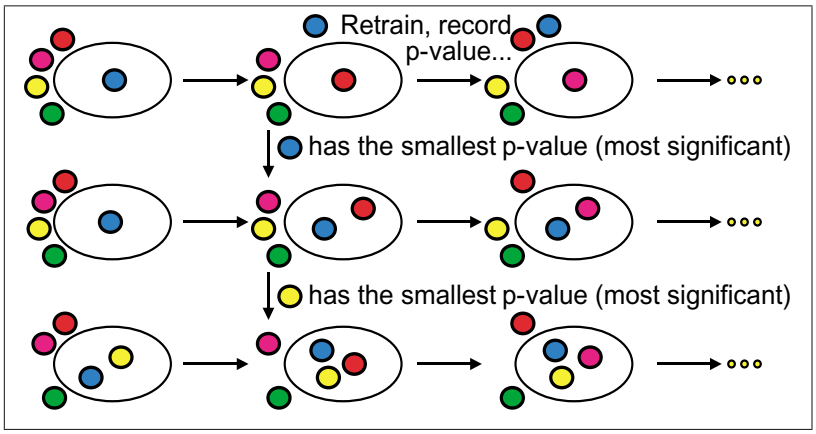

Figure 9.2: Stepwise forward regression

in a 3 variable dataset, if variable 0 was represented by input nodes 0 and 1, variable 1 by input node 2, and variable 2 by input nodes 3, 4 and 5, vector[0] in v would contain (0,1), vector[1] in v would contain (2), and vector[3] in v would contain (3,4,5). All input nodes *must* be represented in v. See section 6.8 for an utility method that parses a string description of input variables, and returns a vector of vector of unsigned.

- void setNetwork(Network* *networkObj*, const double E): sets the *Network* object *networkObj* in the *RegressNet* object, and the final error E of the trained *networkObj*
- void reverse_regress(): performs stepwise reverse regression
- void forward_regress(): performs stepwise forward regression

- *Private* members
  - Network $*netPtr$: pointer to the incoming Network
  - Network $*netCopyPtr$: pointer to a copy of the incoming Network to generate subnetworks
  - bool *historyFlag*: indicates logging to history file
  - double *e_in*: incoming Network's error
  - vector< vector< unsigned > > *variable_defs*: the data structure representing input variables
  - typedef multimap< double, unsigned, less< double > > *ptable*: type definition of a table used to hold stepwise regression results, p-value first, followed by variable number
  - ostringstream *out*: to simplify writing to screen and history file

- strstream *errorOut*: to build an exception message
- string *errorString*: the exception message

- *Private* Utility functions
  - void report(ostringstream& *o*): a utility function which outputs an ostringstream to screen and history file, takes ostringstream *o* as its argument
  - void print_input_structure(const vector< vector< unsigned > > & *v*): a utility function which prints the input variable structure to screen and history file, takes data structure representing input variables *v* as its argument
  - void print_regression_table(const ptable& *table*): a utility function which prints out a stepwise regression table, takes regression table multimap type *ptabletable* as argument
  - bool copy_network(): a utility function which copies the incoming Network object $*netPtr$ into $*netCopyPtr$ so that the latter can be made feature deficient for subnetworks. If you are creating a regressable *Network* object (and chances are that you are,) you *must* add code to this method.
  - string network_name() const: a utility function which returns the name of the type of *Network* object. If you are creating a regressable *Network* object (and chances are that you are,) you *must* add code to this method.
  - inline double Wilks(const double N, const double E_{full}, const double E_{sub}) const: a utility function which calculates the chi-square parameter for 2 networks with N exemplars and which differ by their number of input nodes, the cross-entropy error of the fully trained network with all input nodes being E_{full}, and the fully trained feature deficient subnetwork's error being E_{sub}
  - void copy(const RegressNet& rhs): copy utility function, see section 8.5.2 for discussion.

9.4 Exceptions

The *RegressNet* class throws exceptions of type *RegressNetErr*& *e*, where *e*.what() may contain:

- RegressNet error: maximum node value not *correct*, where *correct* is 1 less than the number of input nodes. This indicates that the vector of vector of unsigned representing the variable structure is incorrectly specified.
- RegressNet error: couldn't copy Network, unknown type. This indicates that the *Network* object has not been properly coded in RegressNet::copy_network().

- RegressNet error: stepwise regression not fully specified. This indicates that *Network* object and its final error have not been set in the *RegressNet* object.

9.5 Example

An example driver may thus appear as:

```
RegressNet regressionObj; // object to regress a Network

// Say you've already trained a Model pointed to by modelPtr, and
//    it was a Network Model, and the last error was lastError
Network* netPtr = dynamic_cast< Network* >( modelPtr );
regressionObj.setNetwork( netPtr, lastError );

vector< vector< unsigned > > variable_defs; // variable definitions
string variable_string; // for entry of variable definitions

cout << "Please enter a string representing the variables: ";
cin >> variable_string;
try {
   variable_defs = util::variable_parse( variable_string );
}
catch ( util::utilErr& e ) {
   cout << e.what() << endl;
}

if ( !variable_defs.empty() ) // parser returned with a good result
{
   try { // try to set the input structure in the stepwise regression
   object
      regressionObj.setInputStructure( variable_defs );
   }
   catch ( RegressNet::RegressNetErr& e ) {
      cout << e.what() << endl;
   }
}

try {
   regressionObj.reverse_regress();
}
catch ( RegressNet::RegressNetErr& e ) {
   cout << e.what() << endl;
}
```

9.6 Adding code

If you are creating a regressable *Network* object (and chances are that you are,) you *must* add code to the methods RegressNet::copy_network() and RegressNet::network_name().Code added to RegressNet::copy_network() will take the form:

```
else if ( typeid( *netPtr ) == typeid( YourNet ) )
   // Create new object with copy ctor & set pointer to the copy
   netCopyPtr = new YourNet( *dynamic_cast< YourNet* >( netPtr ) );
```

and code added to RegressNet::network_name() will take the form:

```
else if ( typeid( *netPtr ) == typeid( YourNet ) )
   network_type = "YourNet";
```

where "YourNet" is to be substituted with the name of your *Network* derived object.

Your own *Network* derived object must contain code in its copy() constructor for **all** of your derived object member variables and structures, and in its removeInputs(...) method for all member structures that need to be altered to remove input nodes. See section 8.9.1 for a discussion of removeInputs(...).

9.7 Compiling RegressNet with MSVC++

Because RegressNet uses Runtime Type Information (RTTI) to determine the type of *Network* object, if you are compiling with MSVC++, you must set the compiler options accordingly (g++ does this automatically for you.) Under the Project menu, choose the Settings option. Choose the C/C++ tab, and choose C++ Language in the Category Box. Then, click the box for "Enable Run-Time Type Information (RTTI)", as shown in Figure 9.3.

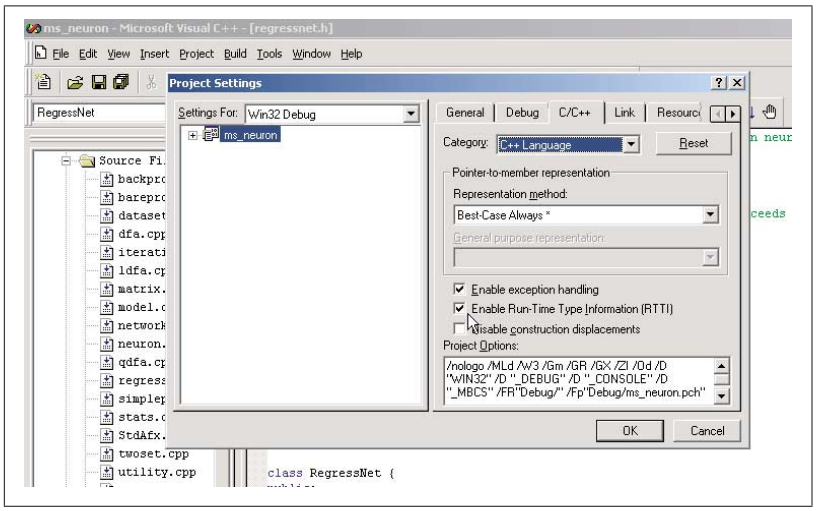

Figure 9.3: Setting RTTI in MSVC++

Chapter 10

The Driver

You of course may design your own driver for the neUROn2++ environment. Naturally, a command line interface driver is under development. The following is the user's guide to that driver.

10.1 Specify dataset

This menu choice includes all of the procedures related to specifying the dataset for the model. A dataset must be specified *before* any other step.

10.1.1 Number of input nodes

This menu choice allows you to set the number of input nodes. Its default is 1 input node, although that should *never* be used.

10.1.2 Number of output nodes

This menu choice allows you to set the number of output nodes. Its default is 1 output node.

10.1.3 Load raw dataset

This menu choice allows you to load a *raw* dataset from a file. The file must be in the same directory as the program. A raw dataset is one whose inputs and outputs have not yet been normalized. Generally, neural computational procedures require a dataset to be normalized, whereas statistical procedures do not. All datasets are of the format:

$$\begin{array}{cccccccc} i_{11} & i_{12} & \dots & i_{1I} & o_{11} & o_{12} & \dots & o_{1O} \\ i_{21} & i_{22} & \dots & i_{2I} & o_{21} & o_{22} & \dots & o_{2O} \\ i_{31} & i_{32} & \dots & i_{3I} & o_{31} & o_{32} & \dots & o_{3O} \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ i_{N1} & i_{N2} & \dots & i_{NI} & o_{N1} & o_{N2} & \dots & o_{NO} \end{array}$$

Where i is an input to the model, and o is an output, and for N examples ("exemplars") in the set, with I total inputs and O total outputs.

These columns may be separated in the file by spaces, commas, or any character which is non-numeric (1 through 9, '.', e, E, - or + are not allowed as column separators.).

10.1.4 Convert a raw dataset into a training set

This menu choice will convert your raw dataset into a training set, by normalizing its inputs and outputs to those specified in the menu choice "specify variate bounds and types." You will be asked if you want to change the number of thresholds used to calculate ROC area.

You will also be asked if you want to save your training set into a file. **This is generally recommended,** as you may want to load the training set at some point in the future to resume training. If you do save your training set, you will be asked for the filename to save its scaling factors. Scaling factors are used in the equation:

$$x_{norm} = lbound + S(x - min) \tag{10.1}$$

where x_{norm} is the normalized result, *lbound* is the lower limit of the normalized x value (1 value for all), S is the scaling factor, and min is the minimum value of each x in the dataset. In this way, should the model be used in the future, you will be able to compute the normalized x_{norm} value from the x value that the user supplies.

10.1.5 Randomize a raw dataset into training and test sets

This menu choice will randomly divide your raw dataset into training and test sets. As it will attempt to keep the fraction of outcomes equal in both sets the same, the number of output nodes *must be* 1, and the output *must be* discrete (e.g. held to 0 or 1). You specify if an output is discrete in the menu choice "specify variate bounds and types."

After the dataset has been randomized, you will be asked if you want to change the number of thresholds used to calculate ROC area, and if you want to save the newly created training and test sets to files.

As with converting a raw dataset into a training set, you will be asked if you want to save your training set and scaling factors into a file (see equation 10.1). You will also be asked if you want to save your test set into a file. Again, **this is generally recommended.**

You will be given the following choices to specify the number of exemplars to be placed in the test set, with the remainder to be placed in the training set ("$N1/N2$" cross-validation):

As a whole number

This choice allows you to specify the number of exemplars in the test set as a whole number.

As a fraction with a numerator and denominator

This choice allows you to specify the number of exemplars in the test set as a fraction of the number of exemplars in the raw dataset. You will be asked for the numerator and the denominator of that fraction.

As a decimal fraction

This choice allows you to specify the number of exemplars in the test set as a fraction of the number of exemplars in the raw dataset. You will be asked for the decimal notation of that fraction (e.g. 0.25 would place $\frac{1}{4}$ of the exemplars from the raw dataset into the test set, and the remainder into the training set.)

10.1.6 Load training set

This menu choice allows you to load a training set from a file. The file must be in the same directory as the program. After the training set has been loaded, you will be asked if you want to change the number of thresholds used to calculate ROC area.

10.1.7 Save training set

This menu choice allows you to save your training set to a file, which will be saved in the same directory as the program. You will also be asked for the filename to save its scaling factors. See equation 10.1 for an explanation of scaling factors.

10.1.8 Save scaling factors

This menu choice allows you to save the scaling factors of the training set, so that should the model be used in the future, you will be able to compute the normalized x_{norm} values from the x values that the user supplies. See equation 10.1 for an explanation of scaling factors.

10.1.9 Load test set

This menu choice allows you to load a test set from a file. The file must be in the same directory as the program. After the test set has been loaded, you will be asked if you want to change the number of thresholds used to calculate ROC area.

10.1.10 Save test set

This menu choice allows you to save your test set to a file, which will be saved in the same directory as the program.

10.1.11 Log dataset operations to history file

Set this option to "yes" if you want the operations you perform in the "Specify dataset" menu to be logged to the history file **neuron.log**. This file will appear in the same directory as the program.

10.1.12 Specify variate bounds and types

In the following menu choices, variates are normalized according to the equation:

$$x_{norm} = lbound + [(\frac{x - min}{max - min}) \times (hbound - lbound)] \tag{10.2}$$

Where *lbound* is the lower boundary of a normalized value, *hbound* is the upper boundary, *min* is the minimum value of a variable *in the training set*, *max* is the maximum value of a variable *in the training set*, x is the variable's value and the normalized value of x is x_{norm}.

Output is discrete

Set this to "yes" if your outputs will always be either 0 or 1.

Threshold for discrete output

Set this to the number over which model outputs will be defined as 1, and under which model outputs will be defined as 0 for discrete output. For example, if the threshold is 0.5, a numeric model output of 0.6 will be transformed to 1, and a numeric model ouput of 0.3 will be transformed to 0. The default is 0.5.

Lower limit of input variates

Choose this menu item to set *lbound* for inputs in equation 10.2. The default is -0.9.

Upper limit of input variates

Choose this menu item to set *hbound* for inputs in equation 10.2. The default is $+0.9$.

Lower limit of output variates

Choose this menu item to set *lbound* for outputs in equation 10.2. The default is $+0.1$ if the output is not discrete (and obviously 0 if it is).

Upper limit of output variates

Choose this menu item to set *hbound* for outputs in equation 10.2. The default is $+0.9$ if the output is not discrete (and obviously 1 if it is).

10.1.13 Number of thresholds to calculate ROC area

Choose this menu item to set the number of thresholds over which ROC area is calculated. The output *must be* discrete, and the number of outputs *must be* 1 for ROC area to be calculated.

10.2 Specify model

Choose this menu item to specify a *neural computational* model. A dataset must be specified *before* this choice, and this menu choice must obviously be specified before "Use model". Discriminant function analysis *does not* require this menu choice, and in fact, it is useless for discriminant function analysis.

10.2.1 Bias nodes

Choose this menu item to specify whether or not bias nodes will be included in your neural computational model. In general, for a real-world model, you should *always* include bias nodes. The default is "yes".

10.2.2 Number of hidden layers / nodes on each layer

Choose this option to specify the number of hidden layers and number of nodes on each layer. The default is 1 layer with 1 node, although neural computational models will generally have > 1 hidden node. Most models will, however, only have 1 hidden layer.

10.2.3 Output error function

Choose this option to specify the output error function. "LMS" is least-mean-squared:

$$E = \frac{1}{2}\sum_{i=1}^{m}(y_i - o_i)^2 \tag{10.3}$$

where E is the error, m the number of outputs, y is the known value and o the model's output.

You may also choose the cross-entropy error function "X-entropy":

$$E = \sum_{i=1}^{m} -y\log(o) - (1-y)\log(1-o) \tag{10.4}$$

For the cross-entropy error function, the output *must be* discrete. Note that for sigmoidal output nodes, where for output $\vec{\mathbf{o}}$,

$$\vec{\mathbf{o}} = \frac{1}{1+e^{-\vec{\mathbf{x}}}}$$

if any element of $\vec{\mathbf{x}}$ is very large or very small, the program will round to 0 or 1, and the log terms in the cross-entropy error function will cause an error. In such cases, an approximation is made (see the document **manifest.pdf** for a detailed explanation of that approximation), and the following error will be generated:

```
WARNING: Numerical out of bounds encountered when calculating error
```

10.2.4 Load network from a file

This menu choice allows you to load a neural computational model from a file. The file must be in the same directory as the program.

10.2.5 Binary logistic regression

This menu choice specifies binary logistic regression, which is mathematically equivalent to a single output node neural network with a bias, the sigmoidal transfer function, no hidden nodes, and the cross-entropy error function. *Always run a binary logistic regression model* **before** *starting to build a neural network.* If your binary logistic model has acceptable goodness-of-fit, you don't need to build a neural network.

Note that at the end of a binary logistic run, β weights will be reported, along with their standard errors and p-values for the Wald test that the β weights are nonzero.[1] Note also that the indexing begins with input number 0, which is often referred to as the y- intercept. *In this case,* β_0 *is* **not** *the* y*-intercept.* It is the β weight of the first variate, indexed to 0. The y-intercept is clearly labeled as "y-intercept" in the report table.

10.2.6 Log last operation to file

Set this option to "yes" (the default) if you want the last neural computational operation performed to be logged to the file **model.txt**. This file will be *over-written* with each run, and will appear in the same directory as the program.

10.2.7 Log to history file

Set this option to "yes" (the default) if you want the neural computational operations that are performed logged to the file **neuron.log**. This file will be *appended* with each operation, and will appear in the same directory as the program.

[1] see Hosmer and Lemeshow, Applied Logistic Regression, 2nd edition, John Wiley & Sons, Inc., New York, 2000, pp 36–42

10.3 Use model

Choose this menu item to use a specified *neural computational* model. A dataset must be specified *before* this choice, and the "Specify Model" menu must also have been entered before "Use model".

10.3.1 Randomize weights

Choose this option to randomize the weights of the neural computational model. Prior to training the model, weights *must be* set, either by randomization or by loading the model from a file in the "Specify model" menu. By default, weights are randomized between -0.5 and $+0.5$.

10.3.2 Set learning rate

Choose this option to set the learning rate η of the neural computational model. The default is $\eta = 1/N$ for on-line backpropagation (where N is the number of exemplars), and $\eta = 1$ for off-line ("batch" or "epoch") training.

You may choose the learning rate to be "automatic" (see the Manifest documentation for a fuller explanation of automatic stepsize selection.) Briefly, automatic stepsize selection is designed to prevent oscillation around a local error minimum. Due to the increase in time required for this feature, it may be most optimal to train to near a local error minimum before setting automatic stepsize selection *on*.

10.3.3 Set stopping conditions

Choose this option to set conditions by which the neural computational model will stop training. These include:

Set maximum number of iterations

The model will stop training after the specified number of iterations. The default is 1000000, *which is arbitrary. Do not consider any neural computational model to be trained by stopping at any arbitrary threshold. You* must *inspect the model yourself to see if it has stopped training.*

Lower error limit

Use this choice to set the lower error limit that, if the error of the model decreases below, will result in cessation of training. For example, you could set this to the limit of your compiler. By default, no lower error limit is set.

Limit of change in error over 1 iteration

Use this choice to set the lower error limit that, if the change in error over 1 iteration is below, will result in cessation of training. Thus, in

$$\Delta E = E_{t+1} - E_t \tag{10.5}$$

Where E_t is the error after a training iteration, and E_{t+1} is the error after the next training iteration, if ΔE is less than the specified value, training stops. By default, this condition is not specified.

Stop if increase over error window width

Allows you to set a number of iterations that after which the error increases, training will stop. Thus, in

$$E_{n+\Delta} > E_n \tag{10.6}$$

Where n is the current iteration, E the error at an iteration, and Δ the error window width, training will stop if the inequality is satisfied. By default, this condition is not specified.

Maximum absolute gradient limit

All neural networks update their weights in the following form:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{g}_t \tag{10.7}$$

where $\mathbf{w_t}$ is the weight structure at time t, η is the learning rate, and $\mathbf{g_t}$ is the gradient. This menu choice allows you to stop training when the maximum absolute value of the elements in the gradient structure $\mathbf{g_t}$ decreases below the chosen limit. The default is 10^{-6}.

READ THIS!

The maximum absolute gradient limit should be the primary means by which you judge that training has ended, *not* by the error minimum.

For canonical backpropagation, using the maximum absolute gradient limit will significantly slow training (not by the number of iterations it takes to train, but by the amount of computational time consumed per iteration.) Thus, one strategy is to turn the maximum gradient choice off until you are near a local error minimum as judged loosely by the decrease in the error, then to turn on this menu choice.

Don't be surprised if the maximum absolute gradient fluctuates (increases and decreases.) Remember, there is more than one node and one layer in even the simplest neural network. Keep an eye on the overall trend of the maximum absolute gradient to insure it is decreasing.

10.3.4 Batch Epoch

Choose this option to set batch/epoch learning.

The *on-line* backpropagation learning algorithm is given by the formula:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{g}_k(\mathbf{w}_t) \tag{10.8}$$

where k is the exemplar index, $\mathbf{w}$ is the weight, t the iteration, $\mathbf{g}$ is the error gradient with respect to the weight, and η is the learning rate. By default, if on-line learning is chosen, $\eta = 1/N$, where N is the number of exemplars.

The *off-line*, also known as "batch" or "epoch" backpropagation learning algorithm is given by the formula:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{g}(\mathbf{w}_t) \tag{10.9}$$

where the *average gradient* $\mathbf{g}$ is defined by the formula:

$$\mathbf{g} = (1/N) \sum_{k=1}^{N} \mathbf{g}_k. \tag{10.10}$$

By default, if off-line learning is chosen, $\eta = 1$.

See the neUROn2++ Design Manifest for a more detailed description of on-line and off- line learning.

If batch/epoch is set *on*, off-line, "batch" or "epoch" learning is used. If batch/epoch is set *off*, on-line learning is used.

10.3.5 Weight Decay

The *error function*, E_k, for the kth exemplar is defined by the formula:

$$E_k(\mathbf{w}) = e(y^k, \mathbf{o}_k^{(m)}) + \frac{1}{2\sigma_w^2} \sum_{i=1}^{m} |\mathbf{w}|^2. \tag{10.11}$$

where y is the known or target outcome, and $\mathbf{o}$ is the model's output. The second term on the right hand side of (10.11) is required to guarantee reliable statistical inference. It basically " stabilizes" the learning process by preventing the weights in the network from becoming too large or too small. σ_w is the standard deviation of the weights **chosen by the user.** Thus, a connection weight is expected to lie between $-1.96\sigma_w$ and $+1.96\sigma_w$ with probability equal to about 95%. Adjustment of this parameter can "change results" of the statistical analysis so σ_w is constrained to be exponents of 10 (e.g. 10, 100, 1000, ... 10^{12}. Once chosen, σ_w should not be changed.

A reasonable choice for σ_w might be $\sigma_w = 100$. This basically constrains the "learning process" to find connection strengths for the ANN such that the connection strength weight ranges between -200 and $+200$. The choice of σ_w should be reported in conjunction with all statistical analysis results. Note that as $\sigma_w \to \infty$, the effect of the summation term becomes negligible.

The learning rate equivalent of equation 10.11 is

$$\begin{aligned} \vec{w}_{t+1} &= \vec{w}_t - \eta \left. \frac{\partial E'}{\partial w} \right|_{w_t} \\ &= \vec{w}_t - \eta \left. \frac{\partial E}{\partial w} \right|_{w_t} - 2\eta\lambda\vec{w}_t \end{aligned}$$

$$= \quad (1 - 2\eta\lambda)\vec{w}_t - \eta \left.\frac{\partial E}{\partial w}\right|_{w_t}$$

where $(1 - 2\eta\lambda)$ is "weight decay". λ may be set by either of 2 methods:

- The complexity of the population objective function >> sample objective function:

$$\lambda = \frac{1}{N 2\sigma_w^2} \tag{10.12}$$

- The complexity of the population objective function $\simeq$ sample objective function:

$$\lambda = \frac{1}{2\sigma_w^2} \tag{10.13}$$

In general, as we don't know the complexity of the population objective function a priori, we choose equation 10.13 to calculate λ.

The following choices allow you to specify weight decay:

Don't change anything

Allows the uninformed user (which, assuming you've read this section, you are *not*,) to exit without changing the way in which weight decay is calculated.

lambda = 1 / (2 * sigma^2)

Specifies that the weight decay term be calculated by

$$\lambda = \frac{1}{2\sigma_w^2}$$

and sets σ_w to be an exponent of 10 from 10^1 to 10^{12}.

lambda = 1 / (2 * N * sigma^2)

Specifies that the weight decay term be calculated by

$$\lambda = \frac{1}{N 2\sigma_w^2}$$

and sets σ_w to be an exponent of 10 from 10^1 to 10^{12}.

No weight decay

Turns off weight decay.

READ THIS!

Using too large of a weight decay term will degrade network modeling performance, but it is critical to use weight decay in order for statistical inference, for example, stepwise regression. Thus, setting a proper weight decay value is of utmost importance. Here is a good strategy to determine the proper weight decay value:

(Note that the default value for the weight decay term is $\frac{1}{2\sigma_w^2}$ *with* $\sigma_w = 100$*. This is to prevent the uninformed user from making incorrect statistical inferences. However, you are no longer the uninformed user. In the first step, you will turn off weight decay.)*

1. Turn weight decay off.
2. Experiment with different neural network architectures until you find one that is as near as possible a strict local minimum, and with as high ROC area as possible.
3. Save the neural network from Step 2.
4. Load the neural network from Step 2, and turn on weight decay. Use the choice for $\lambda = \frac{1}{2\sigma_w^2}$, and choose a low initial value for σ, for example 10^2.
5. Train the neural network to as near a possible a strict local minimum (you'll be starting with the final weights from Step 2.)
6. Compare the ROC area to that from Step 2. If it is similar, go to step 8.
7. If the ROC area has substantially degraded, increase σ, load the neural network from Step 2, and go to Step 5.
8. Save the neural network, and record σ_w, as you will cite it when you publish your results. Perform stepwise regression with this value in the weight decay term, beginning with the final weights from the network you saved in this Step.

10.3.6 Print counter

This menu choice allows you to indicate whether printing will be linear (prints after a set number of iterations) or logarithmic. Logarithmic is recommended.

10.3.7 Train model

This menu choice initiates neural computational model training. You will choose the type of gradient descent. For an explanation of conjugate gradient descent

and the Shanno algorithm, see Golden, pp. 217–218 and 221–222.[2] A brief explanation of these methods may also be found in the design manifest.

The menu choices are ordered by robustness of the algorithm, from less to more. Thus, conjugate gradient descent is less robust than the Shanno algorithm.

READ THIS!

Conjugate gradient descent and the Shanno algorithm generally are much faster at finding a local error minimum than canonical backpropagation *once one is near the local error minimum.* Thus, your training strategy should be to use canonical backpropagation until you are near a local error minimum (as judged by the maximum absolute gradient), and then apply either conjugated gradient descent or the Shanno algorithm. Empirically try each advanced algorithm, and choose the one to use based on your trials.

Canonical backpropagation

This menu choice implements standard, plain vanilla backpropagation.

Conjugate gradient descent

This menu choice implements conjugate gradient descent. See the design manifest and Golden pp. 221–222.

Shanno algorithm

This menu choice implements the Shanno algorithm. See the design manifest and Golden pp. 217–218.

10.3.8 Save the network to a file

This menu choice allows you to save your trained neural computational model with its weights to a file. You may train the model repeatedly prior to saving it, if you so choose. If you have one output, and it is discrete, you will also be asked if you want to save the network's guesses to a file. These guesses will be saved as a 2 column file, with the first column being the known value, and the second column being the network's guess. You will be asked if you want to save guesses for the training set, and if it is loaded, the test set.

10.3.9 Stepwise regression

Stepwise regression allows you to determine which input variables are statistically significant to your trained neural computational model. For a more complete discussion, see the neUROn2++ Design Manifest. If you have not

[2] Richard M. Golden, Mathematical Methods for Neural Network Analysis and Design, 1996, MIT Press, ISBN 0-262-07174-6, pp. 217–218, 221–222

just finished training a neural network, you will be asked for the final error of the network model to regress.

Specify variables

A variable may be represented by more than 1 input node. For example, ethnicity may be represented by 4 nodes: African-American, Asian, Hispanic and Caucasian. Or a variable may be represented by 2 nodes: 1 in which the value is present or absent, and the other by the value of the variable itself. Variable structure may be input from the command line or loaded from file.

Input characters include:

- **;** delimits variables
- **,** delimits nodes
- **-** specifies a range of nodes
- spaces are ignored

For example, "0-3,5; 4,8; 6,7" indicates that variable 0 is composed of input nodes 0,1,2,3 and 5, variable 1 is composed of input nodes 4 and 8, and variable 2 is composed of input nodes 6 and 7.

Stepwise reverse regression

Stepwise reverse regression proceeds by training the full network, and recording that network's final error. Input variables are removed 1 at a time, the subnetworks so generated are trained to completion and their errors recorded, and the variable with the largest p-value (least significant) is then recorded with its p-value. Using the final error of the network which is deficient the one node with the largest p-value as a starting point, input variables are then removed 2 at a time, with each pair containing the 1 variable with the largest p-value and 1 of the remaining input variables. The pair of variables with the largest p-value (least significant) is then recorded with its p-value, and the process repeats until a chosen threshold p-value is reached.

Figure 1 demonstrates stepwise reverse regression.

Stepwise forward regression

Stepwise forward regression proceeds by training a baseline network with no input nodes, and recording that network's final error. Input variables are added 1 at a time, the subnetworks so generated are trained to completion and their errors recorded, and the variable with the smallest p-value (most significant) is then recorded with its p-value. Using the final error of the network which is the one node with the smallest p-value as a starting point, input variables are then added 2 at a time, with each pair containing the 1 variable with the smallest p-value and 1 of the remaining input variables. The pair of variables with the

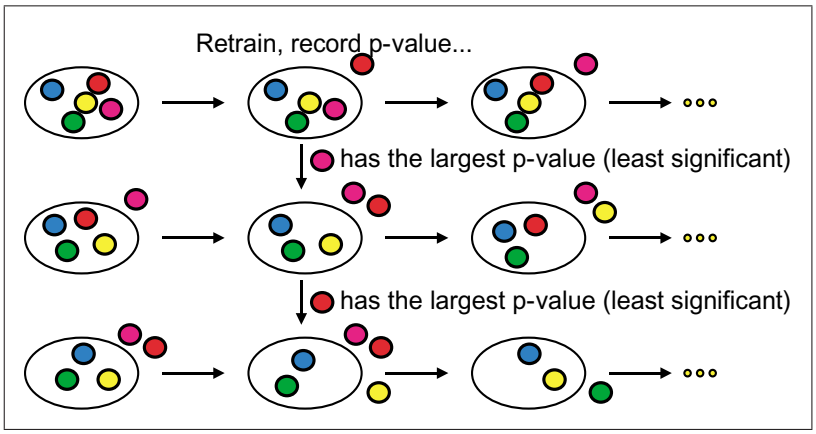

Figure 10.1: Stepwise reverse regression

smallest p-value (most significant) is then recorded with its p-value, and the process repeats until a chosen threshold p-value is reached.

Figure 2 demonstrates stepwise forward regression.

10.4 Discriminant function analysis

Choose this menu item to use discriminant function analysis to model your dataset(s).

10.4.1 Linear discriminant function analysis

Choose this menu item to model your dataset(s) with *LDFA*, linear discriminant function analysis.

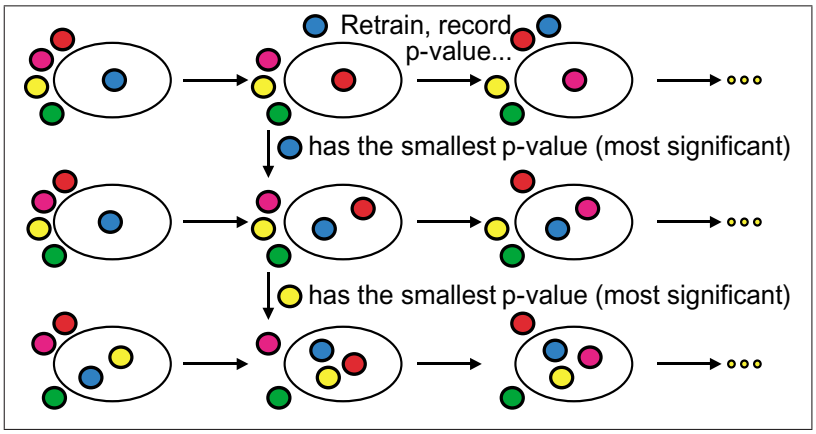

Figure 10.2: Stepwise forward regression

10.4.2 Quadratic discriminant function analysis

Choose this menu item to model your dataset(s) with *QDFA*, quadratic discriminant function analysis.

10.4.3 Log last operation to file

Set this option to "yes" (the default) if you want the last statistical operation performed to be logged to the file **model.txt**. This file will be *overwritten* with each modeling procedure, and will appear in the same directory as the program.

10.4.4 Log to history file

Set this option to "yes" (the default) if you want the statistical operations that are performed logged to the file **neuron.log**. This file will be *appended* with each operation, and will appear in the same directory as the program.

Chapter 11

Utility Scripts

11.1 commenttex.pl

It is very useful to include LaTeXmathematical equations in the documentation of code, e.g.:

```
// Calculate the error term for the output unit
// (Freeman & Skapura p. 102, #6)
// $\delta^o_{pk} = (y_{pk} - o_{pk}){f^o_k}'(net^o_{pk})$ for mean squared error
// Note that the sign is changed ( o - y ) instead of ( y - o ) to conform to the
//    Methodology section
// Methodology equation 2.8 (t=y, a=o)
// $\delta^{(m)}_k = -({\bf t}_k - {\bf a}^{(m)})$ for mean squared error,
// or equation 2.9
//
$\delta^{(m)}_k = -({\bf t}_k - {\bf a}^{(m)}) ./ [{\bf a}^{(m)} .* ({\bf 1} - {\bf a}^{
//    for x-entropy error,
//    multiplied by the ${\bf Df}^{(j)}_k$ term in equation 2.7
// Note that if the error is x-entropy, then dE/do = (y-o)/(o(1-o))
//    and the o(1-o) in the denominator cancels d_sigmoidal
//   ($={f^o_k}'$, $={\bf Df}^{(j)}_k$) = o(1-o)
//    hence splitting this code into 2 lines with an if:
o_err = o - y[ example ];
if ( errorType == 0 ) // error is LMS
   o_err *= d_sigmoidal()( o );
```

The Perl script *commenttex.pl* will create a LaTeXdocument with the LaTeXequations identified by $...$ output in normal LaTeXcode, and everything else "verbatim". Once that LaTeXdocument is then processed, you can easily 'see' the mathematical formulae.

As an example, suppose you wanted to 'see' the above code from bareprop.cpp. You would enter the following lines (assuming you have a working LaTeXinstallation on your computer):

```
commenttex.pl bareprop.cpp > bareprop.tex
latex bareprop
dvipdfm bareprop
```

(You would of course not enter the '$s', as they are the command prompts.) The first line creates the LaTeXfile, the second line runs the LaTeXfile, and the final line creates an Adobe pdf file (and is not strictly necessary, but certainly convenient.) The code would then appear as:

...

```
// Calculate the error term for the output unit
// (Freeman & Skapura p. 102, #6)
```

// $\delta^o_{pk} = (y_{pk} - o_{pk}) f^{o\prime}_k(net^o_{pk})$ for mean squared error

```
// Note that the sign is changed ( o - y ) instead of ( y - o )
to conform to the
//    Methodology section
// Methodology equation 2.8 (t=y, a=o)
```

// $\delta^{(m)}_k = -(\mathbf{t}_k - \mathbf{a}^{(m)})$ for mean squared error,

```
// or equation 2.9
```

// $\delta^{(m)}_k = -(\mathbf{t}_k - \mathbf{a}^{(m)})./[\mathbf{a}^{(m)} .* (\mathbf{1} - \mathbf{a}^{(m)}]$

```
//    for x-entropy error,
```

// multiplied by the $\mathbf{Df}^{(j)}_k$ term in equation 2.7

```
// Note that if the error is x-entropy, then dE/do = (y-o)/(o(1- o))
//    and the o(1-o) in the denominator cancels d_sigmoidal
```

// $(= f^{o\prime}_k, = \mathbf{Df}^{(j)}_k) =$ o(1-o)

```
//    hence splitting this code into 2 lines with an if:
o_err = o - y[ example ];
if ( errorType == 0 ) // error is LMS
   o_err *= d_sigmoidal()( o );
```

11.2 mkdataset.pl

Most often, datasets begin with a Microsoft Excel file. Often, these files contain blank spaces in the columns, and it is your job to recode the file so that no blank spots remain. The Perl script *mkdataset.pl* was written to help you do just that. Columns with blank entries will be replaced by 0, 0 where the entry is blank, and 1, *data value* where the entry is populated. Fully populated columns are simply printed. For example, the following file:

```
Age, Gender, Voting preference
32, 1, 1
58, , 0
63, 0, 0
37, 0,
60, 0, 1
```

Would be converted to:

```
32, 1, 1, 1
58, 0, 0, 0
63, 1, 0, 0
60, 1, 0, 1
```

See below for an explanation of why the line

```
37, 0,
```

was eliminated.

Obviously, you'll still need to first transform the columns from an Excel file into their numerical counterparts (e.g. if M and F are in a column, you would need to change them to 0 and 1.) But this Perl script will take you much of the way towards a dataset that you can then load into neUROn2++.

This Perl script expects a comma delimited file, so first save your Excel spreadsheet in the .csv format (choose File, Save As, and then in the "Save as type" box, choose "CSV (Comma delimited)".)

Then run the Perl script as follows:

```
mkdataset.pl [options] [FILE]
```

Note that the filename is passed as the last argument.

Options include:

-h, -help or ?:	Prints this list
-quiet:	Suppress messages
-noheader:	No first line of labels will be processed
-key [FILE]:	Create a key file from the label line
-noelim:	Do not eliminate rows with the last column blank
-inputs [FILE]:	Create a file with the input node data structure
-o [FILE]:	Output file

11.2.1 Notes:

-noheader

Typically, an Excel file will have a first row that consists of the labels for the columns. If this option is specified, no such first row will be processed.

-key [FILE]

If a first row is present that consists of the labels for the columns, you may specify a file that will record those labels, prettily printed with column numbers.

-noelim

Typically the last column of an Excel spreadsheet represents the outcome variable (note this is not the case with multiple output datasets.) Also typically you'll want to eliminate those rows that have this last column blank. If this option is specified, rows with blank last columns will *not* be eliminated.

-inputs [FILE]

Writes the input node data structure to file suitable for input to neUROn2++ stepwise regression.

11.2.2 Typical usage:

A typical incarnation of *mkdataset.pl* for an Excel comma delimited file *myfile.csv* with the first row containing the labels of the columns would be

```
mkdataset.pl -key key.txt -o output.csv myfile.csv
```

which would write the key file to *key.txt* and the output file to *output.csv*.

11.3 neuron2html.pl

Created by Ashay Kparker for SimpleProp networks, and modified by Gaurav Bansal for BackProp and Binary Logistic Regression models, *neuron2html.pl* is one of the most powerful Perl scripts for the neUROn2++ user.

Once you've fully trained your model, you'll want to make it usable by others. The World Wide Web offers a way to widely distribute your trained neural network in a user-friendly manner. Running *neuron2html.pl* (assuming, of course, you have Perl installed,) you'll be asked to enter:

- The name of the file with the scaling factors that you saved from running part 1 of the driver, "Specify dataset" (now you know why it's so important to save those scaling factors!)
- The name of the file that you saved with the weights of your fully trained model from running part 3 of the driver, "Use model" (now you know why you should save every trained model!)
- A specification file that you create that will tell the Perl program what to use as the names of your variables
- The name of the output html file (be sure to name it with the extension .htm or .html!)

11.3.1 The specification file

You'll need to make a file that will tell the Perl program what type of variables you have, and what to name them. This also includes your outcome variable. Each line in the file refers to a variable. In general, the first letter (or 2) of each line refers to the kind of variable you have, and the remaining entries on that line tell the program how to name that variable. Entries on a line are separated by the delimeter %%.

Numerical variables

Numerical variables are specified in the manner:
N %% Numerical Field Name %% Numerical Field Units

Example:

```
N %% Age %% years
```

Categorical variables

Categorical variables are specified in the manner:
C %% Categorical Field Name %% Choice 1 %% ... %% Choice n

Example:

```
C %% Ethnicity %% African American %% Hispanic %% Caucasian %% Asian
```

Binary variables

Binary variables are specified in the manner:
B %% Boolean Field Name %% Unit True %% Unit False

Example:

```
B %% Digital rectal exam %% Abnormal %% Normal
```

Variables whose values may be unknown

Any of the tags above may be preceded by a K tag to specify if the variable following it is known or unknown in a not fully populated dataset. This addresses the coding of unknown variables in a not fully populated dataset by adding an additional binary variable to indicate if the succeeding variable is present or absent, e.g. if the first variable is present or absent, and the second variable is length in inches:
1 3 indicates length known, and length = 3 inches
1 0 indicates length known, and length = 0 inches
0 0 indicates length unknown

For known/unknown numerical variables:
K %% N %% Numerical Field Name %% Numerical Field Units

Example:

```
K %% N %% Age %% years
```

For known/unknown categorical variables:
K %% C %% Categorical Field Name %% Choice 1 %% ... %% Choice n

Example:

```
K %% C %% Ethnicity %% African American %% Hispanic %% Caucasian %%
Asian
```

For known/unknown binary variables:
K %% B %% Boolean Field Name %% Unit True %% Unit False

Example:

```
K %% B %% Digital rectal exam %% Abnormal %% Normal
```

Reporting the outcome

After the user presses the "predict" button, a window will pop up with the model's predicted outcome. The 'O' and 'R' tags specify what that window will say.

To specify what the discrete output means, use the 'O' tag:
O %% 0 Response %% 1 response

Example:

```
O %% Cancer absent %% Cancer present
```

To specify how to label the odds for the prediction, use the 'R' tag:
R %% Odds ratio label

Example:

```
R %% Odds of cancer on prostate biopsy
```

11.3.2 Example specification file

Suppose that you had trained a model with a dataset that had age, gender and ethnicity as input variables, and voting preference (Democrat or Republican) as an outcome variate. For simplicity, we'll say that the dataset was fully populated, so you didn't need to encode for missing variates. Your 1^{st} column in the dataset was age, encoded in years old, and your 2^{nd} column was gender, with female encoded as '0', and male encoded as '1'. Your 3^{rd}, 4^{th}, 5^{th} and 6^{th} columns were encoded with ethnicity, with a '1' in the 3^{rd} column representing African-American, a '1' in the 4^{th} column representing Hispanic, a '1' in the 5^{th} column representing Caucasian, and a '1' in the 6^{th} column representing Asian. (All of the columns beside the column marked '1' in the ethnicity columns would be marked '0'.) In the 7^{th} column of your dataset was voting preference, with '0' Democrat, and '1' Republican.

Your specification file might then look like:

```
N %% Age %% years
B %% Gender %% Male %% Female
C %% Ethnicity %% African American %% Hispanic %% Caucasian %% Asian
O %% Democrat %% Republican
R %% Odds of voting Republican
```

The generated html file and its pop-up window when the user pressed the "predict" button would then appear as in Figure 11.1.

11.3.3 Things to tweak

Aside from adding some explanatory text to the created html file, there are really only 2 things you'll likely need to tweak in the file:

The title

You can change the title of the main window by altering

```
<title>Neural Net</title>
```

which is very close to the top of the file. Just change 'Neural Net' to anything you like.

You should only need to change 1 thing in the javascript portion of the file–the part starting with

```
<script language = "JavaScript"><!--
```

and ending with

```
//--></script>
```

which, you should notice, is a very large part of your generated html file.

You can change the title of the pop-up output window in the line:

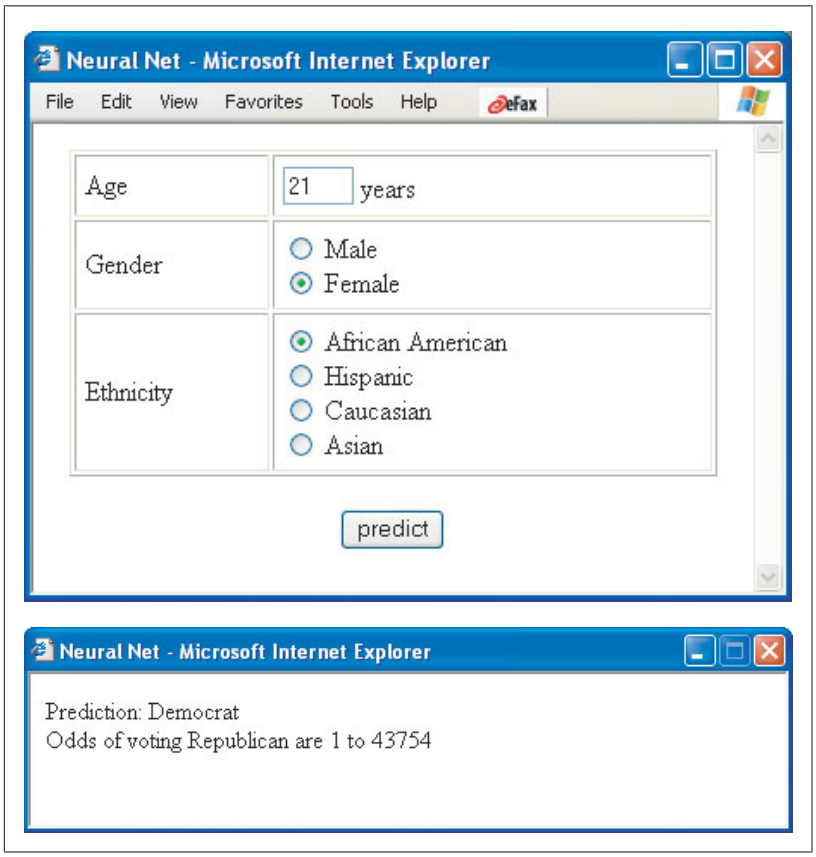

Figure 11.1: Example generated html file and its pop-up window

```
OutputWindow.document.write("<html><head><title>Neural Net</title></
head>");
```

Just change 'Neural Net' in that line to anything you like.

Default values

The Perl program will take a stab at default values by using the minimum values in your original dataset reconstructed by using the scaling factors file. For populated variates, it will give sensible defaults. For unpopulated variates, it will let everything be unknown, which you will likely not want. In either case, you'll want to be able to change the default values. To do that, in the form section of the generated html file (the part which starts with <form>,) for numerical variables just change the number in "value=″...″", and for binary and categorical variables, just move the word "CHECKED" to your default variable.

11.4 neuron2palm.pl

This Perl program is a wizard which creates code for a Palm project that runs your trained neural network. It takes the same files as *neuron2html.pl*, so before you use this program, read the section on converting your trained neural network to html/Javascript.

This wizard assumes that you have the Palm OS Developer Suite installed on your computer. If you don't, detailed instructions on how to install it may be found in the **readme.html** file on your neUROn2++ CD.

3 files will be created: a C++ Palm program (AppMain.cpp), its header file (AppResources.h) and its resource file (AppResources.xrd). These files are then copied into a template Palm OS Developer Suite C++ project that you have created, and then you modify the resource file (the Palm GUI) with the Palm OS Resource Editor and make your project with the Palm OS Developer suite.

11.4.1 Making your Palm OS project

Follow these steps to make your Palm OS project:

Make a blank Palm OS Developer Suite project

First, go to https://www.developerpavilion.com/palmos/, and select the link "Manage My Applications", then "Get a Creator ID", then register a unique 4 character ID for your application.

1. Open the Palm OS Developer Suite. Choose menu **Window → Open Perspective → Palm OS C/C++ Development**
2. Choose menu **File → New → Project**
3. Choose "Managed Make 68K C/C++ Project" under Wizard: Palm OS Development
4. Select "Next", and type in the name of your project in the box
5. Select "Next", and in the next window, enter your unique creator ID in the "Creator ID" box. ***Do not use STRT!***
6. Accept the other defaults, and press "Next". Choose "Simple Application", and then press "Next". Press "Finish".

11.4.2 Run neuron2palm.pl

Enter the same files, with the same descriptions, as neuron2html.pl. Read that section if you haven't already.

11.4.3 Copy created files to the project directories

Open the "src" tree under your project in the left hand pane of the Palm OS Developer Suite window. Right click on "AppMain.c" and choose "Delete".

Copy the *neuron2palm.pl* created files **AppMain.cpp** and **AppResources.h** to your project's "src" directory, and **AppResources.xrd** to your project's "rsc" directory, overwriting the files there. Copy the files in *utilities/perl/Craig's Palm icons* on the neUROn2++ CD into your project's "rsc/bitmaps" directory.

11.4.4 Tweak your GUI

Open the Palm OS Resource Editor program. This program is separate from the Palm OS Developer Suite. Select menu choice **File** → **Open**..., and find your resource file **AppResources.xrd** in your project's "rsc" directory. Move around the elements on **Form 1001 (Main)** to your tastes, and update the reference on **Form 1007 (About)**. Select menu choice **File** → **Save**.

11.4.5 Make your project

Return to the Palm OS Developer Suite. Right click on your project name in the left hand pane of the window, and choose "Refresh". Right click on your project name again, and choose "Rebuild Project".

11.4.6 Test your project

Open the Palm OS Emulator program. This program is separate from the Palm OS Developer Suite. Right click on the emulator, choose "New", and enter the name and location of your ROM file. Remember to soft reset the emulator (also by right clicking on it) between each run! Go back to the Palm OS Developer Suite, and select menu choice **Run** → **Run**. Click on the "New" button, and the name of your Palm program should automatically appear under *Palm OS Application* in the left hand window, and a checked box next to the name of your application in the right hand pane under the *Project* tab. Select the *Target* tag, and choose **Palm OS Emulator** for *Device*. Click on the "Apply" button, then the "Run" button.

Note

Note that if you change your numeric defaults (and you probably should, as they are encoded as the minima for those variates,) you need to change the C++ code **in 2 places**. The first place is near the top of the C++ program, where the *_input* array is initialized–just change the values of that array. It's immediately below the lines:

```
// Initialize INPUT vector !!! Change these values !!!
// Remember to make sure that for binary variables, the initial
//    value is set in the GUI, and for categorical variables,
```

```
//    make sure the initial value is the first category.
```

The second place where you need to change the value of your numeric variates is in the *MainFormHandleEvent* function, and these lines are preceded by the comment:

```
// !!! Change to match your ... field name and default !!!
```

where ... is replaced by your numeric field name. Immmediately below these lines, you'll find a *WriteToField* function call, just change the last value inside of the quotation marks to match what you entered in the *_input* array.

11.5 neuron2iphone.rb

This Ruby/Tk program is a wizard which creates code for a iPhone/iPod Touch project that runs your trained neural network. You need to have Ruby installed with the Tk bindings on your computer. Fortunately, modern MacOS installations such as Leopard come with Ruby and the Tk bindings, and since you *have to* use Xcode to make iPhone/iPod Touch applications, you'll be able to run *neuron2iphone.rb* without difficulty on the same platform that you'll be developing your iPhone/iPod Touch application. However, if you are determined to run *neuron2iphone.rb* on a different operating system than the one on which you'll be developing iPhone/iPod Touch apps, just be sure your Ruby installation includes the Tk bindings.

neuron2iphone.rb takes the same files as *neuron2html.pl*, so before you use the program, read the section on converting your trained neural network to html/Javascript. *Note: the wizard is currently only implemented for SimpleProp neural networks and Logistic Regression models.*

Three files will be created by *neuron2iphone.rb*, the ViewController .h file, the .m file, and a detailed instruction file explaining step-by-step how to create an iPhone/iPod Touch application with Xcode. As the only way to deploy your application is using the iTunes App Store and getting your app into the store is not trivial, the iPhone Developer instructions for submitting an application are included in the *scripts* directory of the neUROn distro. Running *neuron2iphone.rb* is fairly easy and self explanatory as it has a graphical user interface.

Chapter 12

Old File Structure

What follows is a description of the structure of neUROn++ files prior to the design of neUROn2++.

12.1 Files specifying input data

The following files specify data to be modeled:

12.1.1 The input file (*filename*)

In neUROn++, the input file is specified by each exemplar as a row delimited by spaces and truncated by a carriage return, containing a list of the input variables followed by the output variable, e.g.:

$$\begin{array}{cccc} i_{11} & i_{12} & \dots & o_1 \\ i_{21} & i_{22} & \dots & o_2 \\ i_{31} & i_{32} & \dots & o_3 \\ \dots & \dots & \dots & \dots \\ i_{n1} & i_{n2} & \dots & o_n \end{array}$$

For example, the exclusive-or problem is encoded:

$$\begin{array}{ccc} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{array}$$

It is the job of the program to count the number of rows as the number of exemplars. The program is thus required to identify blank lines and discard them (as some word processing programs, for example, append blank lines to the end of a file without notifying the user.)

12.1.2 The raw training file (*filename*_train.txt)

The raw training file is generated by the "big lotto" module, which randomizes the raw data set into training and test sets, with a specified number of exemplars in each, and insures that, while random, the frequency of outcomes in the test and training sets are similar.

Format of the raw training file is (space delimited) number of exemplars, number of input nodes, number of output nodes, threshold value[1] followed by lines containing each exemplar as specified in the preceeding section, e.g.:

N_x *exemplars*	N_o *outputs*		N_i *inputs*	*threshold* T
	i_{11}	i_{12}	...	o_1
	i_{21}	i_{22}	...	o_2
	i_{31}	i_{32}	...	o_3
	...	...	...	...
	i_{n1}	i_{n2}	...	o_n

An example of an exclusive-or raw training file is:

```
75  1  2  0.5
1  0  1
0  1  1
0  1  1
1  0  1
0  1  1
1  0  1
0  1  1
1  0  1
0  1  1
1  0  1
```

...

12.1.3 The raw test file (*filename*_test.txt)

The raw test file is also generated by the "big lotto" module, and conforms to the specification of the raw training file, while containing the test data rather than the training data.

12.1.4 training_set

This file contains the number of examples in the training set, followed by the normalized training set. If *lbound* is the lower boundary of a normalized value, *hbound* is the upper boundary, *min* is the minimum value of a variable, *max*

[1] threshold T is for discrete output values: if network guess $>= T$, output is 1, if $< T$, output is 0

is the maximum value of a variable, and x is the variable's value, then the normalized value of x, x_{norm} is:

$$x_{norm} = lbound + [(\frac{x - min}{max - min}) \times (hbound - lbound)] \qquad (12.1)$$

Note that $x_{norm} = lbound$ when $x = min$ and $x_{norm} = hbound$ when $x = max$.

Note also that outputs are normalized (to the output bounds specified in the header file) if "discrete output" is specified *off* in the header file, but set to 0 or 1 if "discrete output" is specified *on* in the header file.

An example of an exclusive-or raw training_set file is:

```
75
 9.000000e-01 -9.000000e-01 1.000000e+00
-9.000000e-01  9.000000e-01 1.000000e+00
-9.000000e-01  9.000000e-01 1.000000e+00
 9.000000e-01 -9.000000e-01 1.000000e+00
-9.000000e-01  9.000000e-01 1.000000e+00
 9.000000e-01 -9.000000e-01 1.000000e+00
-9.000000e-01  9.000000e-01 1.000000e+00
 9.000000e-01 -9.000000e-01 1.000000e+00
-9.000000e-01  9.000000e-01 1.000000e+00
 9.000000e-01 -9.000000e-01 1.000000e+00
-9.000000e-01  9.000000e-01 1.000000e+00
```

...

12.1.5 test_set

Like training_set, test_set is derived from *filename*_test.txt, and conforms to the specification of training_set, while containing the test data rather than the training data.

12.2 Files specifying network architecture

The following files pertain to network architecture:

12.2.1 minimax

The file minimax contains the minimum and maximum values for each variable, and is generated by the prepare module. Its format is 2 lines, with the first line the mininum values, and the second the maximum values. The final value on each line is the output variable. Each value is separated by blank space, with the lines truncated by carriage returns. File format is thus:

$$\begin{array}{ccccccc} i1_{minimum} & i2_{minimum} & i3_{minimum} & \ldots & in_{minimum} & o_{minimum} \\ i1_{maximum} & i2_{maximum} & i3_{maximum} & \ldots & in_{maximum} & o_{maximum} \end{array}$$

For example, the minimax file for the exclusive-or problem is:

```
0.000000e+00 0.000000e+00 0.000000e+00
1.000000e+00 1.000000e+00 1.000000e+00
```

$$\begin{array}{cccccccc} i1_{minimum} & i2_{minimum} & \dots & in_{minimum} & o1_{minimum} & o2_{minimum} & \dots & om_{minimum} \\ i1_{maximum} & i2_{maximum} & \dots & in_{maximum} & o1_{maximum} & o2_{maximum} & \dots & om_{maximum} \end{array}$$

Where the number of input nodes and the number of output nodes is specified in the header file.

12.2.2 hidden_weights

The file hidden_weights contains the hidden weight matrix, in the form of multiple lines separated by carriage returns, with each line representing one hidden node, and each line containing the input weights for that node. All weights are bounded by bounds specified in the header file. File format is thus:

$$\begin{array}{ccccc} w_{h_1 i_1} & w_{h_1 i_2} & w_{h_1 i_3} & \dots & w_{h_1 i_n} \\ w_{h_2 i_1} & w_{h_2 i_2} & w_{h_2 i_3} & \dots & w_{h_2 i_n} \\ w_{h_3 i_1} & w_{h_3 i_2} & w_{h_3 i_3} & \dots & w_{h_3 i_n} \\ \dots & \dots & \dots & \dots & \dots \\ w_{h_m i_1} & w_{h_m i_2} & w_{h_m i_3} & \dots & w_{h_m i_n} \end{array}$$

This is a file of critical importance, as after the program is run, it represents a part of the modeling solution.

An example is from a 2 input network with 3 hidden nodes:

```
1.022590e-01 -4.725589e-01
4.030996e-01 -4.004863e-01
-1.634707e-01 2.717012e-01
```

12.2.3 hidden_weights.full

This file is generated in neUROn++ for the "black input" feature, where input nodes are held to 0, and the network is retrained. In this implementation, the hidden_weights file is copied to hidden_weights.full, and then subsequent hidden_weights files represent full training of feature deficient (e.g. specific input variables held to 0) subnetworks. It has the same format as the hidden_weights file.

This type of file structure appeared early on in neUROn++, when single input variables were commonly held to 0, and the network retrained. Subsequently, neUROn++ was reprogrammed so that input variables were held to 0 in a combinatorial fashion or in a stepwise regression algorithm.

12.2.4 hidden_bias

This file contains one line, with the weight values of the bias nodes on the hidden layer separated by spaces. It thus takes the format:

$$w_{h_1 b} \quad w_{h_2 b} \quad w_{h_3 b} \quad \ldots \quad w_{h_m b}$$

12.2.5 hidden_bias.full

Like hidden_weights.full, hidden_bias.full was designed for the "black input feature" in the initial stages of neUROn++. It has the same utility as the hidden_bias.full file.

12.2.6 output_weights

Like hidden_weights, output_weights contains the output weight matrix. It is a single line (for a single output node) containing the output weights separated by spaces, and takes the form:

$$w_{o_1} \quad w_{o_2} \quad w_{o_3} \quad \ldots \quad w_{o_n}$$

12.2.7 output_weights.full

Like hidden_weights.full, output_weights.full was designed for the "black input feature" in the initial stages of neUROn++. It has the same utility for the output layer as the hidden_weights.full file does for the hidden layer.

12.2.8 output_bias

The file output_bias contains one value (for a single output node) as the weight of the bias on the output node.

12.2.9 output_bias.full

Like hidden_bias.full, output_bias.full was designed for the "black input feature" in the initial stages of neUROn++. It has the same utility for the output layer as the hidden_bias.full file does for the hidden layer.

12.3 Training multiple subnetworks

Using the "black input" feature, subnetworks are generated and trained with individual or sets of input nodes held to 0. These trained subnetworks are then statistically compared to the full network to determine the significance of the input node(s) that is(are) "blacked out". The method to specify generation of such subnetworks is either automatic, as in stepwise regression analysis, or specified by the user, as in combinatorial analysis. In both, it is necessary to define a specific feature (or variable) to be "blacked" out as a set of the input

nodes which characterize it. For example, input node 17 may represent patient age, and input node 16 represents if patient age is available in the database (not available is 0, available is 1). In this example, both nodes 16 *and* 17 must be simultaneously held to 0 to "black out" the feature. The file class_define specifies how input nodes are grouped into features to be "blacked out".

The file format of the class_define file is as follows:

$feature_1$, $feature_2$,..., $feature_N$

Where "feature" is a number, multiple numbers with a plus (+) in between indicating "and", or two numbers with a dash (-) in between, indicating "through".

For example:

```
1-4,5,6-10,11,12,13+15,14
```

indicates that there are 15 input nodes divided into 7 features. The first feature is input nodes 1 through 4, the second feature is node 5, the 3rd feature is input nodes 6 through 10, the 4th is node 11, the 5th is node 12, the 6th is nodes 13 and 15, and the 7th is node 14.

12.4 Reports

The following files contain reports generated during training and testing:

12.4.1 error_file

The file error_file is the master record file. It should contain *all* of the information present in the header file organized in a readable fashion, and all calculated errors at specified intervals. It is used as documentation for an experiment that consists of fully training a neural network.

Example:

```
Mon Jul 10 14:12:36  2000
============================
Neural network error file:

  Target error not available. AUTO_ERROR ON. ERROR_WINDOW_WIDTH 1000
  STOP_AT_MIN ON
  STOP_INCREASE ON
  MOMENTUM OFF
  WEIGHT DECAY OFF
  2 INPUTS; 1 OUTPUT
Number of hidden nodes on the 1 hidden layers are:
3
  BIAS ON
  Transfer function output low bound 0.100000, high bound 0.900000
```

```
    Input low bound -0.900000, high bound 0.900000
  DISCRETE_OUTPUT ON
  XENTROPY ON
  BATCH/EPOCH LEARNING DISABLED
  BLACK_INPUT OFF

  (degree of freedom is 13)

75 examples in training set

N          Set Error     XE Error      CErr Train  CErr Test

       1  1.282783e-01  7.064047e-01  50.666667   48.000000
       2  1.274456e-01  7.030068e-01  50.666667   48.000000
       3  1.269191e-01  7.008692e-01  50.666667   48.000000
       4  1.265779e-01  6.994885e-01  50.666667   48.000000
       5  1.263515e-01  6.985742e-01  50.666667   48.000000
       6  1.261979e-01  6.979550e-01  50.666667   48.000000
       7  1.260917e-01  6.975270e-01  50.666667   48.000000
       8  1.260168e-01  6.972256e-01  50.666667   48.000000
       9  1.259631e-01  6.970095e-01  50.666667   48.000000
      10  1.259239e-01  6.968520e-01  50.666667   48.000000
      20  1.257945e-01  6.963322e-01  50.666667   48.000000
      30  1.257459e-01  6.961384e-01  50.666667   48.000000
      40  1.257018e-01  6.959635e-01  50.666667   48.000000
      50  1.256583e-01  6.957919e-01  50.666667   48.000000
      60  1.256143e-01  6.956188e-01  50.666667   48.000000
      70  1.255687e-01  6.954400e-01  50.666667   48.000000
      80  1.255205e-01  6.952518e-01  50.666667   48.000000
      90  1.254686e-01  6.950500e-01  50.666667   48.000000
     100  1.254120e-01  6.948302e-01  50.666667   48.000000
     200  1.240699e-01  6.896549e-01  24.000000   28.000000
     300  1.130212e-01  6.450682e-01  24.000000   28.000000
     400  9.205705e-02  5.416243e-01  24.000000   28.000000
     500  4.299956e-02  3.227091e-01  0.000000    0.000000
     600  4.685616e-03  9.736120e-02  0.000000    0.000000
     700  1.376876e-03  5.197709e-02  0.000000    0.000000
     800  6.277901e-04  3.487233e-02  0.000000    0.000000
     900  3.535863e-04  2.607810e-02  0.000000    0.000000
    1000  2.251775e-04  2.076367e-02  0.000000    0.000000
    2000  2.373450e-05  6.694852e-03  0.000000    0.000000
    3000  8.330674e-06  3.959491e-03  0.000000    0.000000
    4000  4.188223e-06  2.805047e-03  0.000000    0.000000

!! Error changing decreased under MIN_DELTA_ERR=1.000000e-03
```

```
To reach an overall Xentropy error of 2.638119e-03 required 4218
iterations.

-------------------------------------------------------------------------
-------
For the TEST SET after iteration 4217:
-------------------------------------
true positive rate = 12    true negative rate = 13
false positive rate = 0    false negative rate = 0

sensitivity = 1.000000                    specificity = 1.000000
positive predictive value = 1.000000      negative predictive value =
1.000000
-------------------------------------------------------------------------
---------

Training time (HHHH:MM:SS) 0000:00:04============================
Mon Jul 10 14:12:42  2000
```

12.4.2 black_report

This file has an identical format to error_file, and is generated when an input node or set of input nodes is held to 0, as in the "black input" feature.

12.4.3 xentropy.full

This file contains the cross-entropy error calculated for a fully trained network. Like many features, it was designed early in neUROn++'s life cycle, prior to combinatorial or stepwise regression analysis.

12.4.4 out-file

The file outfile contains one line with the number of examples in the test set, followed by the number of output nodes, followed by the threshold value. Subsequent lines are pairs of the known value followed by the network's guess. For a one-output network, this takes the form:

$$
\begin{array}{cc}
N_x \ exemplars & N_o \ outputs \quad threshold \ T \\
known_1 & guess_1 \\
known_2 & guess_2 \\
known_3 & guess_3 \\
\ldots & \ldots \\
known_n & guess_n
\end{array}
$$

For example,

```
25 1 0.500000
1.000000 0.996711
1.000000 0.998407
1.000000 0.998407
1.000000 0.996711
1.000000 0.996711
1.000000 0.998407
1.000000 0.996711

...
```

Chapter 13

The Old Header File

Prior to full C++ implementation in neUROn2++, as much of the network description as possible was encoded in pre-processor definitions in a header file, so that the compiled executable is as tailored as possible to the data model. The following section details these pre-processor definitions.

13.1 Global defines

13.1.1 Limits of computability

The TOLERANCE preprocessor definition specifies the precision of the compiler, e.g. 1e-8, 1e-12, etc.

13.2 Neurocomputer architecture

The following pre-processor definitions encode neurocomputer architecture.

13.2.1 BIAS

Bias nodes are nodes held to 1 on a layer. In neUROn++, BIAS ON indicates bias nodes on all layers, whereas BIAS OFF indicates bias nodes on no layers.

```
#define BIAS {1, 3, 4}
```

would specify bias nodes on layers 1, 3 and 4.

13.2.2 Layers

In neUROn++, up to 3 hidden layers could be specified. The following pre-processor definitions describe layer architecture:

- NUMBER_NODES_MAX: specifies the maximum number of nodes on each layer, and is used for initializing arrays

- NUMBER_HDLAYER: specifies the number of hidden layers
- NUMBER_HIDDEN: specifies the number of hidden nodes in the 1st layer
- NUMBER_HIDDEN1: specifies the number of hidden nodes in the 2nd layer
- NUMBER_HIDDEN2: specifies the number of hidden nodes in the 3rd layer
- NUMBER_OUTPUT: specifies the number of output nodes
- NUMBER_INPUT: specifies the number of input nodes

13.2.3 Activation function

In neUROn++, the activation function is defined in the header file, as is its derivative for the backpropagation algorithm. The activation function and its derivative are defined in macros:

- TRANSFER_FUNCTION(x)
- DIFF_TRANSFER(o)

Because backpropagation uses the sigmoidal activation function:

$$t_k = \frac{1}{1 + e^{-net_k}} \tag{13.1}$$

and its derivative:

$$f\prime(net_k) = o_k(1 - o_k) \tag{13.2}$$

The following macro definitions are:

```
#define TRANSFER_FUNCTION(x) 1.0 / (1.0 + exp( -(x) ))
#define DIFF_TRANSFER(o) (o) * ( 1 - (o) )
```

13.3 Preparation

The following pre-processor definitions describe data and model processing prior to modeling.

13.3.1 Data set

The pre-processor definition TRAINING_MAX specifies the maximum number of exemplars in the training data set.

13.3.2 Scaling

Input values

As described in equation 12.1, input values are scaled according to the following formula:

$$x_{norm} = lbound + [(\frac{x - min}{max - min}) \times (hbound - lbound)]$$

Where *lbound* is the lower boundary of the input value to be scaled, *hbound* is the upper boundary, *min* is the minimum value of the variable, *max* is the maximum value of the variable, and x is the variable's value. Thus, *lbound* and *hbound* for input variables are specified in the header file as:

- INPUT_LBOUND = *lbound*
- INPUT_HBOUND = *hbound*

Output values

In neUROn++, *nondiscrete* (e.g. the pre-processor definition DISCRETE_OUTPUT is set OFF) output values were scaled using:

- OUTPUT_LBOUND
- OUTPUT_HBOUND

where OUTPUT_LBOUND and OUTPUT_HBOUND represented the low and high bounds respectively of the output value.

13.3.3 Randomizing initial weights

Initial weights are randomized to a value bounded by $-bound$ to $+bound$. This *bound* is specified by the pre-processor definition RANDOM_THRESHOLD, which is generally set to 0.5. Thus, weights are generally initialized to values between -0.5 and $+0.5$.

13.4 Training control

The following pre-processor definitions describe global control of training, including report generation and how training ends.

13.4.1 Reporting

The following pre-processor definitions describe report generation:

- PRINT_COUNT
- WITH_CLASS

- OUTCOMES_FILE
- TEST_SET_STATS

PRINT_COUNT specifies at which iteration error outcomes are reported. It may take the form of a number, e.g. a PRINT_COUNT of 1000 would specify error reportage every 1000 iterations, or an equation, e.g.:

```
#define PRINT_COUNT (int) ( pow(10, (int) log10(set_count) ) )
```

Specifies a highly useful exponential count.

WITH_CLASS specifies if classification error in the training and test sets are reported at PRINT_COUNT. These are useful to generate graphs of neurocomputer training and identify such behavior as overlearning. Classification error is described as:

$$CE = \frac{N_{correct}}{N_{total}} * 100\% \tag{13.3}$$

Where CE is the classification error (in either training or test sets), $N_{correct}$ is the number of correct guesses, and N_{total} is the total number of guesses.

If OUTCOMES_FILE is specified ON, "out-file" is generated as specified in section 12.4.4.

If TEST_SET_STATS is specified ON, specificity, positive predictive value, and negative predictive value are reported in "error_file" as specified in section 12.4.1.

13.4.2 Ending training

The following pre-processor definitions control how training ends.

- ITERATIONS_MAX specifies the maximum number of iterations at which training ends. Even if the error is not at a minimum, training will end at the number of iterations specified by this pre-processor definition.
- AUTO_ERROR controls the next 3 definitions. If AUTO_ERROR is OFF, *all* of these definitions will be ignored. If AUTO_ERROR is ON, *any* any of these definitions is sufficient to end training.
  - ERROR_WINDOW_WIDTH defines the number of iterations at which training will end if either the STOP_INCREASE or the STOP_AT_MIN conditions are satisfied.
  - STOP_INCREASE specifies that training will end if the error at any iteration + ERROR_WINDOW_WIDTH is greater than the error at that iteration, e.g.:

    $$E_{n+\Delta} > E_n$$

    where n is the iteration, E is the error, and Δ the ERROR_WINDOW_WIDTH.

– If STOP_AT_MIN is ON, training will end if the change in error over the number of iterations specified by ERROR_WINDOW_WIDTH is less than MIN_DELTA_ERR, and
– MIN_DELTA_ERR specifies the minimum change in error at which training will end over any ERROR_WINDOW_WIDTH. Thus, training will end if

$$E_{\Delta min} > E_{n+\Delta} - E_n$$

where n is the iteration, E is the error, Δ is ERROR_WINDOW_WIDTH, and $E_{\Delta min}$ the MIN_DELTA_ERR.

13.5 Training

The following pre-processor definitions specify how weights are updated within a neural network, generally referred to as "training".

13.5.1 Global definitions

The following pre-processor definitions apply to any neural network training methodology.

Batch/Epoch

"Online" learning refers to the scheme whereby weights are updated with each training exemplar. Thus, if i refers to an individual training exemplar, then network weights w_i are updated by:

$$w_i \leftarrow w_i + \Delta w_i$$

for each training exemplar, where Δw_i is the weight update value.

"Offline" learning refers to the scheme whereby the weight updates are calculated for the entire training set, summed, and applied to the network's weights *after* the entire training set is processed:

$$w_i \leftarrow w_i + \sum \Delta w_i$$

"Offline" learning is often referred to as either "Batch" or " Epoch" learning. If the pre-processor variable BATCH_EPOCH is ON, offline learning is specified, if BATCH_EPOCH is OFF, online learning is specified.

Discrete output

If the pre-processor definition DISCRETE_OUTPUT is ON, then output(s) are defined to be 0 if the output value is less than a threshold value, and 1 if the output value is greater than or equal to the threshold value:

$$O \leftarrow O \geq T|_0^1$$

where O is the output value and T is the threshold. In neUROn++, the threshold T is specified in the raw training file as described in section 12.1.2.

13.5.2 Backpropagation

Backpropagation is specified by the pre-processor definition CLASSIC_BP. Treatment of the backpropagation algorithm may be found in a variety of texts such as Timothy Masters "Practical Neural Network Recipes in C++" (Academic Press 525 B St, Suite 1900, San Diego CA 92101-4495 www.apnet.com, www.academicpress.com ISBN 0-12-479040-2) 1993.

Briefly, canonical backpropagation proceeds as follows. The input vector is applied to the network, and outputs of the first hidden layer are calculated by the activation function t_k which is set to be the the sigmoidal:

$$t_k = \frac{1}{1 + e^{-net_k}} \tag{13.4}$$

The outputs node values of the first hidden layer are applied to the inputs nodes of the second hidden layer, etc. until the output layer's output node values are calculated.

The output node's weights are then updated by:

$$w_{jk} \leftarrow w_{jk} + \eta \delta_k o_j \tag{13.5}$$

for learning rate η, j output units and k hidden units, where

$$\delta_k = (t_k - o_k) f\prime(net_k) \tag{13.6}$$

o_k is the known outcome value in the training set, and the the derivative $f\prime(net_k)$ is:

$$f\prime(net_k) = o_k(1 - o_k) \tag{13.7}$$

Weights are then sequentially updated in the hidden layers in the direction of *from* the ouput layer *to* the input layer, hence the term "backpropagation".

Learning rate

The learning rate η is specified by the preprocessor definition ETA.

Momentum

Momentum adds a fraction of the previous iteration's weight update to the current weight update:

$$\Delta w_t \leftarrow \Delta w_t + \alpha \Delta w_{t-1} \tag{13.8}$$

where Δw_t refers to the current weight update, and Δw_{t-1} to the previous weight update. The "momentum" term is α. The weight update rule in backpropagation then becomes:

$$\Delta w_t \leftarrow \eta \delta x + \alpha \Delta w_{t-1} \tag{13.9}$$

The following pre-processor definitions specify momentum:

- MOMENTUM, if ON specifies that a momentum term will be applied.
- ALPHA is the value of the momentum term.

Weight decay

Weight decay subtracts a fraction of the previous iteration's weight (not weight update as in momentum) from the current weight update:

$$\Delta w_t \leftarrow \Delta w_t - d w_{t-1} \tag{13.10}$$

where Δw_t refers to the current weight update, and w_{t-1} to the previous weight. The "decay" term is d. The weight update rule in backpropagation then becomes:

$$\Delta w_t \leftarrow \eta \delta x - d w_{t-1} \tag{13.11}$$

The following pre-processor definitions specify weight decay:

- WEIGHT_DECAY, if ON specifies that a weight decay term will be applied.
- DECAY is the value of the weight decay term.

Cross-entropy

Cross-entropy is an error function applied *only at the output layer.* It is defined by:

$$E = -t \log(o) + (1 - t) \log(1 - o) \tag{13.12}$$

where t is an output node's value derived by the network, and o is the known value of that output in the data set.

If the pre-processor definition XENTROPY is ON, then the cross-entropy error function is calculated at the output node. *It should be clear that it is a requirement for XENTROPY that DISCRETE_OUTPUT is ON.*

13.5.3 Shanno's algorithm

This serves as an excellent example of the importance of documentation. A previous graduate student coded Shanno's memoryless algorithm from Richard Golden's "Mathematical Methods for Neural Network Analysis and Design" (1996) in neUROn++. Experiments demonstrated that training using this method resulted in speed improvements up to 2 orders of magnitude. Not only was the algorithm itself coded, however, a very elegant method of automatically switching between backpropagation and Shanno's algorithm was also encoded.

Unfortunately, because the code was so poorly documented, subsequent code modifications failed to include this module. And also, as a result, I am unable to reconstruct the meaning of the pre-processor definitions here, only to list them as they appear in neUROn++ header file.

```
#define SHANNO     OFF        // If SHANNO is ON, XENTROPY must be ON!

#define FAIL_LIMIT  2         // ensure shanno's algo working under
control
#define SWITCH_IT  60         // number of iterations for BP training
#define SHOW_SWITCH OFF       // ON to see the engine switch--hurt
performance!
#define SHANNO_MIN_DERROR 1e-4 // control of sensitivity of Shanno
switch

#define ALPHA_SHANNO 0.0001
#define BETA_SHANNO  0.999    // They are used for decide proper step
```

13.6 Feature extraction

In neUROn++, feature extraction using Wilk's GLRT was originally encoded in combinatorial fashion, and specified in the header file. Later, with virtually no additional overhead, stepwise regression was added as a set of internal modules. Because these modules call the training routines, the efficiency of the training routines do not suffer by their presence in the compiled executable, only that the executable is somewhat larger as a result. The combinatorial feature extraction definitions encoded in the header file were:

- If DEFAULT_BLACK was ON, all combinations of features were automatically extracted. For example, if features f1, f2 and f3 represented the input variables, the following feature deficient networks were statistically compared to the full network (f1, f2, f3): (f1, f2), (f1, f3), (f2, f3), (f1), (f2), and (f3).
- If INTERACT_BLACK was ON, specific combination subsets of features could be feature extracted. Using the above example, just (f1, f2), (f1, f3), (f2, f3) could be compared to the full network (f1, f2, f3).

- MAX_CLASS specified the maximum number of features to be extracted (not necessarily the number of input nodes, as more than one input nodes commonly encode a single feature.)
- MAX_CASE specified the maximum number of possible combinations of features, or 2^{MAX_CLASS}.
- D_NARY = $MAX\_CLASS + 1$.

Index